A Blueprint for Equilibrium-Based Differentiable Continuous-Variable Thermodynamic Computing

Owen Lockwood,[1,*] Jérémy Béjanin,[1] Joost Bus,[1] Christopher Chamberland,[1] Patrick Huembeli,[1,2] Frank Schäfer,[1] and Guillaume Verdon[1,3,†]

[1] *Extropic Corp., San Francisco, California 94111, USA*
[2] *Noumenal Labs Inc, Dallas, Texas 75229, USA*
[3] *Department of Applied Mathematics, University of Waterloo, Ontario N2L 3G1, Canada*

(Dated: August 18, 2026)

To help address the escalating energy and latency demands of machine-learning workloads, we introduce a blueprint for an energy-efficient and fast thermodynamic computing stack that leverages stochastic analog processes in physical hardware. In this work, we focus on energy-based thermodynamic computing where the stochastic process is well described by Langevin dynamics with tunable energy potentials. The implementation of such potentials in physical hardware enables us to generate and sample from basic parameterized energy-based models. We demonstrate how to construct and train popular classes of machine learning models based on these hardware-native energy-based models, using the framework of probabilistic graphical models. We analyze the runtime and energy consumption of different models in this thermodynamic paradigm based on theoretical considerations and numerical studies. As a preliminary experimental realization of such hardware, we present our stochastic analog superconducting circuits driven by thermal noise. Together, these results outline a path toward energy-efficient thermodynamic hardware for probabilistic machine learning.

arXiv:2607.16183v2 [cs.LG] 14 Aug 2026

I. INTRODUCTION

In an era of dramatically increasing demand for compute powering machine learning based workloads, the energy and time that digital computers consume have become a problem in need of improvement [1–4]. In this paper, we demonstrate a thermodynamic computing paradigm based on the equilibria of energy functions which undergo Langevin dynamics, and which has potential to impact the computing landscape through increased time and energy efficiency. Thermodynamic computing as a method is distinguished from other kinds of computing in that it harnesses stochastic fluctuations as a core computing resource.

In classical (i.e. digital and deterministic) and quantum computing, much effort is put into the elimination of random fluctuations from within the system. In the classical case, this is because digital computers are engineered to operate deterministically, and any stochastic effects due to random perturbations must therefore be mitigated through hardware engineering or error correction. Classical computers are designed such that the flow of electrons within their circuits can be systematically mapped to the computation of Boolean functions. The entropy and heat produced by the non-reversible operations in the computer must be pumped out of the system; otherwise, the thermal fluctuations that are produced by computations would disrupt the deterministic, binary logic of the computer. Quantum computers [5–7], similarly, require keeping stochastic fluctuations at bay because such fluctuations cause quantum states to decohere and thereby to become unusable for computation. Quantum computing systems require substantial effort (e.g. radiation shields, fridges, etc.) to minimize these fluctuations that threaten quantum coherency. Thus, classical and quantum computers, in different ways, depend on the elimination of stochastic fluctuations. In both cases, these require additional energy being used by the system.

Yet in modern algorithms, we often are forced to reintroduce stochastic fluctuations (as they rely on noisy gradients, Monte Carlo estimates, sampling-based inference, etc.), despite having engineered them out of the hardware to the best of our ability. Since modern digital methods must actively counter the thermodynamic nature of computation [8, 9] to enable deterministic computations, this results in decades of research optimized for hardware that fights the physics of nature, rather than embraces it. Fundamentally, this results in an algorithmic lock-in that we believe must break [10]. This lock-in is clearly revealed for algorithmic methods that require stochasticity and are naturally suited for physical systems, which are ubiquitous in machine learning fields (e.g. Markov chain Monte Carlo sampling, diffusion modeling, etc.). Deterministic digital hardware is used for stochastic and machine learning routines, with stochasticity introduced via software, architecture, and algorithm design, resulting in significant inefficiencies. While generating randomness itself is relatively inexpensive on modern hardware (and not the bottleneck of modern systems, as they have been designed in that way), using deterministic digital systems to represent and manipulate inherently probabilistic distributions is not. As the collapse of informal scaling laws, such as Moore's law, continues to play out, the design strategy of keeping stochasticity at bay cannot last forever. To overcome these limitations, a new computing hardware approach is required; one in which

* owen@extropic.ai
† gill@extropic.ai

the physics of the substrate aligns with the dynamics of the computation rather than fighting it.

Thermodynamic computing is a paradigm that aims to resolve this incongruity and leverage thermal fluctuations as a resource for stochastic computation [11]. This presents a way for probabilistic computing to be implemented more naturally in the stochastic dynamics of physical systems. In this approach, similar to quantum computing, measuring the final state gives rise to a distribution rather than a deterministic outcome. However, thermodynamic computing is distinctly different from quantum computing as it intentionally operates at finite temperature and does not seek to exploit quantum properties such as superposition or entanglement. The key feature of thermodynamic computing devices is that their components exist at the same scale as the relevant thermodynamic fluctuations, making them available as a resource, and that these fluctuations are used for inference and learning on-chip. Thermodynamic computing is especially interesting in the current context due to its natural connection to sampling-based methods, and in particular to the Energy-Based Models (EBMs) in machine learning research, which the dynamics of physical systems can naturally express and hence run potentially far more efficiently, on thermodynamic hardware.

In this paper, we demonstrate energy-based thermodynamic computing through a set of M thermodynamic neurons, which are composable subsystems $\mathcal{S}$, each described by Langevin dynamics [12, 13], such that the full state space is $\mathcal{X} = \mathcal{S}_1 \times \cdots \times \mathcal{S}_M$. We first outline the concepts required for this equilibrium based computing to provide a proof of principle and discuss the computational primitives. We demonstrate one can tune the equilibration times and distributions via control of the potential and temperature. We then show how this approach can be used for the kind of probabilistic inference that is required for modern machine learning. We then introduce a framework based on probabilistic graphical models that couples these subsystems together to form expressive models whose energy function depends on the node and inter-node coupling potentials, both of which can be parametrized and controlled. We discuss how these building elements can be used to realize complex applications, including large scale machine learning models. Finally, we present our preliminary findings on constructing a basic but essential building block for an analog thermodynamic computer based on superconducting hardware. Superconducting circuits allow for quadratic energy terms from inductance, as well as subsystems with double-well potentials and nonlinear couplings between them using the Josephson junction [14]. Furthermore, on-chip dissipation in superconducting circuits is naturally very low and allows information processing at energy levels many orders of magnitude closer to the Landauer limit than classical computing [15]. Additionally, the associated temperature and energy regimes allow us to harness ambient thermal fluctuations, and do not require injecting noise artificially.

II. ENERGY-BASED MODELS FROM EQUILIBRIA OF PHYSICAL SYSTEMS

A. From the Gibbs distribution to energy-based models

The core of thermodynamic computing is the Gibbs distribution (also called the Boltzmann distribution),

$$\pi_\theta(x) = \frac{1}{Z_\theta} e^{-\beta E_\theta(x)}, \tag{1}$$

describing the state space distribution of a physical system at thermal equilibrium with an energy function $E_\theta(x)$ parametrized by parameters θ, where $x \in \mathcal{X}$ is the state described by a set of system variables, $\beta = (k_B T)^{-1}$ is the inverse temperature, and $Z_\theta = \int_{\mathcal{X}} e^{-\beta E_\theta(x)} dx$ is the partition function where the integral is taken over the space $\mathcal{X}$, the set of accessible states.

The Gibbs distribution [Eq. (1)] has inspired a class of machine learning models called energy-based models (EBMs) [16–18]. These physics-inspired models employ the notion of energy to define probability distributions based on Eq. (1) by associating a scalar energy to a model variable configuration. Learning amounts to finding an energy function that associates low energy with probable data configurations and high energy with unfavorable configurations. This is achieved by finding the optimal set of parameters θ^*. EBMs are highly flexible machine learning ansätze that have potent features like composability and the ability to do conditional inference through clamping. However, EBMs are difficult to scale on traditional digital hardware due to the computational cost of sampling routines dealing with the intractable normalization constant Z_θ [19].

Due to their roots in physics, EBMs lend themselves to be implemented in analog hardware. Many ideas have been explored, such as Ising machines [20–27], probabilistic computers [28–33], quantum annealers [34–36], neuromorphics [37–40], and linear Ornstein–Uhlenbeck (OU) processors [41, 42]. However, some early "thermodynamic" proposals [41–45] fail to fully leverage thermodynamic fluctuations, as they rely on artificially injected Gaussian noise. As a result, they are unlikely to provide meaningful speedups or energy savings.

EBMs have found applications in a variety of areas [46–48] as they offer a number of advantages such as flexibility of parametrization, composability, ease of conditional sampling and more [18]. Consider a parametrized energy function $E_\theta(x, y)$ over two states $x \in \mathcal{S}_1$ and $y \in \mathcal{S}_2$, which gives rise to a joint probability density

$$\pi_\theta(x, y) = \frac{1}{Z_\theta} e^{-\beta E_\theta(x,y)}, \tag{2}$$

with normalization $Z_\theta = \int_{S_1 \times S_2} e^{-\beta E_\theta(x,y)} dx dy$. To sample from the conditional distribution $\pi_\theta(x|y) = \frac{1}{Z_\theta(y)} e^{-\beta E_\theta(x,y)}$, where $Z_\theta(y) = \int_{S_1} e^{-\beta E_\theta(x,y)} dx$, one can simply fix the value of y, and sample only the x variables

using the same method used to sample the joint distribution. This process, herein referred to as "clamping", allows for efficient inference in scenarios where partial information is known or when exploring conditional relationships in the data. Conditional sampling also reveals the ease of composing EBMs. Multiple energy functions can be composed to form more expressive distributions directly in energy space. Through simple operations combining elementary energy functions (e.g., addition, subtraction, etc.), we can express conditional relationships between variables, such as logical AND/OR/NOT [49]. While the energy function in most ML cases is dimensionless, clearly physical energy functions are not. In this work we generally operate in a standard dimensionless setting (in which physical parameters are represented through dimensionless variables or incorporated into the energy function itself).

The standard approach to training EBMs is through maximum likelihood estimation (MLE). In MLE, the goal is to maximize the likelihood of the observed data under our model. Equivalently, we can minimize the negative log-likelihood. When there are no latent states, the objective is given by

$$\mathcal{L}(\theta) = \mathbb{E}_{x\sim\pi_{\text{data}}}\left[-\log \pi_\theta(x)\right], \tag{3}$$

where π_{data} is the true data distribution, and $\mathbb{E}$ denotes the expected value.

To optimize this objective for an EBM $\pi_\theta(x)$, a variety of techniques can be employed [19]. Here, we focus on the contrastive divergence (CD) learning rule [50–53], which is less commonly used in modern machine learning compared to score-based methods [54]. This is largely due to the difficulty of sampling from high-dimensional models (a challenge which score matching sidesteps). In this work, we focus on hardware which accelerates the sampling subroutine, and thus do not have to rely on score-based methods and their approximations to the difficult to compute Hessian of energy. In CD-based learning rules, we compute the gradient via

$$\begin{aligned}\nabla_\theta \mathcal{L}(\theta) &= -\mathbb{E}_{x\sim\pi_{\text{data}}}\left[\nabla_\theta \log \pi_\theta(x)\right] \\ &= \mathbb{E}_{x\sim\pi_{\text{data}}}\left[\beta\nabla_\theta E_\theta(x)\right] - \mathbb{E}_{x\sim\pi_\theta}\left[\beta\nabla_\theta E_\theta(x)\right].\end{aligned} \tag{4}$$

In the equation above, the left term, $\mathbb{E}_{x\sim\pi_{\text{data}}}[\beta\nabla_\theta E_\theta(x)]$, is often called the "positive phase" which is based on evaluating on the data (and is often much easier to compute) and the right, $\mathbb{E}_{x\sim\pi_\theta}[\beta\nabla_\theta E_\theta(x)]$, is called the "negative phase" (and is usually the source of the vast difficulty for EBM training). In the case of latent variables, our objective reads

$$\mathcal{L}(\theta) = \mathbb{E}_{x\sim\pi_{\text{data}}}\left[-\log \int_{\mathcal{S}_2} \pi_\theta(x,y)dy\right] \tag{5}$$

We then obtain:

$$\begin{aligned}\nabla_\theta \mathcal{L}(\theta) = \mathbb{E}_{x\sim\pi_{\text{data}}}\mathbb{E}_{y\sim\pi(\cdot|x,\theta)}\left[\beta\nabla_\theta E_\theta(x,y)\right] \\ - \mathbb{E}_{(x,y)\sim\pi_\theta}\left[\beta\nabla_\theta E_\theta(x,y)\right]\end{aligned} \tag{6}$$

Equations (4) and (6) allow us to use standard stochastic gradient-based optimization methods to optimize the parameters θ of the EBM if we can approximate the expected values through sample averages.

B. Sampling based on digital Langevin dynamics

A common approach to sampling from EBMs is to simulate Langevin dynamics, which can be programmed to converge to a target distribution based on the gradient of the log probability [55–58].

The celebrated Langevin Monte Carlo method [59] represents such an approach to draw samples ($x \in \mathbb{R}^D$) from an EBM. The method is based on the concept of a gradient flow that brings the distribution $P(t)$ starting from an initial distribution $P_0 = P(t = 0)$ progressively closer to the distribution π_θ with increasing time. Indeed, samples can be obtained from the path of the Langevin diffusion process, whose states $x(t)$ have distribution $P(t)$ and evolve according to the stochastic ordinary differential equation

$$\gamma_i dx_i = -\frac{\partial U_\theta(x)}{\partial x_i}dt + \sqrt{\frac{2\gamma_i}{\beta}}dW_t^{(i)}, \tag{7}$$

where $dW_t^{(i)}$ are the increments of independent Wiener processes, i indexes the D degrees of freedom, and we typically set $\gamma_i = 1$. To draw samples numerically, we discretize this diffusion process, e.g., via the Euler-Maruyama scheme or custom higher-order schemes. This leads to a biased numerical sampling scheme (that for low-order methods can be fixed via a Hastings correction).

Despite many ongoing efforts and the strong potential benefits, sampling from EBMs [and therefore also training EBMs using the gradient estimators in Eqs. (4) and (6)] is extraordinarily difficult for high-dimensional models. Fundamentally, this difficulty comes from sampling, as in general, sampling in high-dimensional models is done via iterative locally informed proposals (such as digital Langevin dynamics), which means that the probability of overcoming barriers and valleys in this landscape can be very low, resulting in inefficient iterative sampling. Thus, it is natural to ask, rather than crudely and inefficiently simulating these dynamics numerically, can we build an analog physical device which exhibits controllable Langevin dynamics to efficiently and speedily sample? This idea represents the core backbone of our energy-based thermodynamic paradigm.

C. Sampling based on physical Langevin dynamics

Consider the dynamics of a physical system with D degrees of freedom (again, $x \in \mathbb{R}^D$) where inertia plays a significant role, and the motion occurs in the presence of

damping and random fluctuations. Such stochastic processes commonly occur in statistical mechanics when the time evolution of degrees of freedom in an energy landscape influenced by thermal noise is of interest, and are characterized by the underdamped Langevin equation, which is a stochastic ordinary differential equation of the following form

$$\begin{aligned} dx_i &= \frac{p_i}{m_i} dt \\ dp_i &= -\left(\frac{\partial U_\theta(x)}{\partial x_i} + \frac{\gamma_i}{m_i} p_i \right) dt + \sqrt{\frac{2\gamma_i}{\beta}} dW_t^{(i)}, \end{aligned} \tag{8}$$

where $x_i(t)$ and $p_i(t)$ are the position and conjugate momentum of the i-th degree of freedom at time t, respectively, $U_\theta(x)$ is the potential energy function describing the energy landscape, γ_i is the damping coefficient that quantifies resistance to motion, and m_i is the mass. The term $\sqrt{\frac{2\gamma_i}{\beta}}$ ensures that the system satisfies the fluctuation-dissipation theorem, linking damping and noise in thermal equilibrium [60]. We will later present our superconducting hardware realization, which is a tunable double-well system engineered to be operated in the thermodynamic domain where its time evolution can be modeled by underdamped Langevin equations.

The Langevin equation (8) corresponds to a Fokker-Planck equation which is a partial differential equation describing the normalized probability density of x and p at time t by $P(x,p,t)$ that can generally be written in operator form as $\frac{\partial}{\partial t} P(x,p,t) = \mathcal{L}^* P(x,p,t)$, where $\mathcal{L}^*$ is the adjoint operator [61]. In the case of underdamped Langevin dynamics, this Fokker-Planck equation is also called the Klein-Kramers equation, and the adjoint operator acts as

$$\mathcal{L}^*(\cdot) = \left[\sum_{i=1}^{D} \left(-\frac{p_i}{m_i} \frac{\partial}{\partial x_i} + \frac{\partial}{\partial p_i} \left(\frac{\partial U}{\partial x_i} + \frac{\gamma_i}{m_i} p_i \right) + \frac{\partial^2}{\partial p_i^2} \frac{\gamma_i}{\beta} \right) \right] (\cdot) \tag{9}$$

The Fokker-Planck equation provides a deterministic way to describe the time evolution of the probability distribution associated with the stochastic process. This is useful for understanding ensemble behavior rather than individual sample paths of the Langevin equations (8). For instance, we can use Eq. (9) to identify steady-state distributions, mean first-passage, and escape times.

In particular, it is well known that the Fokker-Planck equation associated with underdamped Langevin dynamics has the equilibrium solution [62]

$$\pi_\theta(x,p) = \frac{e^{-\beta E_\theta(x,p)}}{Z_\theta}, \tag{10}$$

where $\log Z_\theta$ can be interpreted as the free energy (when multiplied by $-\beta^{-1}$) and $E_\theta(x,p) = \frac{p^2}{2m} + U_\theta(x)$. Marginalization over p leads to

$$\pi_\theta(x) = \frac{e^{-\beta E_\theta(x)}}{Z_\theta}, \tag{11}$$

where $E_\theta(x) = U_\theta(x)$, and which is also the equilibrium solution of the overdamped Langevin equation (7) corresponding to the limit of Eq. (8) with small mass and large damping.

Equation (11) represents the backbone of an energy-based thermodynamic computer. Namely, the steady-state distribution of physical Langevin dynamics is a Gibbs distribution with energy potential $U_\theta(x)$. If we can design a physical system with a sufficiently controllable energy potential landscape, so that the time and energy consumption for physical equilibration and readout is smaller than for digital MCMC sampling, we expect computational benefits from such a thermodynamic computing device. The key attributes of such a system would be thermalization time (the amount of time it takes to go from the initial condition to the steady state) and thermalization energy (the amount of energy required to go from the initial condition to the steady state). This idea harkens back to early ideas of thermodynamic computation as computation driven by analog noise, such as "Brownian computers" [63].

D. Computing expected values

In addition to (conditional) sampling from energy functions, we consider averaging as another important building block to further increase the number of possible operations that we can perform with our hardware. This not only helps provide intuition into the energy functions we are working with, but since we expect thermodynamic computers to operate in extremely low energy regimes, we might also expect to gain energy advantages from approximate averaging of deterministic operations. Expected values of EBMs representing a conditional distribution give rise to parametrized deterministic functions of interest

$$f(z) = \mathbb{E}_{y \sim \pi_\theta(\cdot|z)} [y], \tag{12}$$

where we have clamped z. Such an expectation can be achieved by measuring the state y over an extended time or averaging the samples from repeated runs. This can be achieved through analog to digital converters, but this often comes with engineering downsides. We discuss later how such an operation could be achieved without digital conversions. For now, consider an estimator based on repeated runs.

For a set of N samples $\{y_i\}_{i=1}^N$, where each $y_i \in \mathbb{R}^d$, the mean estimator is computed as

$$\bar{y}_N = \frac{1}{N} \sum_{i=1}^{N} y_i, \tag{13}$$

resulting in a vector $\bar{y}_N \in \mathbb{R}^d$ that approximates the expected value $\mathbb{E}[y]$. The sample covariance matrix is

$$\hat{\Sigma} = \frac{1}{N-1} \sum_{i=1}^{N} (y_i - \bar{y}_N)(y_i - \bar{y}_N)^\top, \tag{14}$$

where $\hat{\Sigma} \in \mathbb{R}^{d\times d}$. Its diagonal elements are the sample variances of the components of y, while its off-diagonal elements are cross-covariances. For independent samples with population covariance Σ,

$$\mathrm{Cov}(\bar{y}_N) = \Sigma/N, \tag{15}$$

which can be estimated by $\hat{\Sigma}/N$.

E. Energy-time-precision trade-offs

The Monte Carlo standard error scales as $O(N^{-1/2})$. Unlike traditional floating-point computations where precision is fixed by the number format (e.g., Float32 or Float16), our approach provides (almost) continuously tunable precision (assuming we are operating with an averaging component, there are many cases as we will see later in which we do not need averaging). For computational operations whose output covariance does not depend on the encoded mean, the absolute precision is independent of the magnitude of the computed value: changing $\mathbb{E}[y]$ does not change the standard error. Relative precision is naturally not always magnitude-independent. The number of samples can be adjusted at runtime to meet the required absolute or relative error tolerance. This enables adaptive precision, with more samples allocated to critical computations and fewer samples used where lower precision is acceptable. Halving the error requires four times as many independent samples, obtained either sequentially at increased latency or through four times as many parallel samplers increased area cost. Practical implementations must carefully balance these factors based on application requirements.

Landauer's principle states that erasing a bit of information must dissipate at least $k_B T \ln(2)$ energy, where k_B is Boltzmann's constant and T is the temperature. This limit need not be in terms of energy alone but can be expressed in terms of any conserved quantity [64]. It represents the absolute minimum energy required for non-reversible computation (reversible computation can be performed in finite time with zero error and zero energy dissipation [65]). Landauer's limit is the lower limit of the energy consumed by an irreversible computation, and at room temperature is approximately 3×10^{-21} joules. This limit is many orders of magnitude lower than what is currently used in deterministic digital computing systems [66]. In digital systems, every operation is precisely defined and executed, leading to highly accurate and reproducible results. This precision, however, comes at a significant energy cost. Each operation, whether it is a simple bit flip or a complex arithmetic calculation, requires a substantial amount of energy compared to this theoretical minimum. Such energy expenditure is necessary to maintain the system's state against thermal fluctuations and to ensure the reliability of each operation. In contrast, computation based on thermodynamic principles could potentially operate with energy consumption much closer to Landauer's limit.

In practice, computation cannot simultaneously achieve high precision, high speed, and low power consumption. We can immediately identify some of the key tradeoffs of thermodynamic computation: increasing precision comes at the cost of increased power dissipation or reduced speed; faster computation requires either reduced precision or increased power dissipation; and operating with less energy necessitates either slower or less precise computation. It is also worth pointing out that the system cannot be changed arbitrarily fast because the thermalization time, τ_{therm}, will put a limit on the maximum speed. Therefore, τ_{therm} represents the fastest timescale at which our system can reach equilibrium, setting a natural speed limit for our computations. Of course, in theory we can tune the thermalization time by carefully selecting parameters in the Langevin equations [Eq. (8)]. However, any physical implementation will bound the parameter space and result in some platform-dependent thermalization time.

This discussion can be formalized using the concepts of thermodynamic [67, 68] and information-geometric speed limits [69–72]. The dissipation and work tradeoffs for training EBMs have also been discussed [73]. These limits characterize the trade-offs between entropy production and the transition time required for a system to evolve from an initial probability distribution to a final one, where the distance between the two distributions is measured in terms of the Wasserstein distance. Ito [72] provides a unified theory, referred to as geometric thermodynamics for the Fokker-Planck equation, further connecting these works to optimal transport [74, 75].

After completion of this work, we became aware of Ref. [76], which studies energy-time-accuracy trade-offs and optimal driving protocols rather than the equilibrium model constructions considered here.

Next, we discuss how one can (approximately) compute τ_{therm} for a given physical realization described by the Langevin equations (8).

F. Estimating thermalization time

For simple energy functions, we can analytically compute the characteristic timescales of equilibration via the smallest nonzero eigenvalue (the spectral gap) of the Fokker-Planck operator [Eq. (9)]. In the case of (overdamped) Gaussian potentials, the spectral gap is known to be proportional to $\frac{1}{\gamma}$. If the potential $U_\theta(x)$ has multiple wells, equilibration involves rare transitions over barriers. The equilibration time will be dominated by the Kramers escape time, which exhibits an exponential dependence on well heights [77, 78].

For more generic potentials, estimating τ_{therm} becomes more difficult [79]. We can still numerically approximate τ_{therm}, by discretizing over the states and computing the eigenvalues of the Fokker-Planck operator matrix. Another approach is to numerically integrate the Langevin equations (8) for many different noise realizations. We

then use these trajectories to track how the probability distribution $P(x,p,t)$ converges to the equilibrium Boltzmann distribution π_θ using metrics such as the total variation distance or the KL divergence between $P(x,p,t)$ and π_θ.

G. Estimating thermodynamic work

To characterize the energy dynamics of a thermodynamic computer, we consider time-dependent potentials $U_\theta(x(t),\lambda(t))$, where $\lambda(t)$ is a control parameter. Such time dependence arises, for example, when adjusting the bias of a Gaussian potential toward a target value, or when coupling modes. Changing $\lambda(t)$ performs stochastic control work on the system. The work and heat may have either sign on individual trajectories, as may the system entropy change; the second law constrains the average total entropy production of the system and environment. With work defined as being done on the system, the trajectory-level protocol work is [15, 80–82]

$$W(\tau) = \int_0^\tau \frac{\partial U_\theta(x(t),\lambda(t))}{\partial\lambda}\frac{d\lambda}{dt}dt, \tag{16}$$

which we approximate along trajectories obtained by numerically solving the Langevin equations (8). We evaluate the distribution $P(W(\tau))$ and its mean over many trajectories in our numerical studies. Evidently, $W(\tau) = 0$ when the potential is time-independent after initialization.

For work defined as being done on the system, $\Delta F = F(\tau) - F(0)$ is the minimum average work required in a reversible isothermal transformation between the equilibrium states associated with the endpoint control values. Equivalently, the maximum reversible work extractable from the system is $-\Delta F$. For an equilibrium-initialized ensemble, Jarzynski's non-equilibrium work relation [83] connects the equilibrium free energy difference to the protocol work $W(\tau)$ [Eq. (16)]

$$\Delta F = -\frac{\log \mathbb{E}\left[\exp(-\beta W(\tau))\right]}{\beta}, \tag{17}$$

where the expectation is over complete stochastic trajectories initialized from equilibrium [84]. Practically, this expectation is approximated through sampling. When the dissipated work is large or broadly distributed, a significant number of samples may be required [85], because the exponential average in Jarzynski's relation gives substantial weight to rare trajectories with low work [86].

The trajectory-level dissipated work is $W_{\mathrm{dis}} = W - \Delta F$, which can be negative for individual trajectories. Jarzynski's equality and Jensen's inequality imply the ensemble inequality

$$\langle W_{\mathrm{dis}}\rangle = \langle W\rangle - \Delta F \geq 0. \tag{18}$$

In the following, we use $P(W)$ and $\langle W\rangle$ as idealized work metrics. They are meant to be bounds on the end-to-end energy consumed by the physical system (but omit readout, calibration, refrigeration, etc.).

III. ELEMENTAL POTENTIALS

This section examines the elemental potentials that we use to generate probabilistic graphical models. These potentials are "elemental" because they form the building blocks for many more complicated functions and are representative of simple implementations of quadratic and quartic potentials of oscillators. We consider basic single-particle, coupling, and many-particle potentials U_θ. We conduct analysis of the properties of these potentials based on the approaches outlined in the previous section. Some of these building blocks resemble those presented in Ref. [87]; however, our focus is exclusively on equilibrium regimes.

A. Single-particle potentials

The two main classes of single-particle potentials we focus on are Gaussian (also called single-well or quadratic), where the force is affine in x (here, $x \in \mathbb{R}$) , and nonlinear (also called quartic or double-well) potentials.

The energy function of a Gaussian oscillator takes the form:

$$U_\theta^{\mathrm{sw}}(x) = \frac{1}{2\sigma}(x-\mu)^2, \tag{19}$$

where the parameters $\theta = (\mu,\sigma)$ allow us to tune the mean and variance of the potential. In particular, clamping can be achieved using Eq. (19), where the mean corresponds to the clamped value and the variance is minimized as much as possible. In mechanical oscillators, $\frac{1}{\sigma} = m\omega^2$ is a product of mass m and frequency squared ω^2.

The double-well potential is defined as

$$U_\theta^{\mathrm{dw}}(x) = \lambda_1 x^2(x-1)^2 - \lambda_2 x, \tag{20}$$

where the parameters $\theta = (\lambda_1,\lambda_2)$ determine the height of the energy barrier separating the two wells and the tilt of the potential, which skews the relative depths of the wells, respectively. Double-well potentials are ubiquitous in physics, especially quantum physics [88–90]. Figure 1 illustrates these operations. Note the increased probability of measuring the position x of the oscillator in one of the two wells depending on the tilt. Unlike Gaussian oscillators, thermal fluctuations play a crucial role in the double-well system, enabling transitions between the two wells. At zero temperature, such transitions occur only if the oscillator starts with sufficiently high momentum.

If several independent single-particle potentials are considered (i.e., the energy function is a sum of single-particle potentials), the spectral-gap relaxation timescale is set by the slowest independent component. The thermalization time, τ_{therm}, of Gaussian potentials is on the order of nanoseconds under reasonable superconducting hardware assumptions (100 fF, 100 pH, 20 kΩ, operating

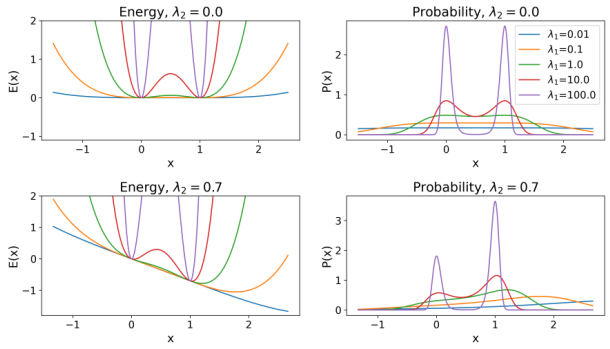


Figure 1. Comparison of energy curves and probability distributions for the double-well potential [Eq. (20)] for varying choices of λ_1 and a bias value of $\lambda_2 = 0$ and $\lambda_2 = 0.7$. The bias tilts the double-well potential towards one side and increases the probability to be in one of the two wells.

at 150 mK, see Sec. VIII). Changes in operating parameters significantly impact this time, making it highly dependent on the hardware implementation. In particular, for the double-well potential, τ_{therm} depends exponentially on λ_1 (as discussed previously in Sec. II F).

B. Coupling potentials

Naturally, non-interacting potentials can only get so far, and coupling potentials together is a next step. A simple version of a coupled potential is a Gaussian coupled to a double well,

$$U_\theta^{\text{sig}}(x,z) = \lambda_1 x^2(x-1)^2 - zx + z^2, \tag{21}$$

which closely resembles the single-particle double-well potential [Eq. (20)]. Here, z represents another oscillator. If z is sufficiently heavy (or if z is a very low variance Gaussian), we can treat it as effectively static and use the same approximations as we did for single-particle potentials where z is just treated as a constant (this will be important, as discussed in Sec. III B 1). As a pedagogical (and later relevant) example, let us consider computing the expected value for an EBM based on Eq. (21) as before and assume z is effectively static. This potential allows us to program the commonly used sigmoid activation σ_{ML}:

$$f(z) = \lim_{\lambda_1 \to \infty} \mathbb{E}_{x\sim\pi_\theta(\cdot|z)}[x] = \frac{1}{1+\exp(-z)} = \sigma_{\text{ML}}(z). \tag{22}$$

It is important to note that the well height, λ_1, in double-well potentials represents a tradeoff. To recover the deterministic sigmoid function, one may want to maximize the well height. However, recall that this comes at the cost of reduced computation speed, as the thermalization time increases exponentially with well height. As expected, a smaller λ_1 results in a shorter τ_{therm} (allowing us to draw the first and subsequent samples with less waiting time, then converging to the sampled mean via $O\left(\frac{1}{\sqrt{N}}\right)$ for N samples), but at the cost of higher error (since it is a worse approximation). These values are based on the same hardware assumptions as before (and computed with the eigenvalues of the overdamped Fokker-Planck operator). The eigenvalues were computed numerically, using SciPy [91].

We can also engineer time-dependent coupling potentials between two oscillators

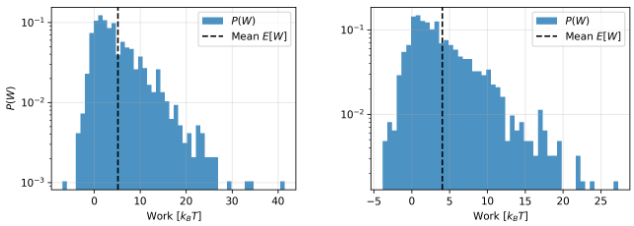
$$U_\theta^c(x,y) = \lambda(t)(x-y)^2, \tag{23}$$

where the coupling is given by

$$\lambda(t) = \lambda_c\left[\sigma_{\text{ML}}(k(t-t_{\text{on}})) - \sigma_{\text{ML}}(k(t-t_{\text{off}}))\right], \tag{24}$$

where $\theta = (\lambda_c, k, t_{\text{on}}, t_{\text{off}})$, λ_c is the coupling strength, and k determines the timescale for the coupling. This protocol switches the coupling on and (optionally) subsequently off. Suppose we have two single-particle potentials, $U_\theta(x)$ and $U_\theta(y)$, along with a coupling potential $U_\theta^c(x,y)$. To achieve an optimal energy-time trade-off, we typically aim to vary the coupling on the same timescale as the equilibration times dictated by the single-particle potentials.

Figure 2 compares the work distribution in two different cases of coupled oscillators. These figures were created via numerical integration using diffrax [92] with higher order SDE solvers [93, 94]. Notably, under the idealized model, the mean work is of order k_BT (this excludes the end-to-end energy consumed by the complete physical system). For scale, in units of k_BT, the Landauer limit is $\ln 2 \approx 0.7$. These work values are not directly comparable with reported end-to-end energies for digital computers [2, 95, 96]. Chaining low-work potentials may reduce one contribution to computation energy [97].



(a) Work distribution of two coupled Gaussians.

(b) Work distribution of a Gaussian coupled to a double-well.

Figure 2. Distributions of the work for the coupling in Eq. (23). Dashed lines indicate sample means. We use $N = 1000$ trajectories, $t \in [0, 100]$, $\lambda_c = 100$, $t_{\text{on}} = 100/3$, and $t_{\text{off}} = 200/3$. Initial positions are sampled uniformly from $[-1, 1]$ and initial momenta from $[-0.3, 0.3]$. The two distributions are similar for this parameter choice.

We can also consider other simple coupling potentials that independently couple independent Gaussian oscilla-

tors $x \in \mathbb{R}^D$ to other oscillators

$$U_\theta^{\text{add}}(x,y,z) = \nu_1 \sum_{i=1}^{D} \frac{1}{2} x_i^2 - \nu_2 \sum_{i=1}^{D} x_i(y_i + z_i), \tag{25}$$

where $y, z \in \mathbb{R}^D, \theta = (\nu_1, \nu_2)$ which results in the expected value of x being $y + z$ (for equal ν),

$$U_\theta^{\text{MVP}}(x,z) = \frac{\nu}{2} \sum_{i=1}^{D} x_i^2 - x^\top W z, \tag{26}$$

where $x \in \mathbb{R}^D$, $z \in \mathbb{R}^M$, $W \in \mathbb{R}^{D\times M}$, $\theta = (\nu, W)$, which results in $\mathbb{E}[x \mid z] = Wz/\nu$, and hence gives the matrix-vector product Wz when $\nu = 1$ (in order to practically apply this operation, we must avoid precomputing the product and just programming it as a Gaussian tilt; and still compute the specific couplings digitally, which asymptotically scales the same as matrix classical operations [98]).

These coupling potentials can enable an analog form of averaged measurement of an oscillator x, an important building block previously referenced in the discussion of methods for computing expected values [cf. Eq. (12)]. For example, an oscillator y can be coupled to a Gaussian oscillator x via Eq. (23) and made nearly static at the value of x at a time t when x and y are again decoupled. This operation allows us to read out and store the value of an oscillator x into y, which conceptually resembles digital "sample-and-hold" devices. In detail, this involves two steps. First, perform N coupling operations (with N independent oscillators $\{y^{(i)}\}_{i=1}^N$ waiting $N\tau_{\text{therm}}$ for N samples, note that we have to wait that long because the N oscillators are coupled to a single input, and thus even in parallel they will experience the same position of the input) based on the coupling potential U_θ^c [Eq. (23)]. Specifically, we can allow y_i to equilibrate to the value of x without perturbing it, then to rapidly increase the effective mass y_i once it is coupled to the oscillator being "measured". This can be achieved by carefully selecting the parameters in the Langevin Equations (8). In theory, the coupling can be done very rapidly. Second, couple these oscillators to another oscillator $y^{(N+1)}$ using the potential in Eq. (25) with $\frac{\nu_2}{\nu_1} = \frac{1}{N}$ and large ν_1. We refer to the oscillators that implement this averaged measurement operation as estimation oscillators (or relay oscillators [99, 100]).

1. *Many-particle potentials*

Hardware implementations may also enable the coupling of multiple oscillators. These many-particle potentials are a specific kind of coupled potential that couple many particles together. An intuitive example is

$$U_\theta^{\text{soft}}(x,z) = \lambda_1 \sum_{i=1}^{D} x_i^2 (x_i - 1)^2 - \sum_{i=1}^{D} x_i z_i + \lambda_2 \left(\sum_{i=1}^{D} x_i - 1 \right)^2, \tag{27}$$

where $\theta = (\lambda_1, \lambda_2)$, $x \in \mathbb{R}^D$, and $z \in \mathbb{R}^D$. This potential is built on top of the sigmoid potential given by Eq. (21). While this potential may look somewhat arbitrary, its expectation yields the softmax function, and we present it as an example of many-particle potentials (there are certainly many others one could consider) and its importance will be highlighted in Sec. IV. Although these potentials imply all-to-all connectivity, physical implementations that do not support this can reduce the degree of connectivity by introducing additional degrees of freedom as has been done in other regimes [101, 102].

Working with the potentials introduced in this section offers a set of building blocks that are potentially extremely fast, operating at or below digital clock cycles for entire thermalization processes, and highly energy-efficient, functioning at many orders of magnitude lower energy than digital computers (subject to the hardware implementation of the thermodynamic computer). However, implementing these building blocks as described in this section is not necessarily trivial.

IV. THERMODYNAMIC DEEP LEARNING AND DIFFERENTIABLE PROGRAMS VIA EXPECTED VALUES

Let us now demonstrate how carefully designed energy landscapes can implement basic computational operations through statistical averaging of their thermal equilibrium states. We have already seen an important example in this context: using Eq. (22), we demonstrated how to implement a sigmoid activation function. More generally, by working with expectations rather than individual samples, we can recover (almost) deterministic operations similar to common digital subroutines while potentially consuming less energy (recall that we can reduce the sample error $O(\frac{1}{\sqrt{N}})$ by drawing N samples which takes time $N\tau_{\text{therm}}$).

Consider a traditional multilayer perceptron (MLP) composed of stacked layers of functions given by $f(z) = \sigma_{\text{ML}}(Wz + b)$. These functions can be further decomposed into matrix-vector multiplication, vector addition, and the application of an activation function. The final layer is often a softmax operation that converts output values into probabilities.

Let us first consider the matrix-vector multiplication $x = f(z) = Wz$ where $W \in \mathbb{R}^{m\times n}$ and $z \in \mathbb{R}^n$. The energy function presented in Eq. (26) has an equilibrium distribution given by

$$\pi(x|z) = \mathcal{N}(Wz, \mathbb{I}), \tag{28}$$

for z clamped and $\nu = 1$. Thus, measuring the expected value of x through estimation oscillators allows us to approximately compute the matrix-vector product Wz.

Similarly, the energy function presented in Eq. (25) has the equilibrium distribution

$$\pi(x|y,z) = \mathcal{N}\left(\frac{\nu_2}{\nu_1}(y+z), \frac{1}{\nu_1}\mathbb{I}\right), \tag{29}$$

for y and z clamped. Measuring the expected value of x using estimation oscillators allows us to add the values of y and z for $\nu_1 = \nu_2$. It might be tempting to program a linearly tilted Gaussian and simply supply Wz as the tilt instead. However, computing Wz purely digitally eliminates any advantage of thermodynamic computation. These tilts and interactions must occur within the thermodynamic computer to preserve its benefits.

Finally, the energy function in Eq. (27) allows us to approximate a softmax operation. In particular, the parameter $\lambda_2 \to \infty$ enforces the simplex constraint $\sum_i x_i = 1$. Then, the parameter λ_1 lets us turn the continuously valued oscillator values x into one-hot vectors with $\lambda_1 \to \infty$. In this hard-constraint limit, the one-hot state e_i has energy $-z_i$, so the equilibrium probability over one-hot states is proportional to e^{z_i}. In total, we have

$$f_i(z) = \lim_{\lambda_1,\lambda_2\to\infty} \mathbb{E}_{x\sim\pi_\theta(\cdot|z)}[x_i] = \frac{\exp(z_i)}{\sum_{j=1}^{D}\exp(z_j)}. \tag{30}$$

Even with λ_1 and λ_2 as small as 10, we observe relatively good convergence to the true deterministic softmax. As we will see, low approximation quality is sufficient for training machine learning models. The approximation quality is illustrated in Figure 3. As expected, larger values of λ_1 and λ_2 lead to longer equilibration times, while smaller values of λ_1 and λ_2 result in poorer approximations of the softmax. In particular, the outputs may not be perfectly normalized but still maintain a similar structure - i.e., the relative ordering of values remains the same, though their magnitudes vary. Convergence times are highly hardware-dependent, but under superconducting hardware assumptions from before, one can achieve high speed.

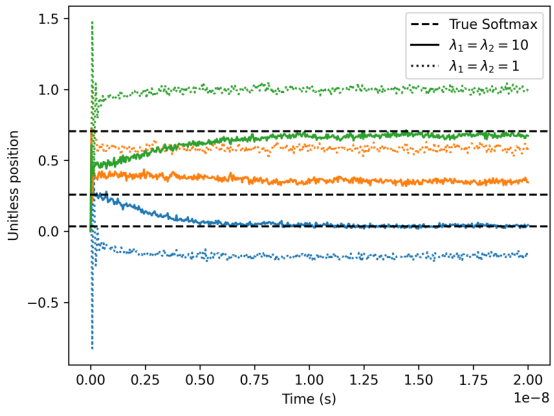


Figure 3. Convergence of the expectation of many trajectories from numerical integration of the softmax potential at different λ values.

With these potentials in mind, there is a clear approach for how to naïvely evaluate machine learning models on a thermodynamic computer. First, the initial data is clamped, then information propagates through each of these potentials, followed by an estimation oscillator, before the output is read. One could also consider a relaxed version of these models, where the blocks are directly coupled without estimation oscillators. In this approach, distributions would be propagated forward based on individual samples rather than expectations. This may lead to different training dynamics or trade-offs, which we leave as an open area for exploration.

A. Gradient computation

Forward evaluation of ML models, while an extremely important step on its own [103, 104], is only half the picture. A key attribute of our systems is that we can compute gradients as well. Differentiability [105] and backpropagation [106] are the backbone of modern ML training and both continue to be an area of active research [107–112].

At first glance, it seems like the above deterministic approximations trivially admit differentiation rules in the case where they are truly deterministic. The derivatives are known for the operations they are approximating and training could be conducted in the same way as on a digital computer (but with the outputs of the function/inputs to the next determined by the samples from the thermodynamic computer). However, this relies on the assumption that our approximations are quite close to the deterministic operations. As shown previously, this is not the case, and additional tests showed that these errors compound quickly making training of machine learning workflows based on these approximations unwieldy. Additionally, this relies on the assumption that our deterministic distribution is a known and easy to differentiate distribution. Since one is working in the energy function landscape (and is likely constrained in terms of what energy functions their substrate provides), we focus on a rule which only requires knowledge of the energy function (and its derivative). This allows for a general purpose approach that works with any energy function and doesn't rely on the strength of the deterministic approximation. As we will see later, in Sec. VII E, this helps chart a path toward components used in larger models.

Given the EBM structure of our thermodynamic computing paradigm, we can derive additional derivative rules that might be more suitable for implementation on thermodynamic hardware (without resorting to digital simulations) [113, 114]. These rules complement Eqs. (4) and (6). Consider an EBM with dimensionless energy function $E_\theta(y|x)$ and partition function $Z(x,\theta)$. The expected value of y is:

$$\mathbb{E}[y] = \int yp(y|\theta,x)dy = \int y\frac{e^{-E_\theta(y|x)}}{Z(x,\theta)}dy. \tag{31}$$

Taking the derivative of the ith component with respect

to a parameter θ_j:

$$\frac{\partial \mathbb{E}[y_i]}{\partial \theta_j} = \int y_i \frac{\partial}{\partial \theta_j}\left(\frac{e^{-E_\theta(y|x)}}{Z(x,\theta)}\right) dy. \tag{32}$$

Using the quotient rule, we have:

$$\frac{\partial}{\partial \theta_j}\left(\frac{e^{-E_\theta(y|x)}}{Z(x,\theta)}\right) = - p(y|\theta,x)\left(\frac{\partial E_\theta(y|x)}{\partial \theta_j} + \frac{1}{Z(x,\theta)}\frac{\partial Z(x,\theta)}{\partial \theta_j}\right). \tag{33}$$

Substituting Eq. (33) back into Eq. (32) leads to

$$\begin{aligned}\frac{\partial \mathbb{E}[y_i]}{\partial \theta_j} &= -\int y_i p(y|\theta,x)\left(\frac{\partial E_\theta(y|x)}{\partial \theta_j} - \mathbb{E}\left[\frac{\partial E_\theta(y|x)}{\partial \theta_j}\right]\right) dy \\ &= -\left(\mathbb{E}\left[y_i \frac{\partial E_\theta(y|x)}{\partial \theta_j}\right] - \mathbb{E}[y_i]\mathbb{E}\left[\frac{\partial E_\theta(y|x)}{\partial \theta_j}\right]\right).\end{aligned} \tag{34}$$

The Jacobian is thus given by the negative cross covariance:

$$\frac{\partial \mathbb{E}[y]}{\partial \theta} = -\mathrm{Cov}\left(y, \frac{\partial E_\theta(y|x)}{\partial \theta}\right). \tag{35}$$

This form connects parameter gradients to statistical correlations in the system, providing a method for computing derivatives through sampling. Similarly, we find

$$\frac{\partial \mathbb{E}[y]}{\partial x} = -\mathrm{Cov}\left(y, \frac{\partial E_\theta(y|x)}{\partial x}\right). \tag{36}$$

The Jacobians in Eqs. (35) and (36) can be used to forward- and back-propagate derivative information. This chain rule applies to a computation graph in which the mean of one block is clamped as the deterministic input of the next block. For instance, in a computation graph $x \to \mathbb{E}[y^{(1)}] \to \cdots \to \mathbb{E}[y^{(l)}] \to \mathbb{E}[y^{(l+1)}] \to \ldots \mathbb{E}[y^{(L)}]$, with $y^{(L)} \in \mathbb{R}$, we can compute the gradient $\nabla_{\theta^{(l)}} \mathbb{E}[y^{(L)}]$ with respect to parameters $\theta^{(l)}$ in the lth EBM as

$$\nabla_{\theta^{(l)}} \mathbb{E}[y^{(L)}] = \frac{\partial \mathbb{E}[y^{(L)}]}{\partial \mathbb{E}[y^{(L-1)}]} \cdots \frac{\partial \mathbb{E}[y^{(l+1)}]}{\partial \mathbb{E}[y^{(l)}]} \frac{\partial \mathbb{E}[y^{(l)}]}{\partial \theta^{(l)}}. \tag{37}$$

The involved covariance matrices must be estimated from samples, which has important implications on error propagation. The potentials themselves are approximating deterministic functions, and we also have sample error on the forward pass, and this also introduces sample error on the backwards pass. As we discuss in the next sections, these error rates are manageable with a reasonable number of samples. The covariance-form estimator of Eqs. (35) and (36) bears some similarity to equilibrium propagation [115–117], which computes gradients in physical networks from the difference between clamped and free phases. These ideas were also explored in Ref. [118] and connects the covariance to the Hessian. Additionally, there may even be advantages to training under noisy/probabilistic/sample-based conditions, as these are used in a variety of traditional machine learning methods from reinforcement learning [119–121] to uncertainty quantification [122–124] to improving learning dynamics [125–128].

V. MODULARITY & SCALABILITY VIA THERMODYNAMIC HYPERGRAPHICAL MODELS

A. Representing complex distributions

Having outlined the construction of basic energy models that allow tuning of the equilibrium distribution and enable sample generation, these models can now be used as primitives for more complex operations. Just as we created interacting nonlinear potentials, it is also possible to couple many different oscillators together. The exact degree of coupling and connectivity, however, may depend on the specific hardware implementation.

Specifically, one could leverage the composability features of EBMs to construct probabilistic graphical models (PGMs) [129]. We can see that the previously discussed class of averaged models are effectively a special case of these graphical models, specifically directed acyclic graphs (DAGs). PGMs provide a powerful framework for representing complex probability distributions through graph structure. Using the factor graph formalism, we can represent both directed and undirected graphical models [130], where models are expressed as bipartite graphs $\mathcal{G} = (\mathcal{V}_v, \mathcal{V}_f, \mathcal{E})$. In this representation, variable nodes correspond to the random variables in our distribution while factor nodes represent functions of these variables. This framework allows us to decompose joint distributions into factors of EBMs with $f(i)$ being the set of factors (energy functions) that are connected to variable node i, and $n(a)$ denoting the set of variables incident to factor a. The Markov blanket of variable i is the set of neighboring variables that share at least one factor with i, $\mathrm{MB}(i) = \left(\bigcup_{a \in f(i)} n(a)\right)$ excluding i. This results in representation via

$$\pi(x_{\mathcal{V}_v}) = \frac{1}{Z}\prod_{a \in \mathcal{V}_f} \exp\left[-E_a(x_{n(a)})\right] \tag{38}$$

$$= \frac{1}{Z}\exp\left[-\sum_{a \in \mathcal{V}_f} E_a(x_{n(a)})\right], \tag{39}$$

where E_a represents the energy function of factor a (note that we drop the parameters θ merely for simplicity of notation). The normalization constant Z ensures the distribution integrates to 1:

$$Z = \int \prod_{a \in \mathcal{V}_f} \exp\left[-E_a(x_{n(a)})\right] \prod_{i \in \mathcal{V}_v} dx_i. \tag{40}$$

A classic example that illustrates both single-variable factors and pairwise interactions is the Ising model [131]. In this model, each variable represents a spin $s^{(i)} \in \{-1, +1\}$, and the probability distribution takes the form:

$$\pi(s^{(1)}, \ldots, s^{(N)}) = \frac{1}{Z} \exp\left(\sum_i h_i s^{(i)} + \sum_{\langle i,j \rangle} J_{ij} s^{(i)} s^{(j)} \right), \tag{41}$$

where h_i represents the local magnetic field at site i (single-variable factors) and J_{ij} represents the coupling between spins (pairwise factors). This naturally decomposes into factors:

$$E_i(s^{(i)}) = -h_i s^{(i)} \quad \text{and} \quad E_{ij}(s^{(i)}, s^{(j)}) = -J_{ij} s^{(i)} s^{(j)}. \tag{42}$$

Since factors can connect any subset of variables, including single variables, this also notably results in a hypergraph (a graph in which a single edge can connect more than two nodes) structure. Factors connected to single variables often represent prior distributions or local constraints, while factors connecting multiple variables capture interactions.

This factor graph framework unifies various types of graphical models. Directed graphical models correspond to products of local conditional probability factors, while undirected models correspond to products of positive compatibility functions or energy factors, as exemplified by the Ising model discussed above. The distinction lies in the semantics and normalization of the factors, not simply in whether a factor is symmetric. The hypergraph structure emerges naturally when factors connect arbitrary sets of variables, enabling the representation of complex higher-order dependencies.

This modular structure makes the model's dependency patterns explicit while allowing us to build complex systems from simpler components. The resulting framework not only facilitates efficient inference algorithms that exploit the graph structure [132] but also provides natural pathways for parallel computation and scalable learning. This not only enables us to increase our model's expressivity using basic building blocks, but also might be necessary for the next era of machine learning [133], as composition and modularity play an increasingly important role in the next era of scaling.

B. Sampling

Having outlined how to construct factor graphs by combining smaller EBMs as factor building blocks, we can move on to a central challenge in probabilistic modeling, which is computing quantities of interest from our models after they have been specified or trained. This process, known as model evaluation and prediction, can take several forms depending on our goals. We might wish to compute marginal distributions $\pi(x^{(i)})$ for individual variables, useful for understanding the behavior of specific components of our system. Or we might need conditional probabilities $\pi(x^{(i)}|x^{(j)})$ to make predictions about some variables given observations of others. Often, we seek maximum probability configurations, which represent the most probable states of our system. These inference tasks become intractable to compute exactly as our systems grow, due to the exponential growth of the state space and the high-dimensional integrals or sums involved. The factor graph structure, however, suggests natural approaches for approximate inference that exploit the locality of interactions in our models.

Gibbs sampling emerges naturally from the factor graph structure as a method to generate samples from the joint distribution [134]. The key insight is that, while sampling from the full joint distribution is difficult, sampling a single variable conditioned on all others is often straightforward due to the local structure of factors. This leads to an iterative algorithm that updates variables one at a time, exploring the probability space through a carefully constructed random walk. Although there are regimes in which Gibbs sampling can be advantageous to do digitally [135], within a single connected thermodynamic chip, the advantage of using something like Gibbs sampling is an open question. It may depend on the ease of hardware measurement, clamping, etc., in addition to whether Gibbs sampling offers any advantages in cases where you have access to a chip that efficiently computes Langevin dynamics over the entire graph (in general, the theoretical effectiveness of Gibbs sampling outside of specific types of graphs remains an open question [136]). However, in cases of multi-chip models, Gibbs sampling can be a valuable tool, as these chips may have purely digital interconnects.

The conditional distribution for each variable takes a particularly simple form in factor graphs:

$$\pi(x^{(i)}|x^{(/i)}) \propto \exp\left(- \sum_{a \in f(i)} E_a(x_{n(a)}) \right), \tag{43}$$

where $x^{(/i)}$ denotes all variables except for $x^{(i)}$ and the sum runs only over factor nodes a that include variable i. This local computation makes each update step efficient, as we need only consider factors directly connected to the variable being updated. Traditional Gibbs sampling is entirely made of iterative loops of clamping and sampling, although there are many variants that could potentially be applied to multichip models [137–139]. In order to draw a sample $x_i \sim \pi(\cdot|x^{(/i)})$ digitally, any sampling method can be used (either exact conditionals if they are known, or a standard MCMC algorithm, in which case this is known as Metropolis-within-Gibbs). For thermodynamic chips, this allows for a hierarchy of sampling, as each variable could in itself be representing a distribution, enabling Gibbs sampling within each Gibbs step one level up. The modular sampling is naturally available at the hardware design level, but can also be controlled through software (since the level at which clamping/sampling is done can be programmed). Although we focus on Gibbs sampling in this work, there are other

methods of prediction on graphical models, for example, our deterministic program DAGs are conceptually similar to mean field belief propagation [140].

C. Training

Training factor graphs extends the methods we discussed in the previous section to handle multiple interacting EBMs. A key difference is that our models often include both observed (visible) and unobserved (hidden) variables. Hidden variables are powerful tools that can capture underlying structure in our data, mediate long-range dependencies, or represent latent factors in our system [141]. This is equivalent to training according to Eq. (6). Computing these expectations now requires marginalizing over hidden variables. The training algorithm typically alternates between inference (or state estimation) and parameter updates.

A key advantage of factor graphs is their modular nature, which one could exploit during training. For large graphs, one might first train individual factors or small subgraphs independently, then fine-tune the full model jointly, and finally, iteratively scale through curriculum learning. The initialization of both parameters and hidden variables can significantly impact training success. Where possible, we can initialize factors based on domain knowledge or pre-train them on simpler tasks. For hidden variables, initialization strategies often depend on their intended role in the model, they might be initialized randomly, or based on prior knowledge about the structure we wish to capture.

VI. TOWARD ON-CHIP THERMODYNAMIC SELF-LEARNING

Thus far, we have treated the model parameters θ as digitally stored values updated by an external optimizer, while the thermodynamic substrate supplies samples for fixed θ (cf. Sec. IV A). A natural next step is to ask whether learning itself can be carried out on-chip by promoting parameters to physical degrees of freedom that evolve stochastically, alongside the visible and latent variables. This section sketches one concrete route: a timescale-separated Langevin system in which the fast variables (x, z) rapidly equilibrate for quasi-static θ, while θ drifts under an effective force that encodes the learning signal.

This idea of inducing learning via adding terms to the Hamiltonian is not dissimilar from recent work on self-training of Ising models [142]. This connection between self-learning and thermodynamic computation also runs much deeper than practical algorithms [143]. Other approaches have been proposed for local learning of out-of-equilibrium models [144], and self-learning machines have been proposed for physical devices [145].

The following Langevin equations, cast in dimensionless form (see Appendix A), describe the dynamics of the system in which there are three sets of equations for each of the $\{x, z, \theta\}$:

$$\begin{aligned} d\{x,z,\theta\}_i &= p_{\{x,z,\theta\}i}dt \\ dp_{\{x,z,\theta\}i} &= \Big[-\partial_{\{x,z,\theta\}}U(x,z,\theta) - \zeta_i p_{\{x,z,\theta\}i}\Big]dt \\ &\quad + \sqrt{2\zeta_i}\xi_i(t)dt. \end{aligned} \tag{44}$$

In the above equations, the x_j are the visible variables, the z_ℓ are the latent variables, and the θ_i are the parameters. It is assumed that the dynamics of the parameters is much slower than those of the variables. In this timescale-separated scenario, a Born-Oppenheimer-like approximation holds, in which effective equations of motion for the (slow) parameter can be derived [62, Sect. 8.3]. To lowest order in the ratio of timescales, they are given by

$$\begin{aligned} d\theta_i &= p_i dt, \\ dp_i &= \left(F_i^{\mathrm{BO}}(\theta) - \zeta_i p_i\right)dt + \sqrt{2\zeta_i}\xi_i(t)dt, \end{aligned} \tag{45}$$

In these effective equations of motion, there appears the effective Born-Oppenheimer force

$$F_i^{\mathrm{BO}}(\theta) := -\int \mathrm{d}^{d_z}z\ \mathrm{d}^{d_x}x\ \frac{\mathrm{e}^{-U(x,z,\theta)}}{Z(\theta)}\frac{\partial U(x,z,\theta)}{\partial\theta_i}, \tag{46}$$

where $Z(\theta) := \int \mathrm{d}^{d_z}z\ \mathrm{d}^{d_x}x\ \mathrm{e}^{-U(x,z,\theta)}$. This expression for $F^{\mathrm{BO}}(\theta)$ affords a clear intuitive picture: the force on the parameter degrees of freedom θ_i is the (negative) gradient of the potential, averaged over the instantaneous equilibrium for the fast variables x_j and z_ℓ. Note that $F^{\mathrm{BO}}(\theta)$ comprises both real forces acting on the parameter degrees of freedom θ_i, and additional contributions owing to the couplings with x_j and z_ℓ.

The key observation is that $F^{\mathrm{BO}}(\theta)$ provides a means of wielding the kinematics of a physical system to measure the terms in the learning rule of Eq. (6), provided one uses a sufficiently short measurement window during which θ remains approximately constant. Indeed, if both x_j and z_ℓ are unclamped, then $F^{\mathrm{BO}}(\theta)$ yields $-\mathbb{E}_{(x,z)\sim\pi_\theta}\big[\nabla_\theta E_\theta(x,z)\big]$, whereas if x_j is clamped to a specific datum sample x, $F^{\mathrm{BO}}(\theta)$ yields $-\mathbb{E}_{z\sim\pi(\cdot|x,\theta)}\big[\nabla_\theta E_\theta(x,z)\big]$. Thus if we can construct such a system, we have a means for estimating the gradients on the chip allowing for more efficient, self-learning like training.

We now expand on how one may go about this. As noted above, we limit ourselves to a short time window during which the (slow) parameters θ do not have time to change significantly. With this assumption, we are free to linearize the dependence of $F^{\mathrm{BO}}(\theta)$ on θ, which yields equations of motion that are exactly solvable (specifically, those of an Ornstein–Uhlenbeck process) [62, Sect. 3.2]

and hold at early times:

$$d\theta_i = p_i dt,$$
$$dp_i = \left(F_i^0 + \sum_j J_{ij}^0 \big(\theta_j - \theta_j^0\big) - \zeta_i p_i \right) dt + \sqrt{2\zeta_i}\xi_i(t) dt, \tag{47}$$

where $F^0 := F^{\mathrm{BO}}(\theta^0)$ and $J_{ij}^0 := \frac{\partial F_i(\theta)}{\partial \theta_j}\Big|_{\theta^0}$ is the Jacobian of the force at the initial position. It is F^0, the effective force at the initial position θ^0, that we will estimate from the early-time dynamics. See Appendix B for more details.

We now describe two more integrated hardware architectures that might yield a path of self-learning, a natural gradient descent (NGD) approach, and a more fully analog NGD approach via relay oscillators. We consider these approaches more ambitious and speculative, but could yield ideas for future research directions.

Natural gradient descent (NGD) [146] preconditions the gradient with the inverse of the information metric, accounting for the curvature of the statistical manifold and yielding updates that are invariant to parameterization. For energy based models, we can define the Fisher information matrix via

$$\mathcal{I}_{jk}^{\mathrm{FIM}}(\theta) = \mathbb{E}_{x\sim\pi_\theta}\big[\partial_j E_\theta(x)\, \partial_k E_\theta(x)\big] - \mathbb{E}_{x\sim\pi_\theta}\big[\partial_j E_\theta(x)\big]\, \mathbb{E}_{y\sim\pi_\theta}\big[\partial_k E_\theta(y)\big], \tag{48}$$

where $\partial_j \equiv \partial/\partial\theta_j$ and $E_\theta(x)$ is the energy function. The NGD update rule is then

$$\theta_{t+1} = \theta_t - \eta_t \big(\mathcal{I}^{\mathrm{FIM}}\big)^+ \nabla_\theta \mathcal{L}(\theta_t), \tag{49}$$

where η_t is the learning rate and $(\cdot)^+$ denotes the pseudoinverse.

The key point, building on the Born-Oppenheimer approximation, is that both the gradient vector and all entries of the Fisher matrix can be estimated from rapid momentum measurements on the synapse oscillators of the thermodynamic chip [147]. Specifically, repeated momentum readouts at intervals δt yield time-averaged estimates of the energy gradients $\partial_j E_\theta$; pairwise products of these measurements then provide the second-moment terms needed for $\mathcal{I}_{jk}^{\mathrm{FIM}}$. In this hybrid protocol, the matrix inversion of Eq. (49) is performed on an external device, the gradient information is merely read off.

A more ambitious approach is to avoid the digital readout and inversion by encoding components directly in auxiliary oscillators. We can imagine a protocol such as the following [148, 149]. First, a set of estimator oscillators is coupled to the energy-gradient observables $\partial_j E_\theta$. Intermediate estimation oscillators sample the energy-gradient products at different times, and a final estimation oscillator, coupled to all intermediates, approximates the expectation value $\mathbb{E}[\partial_j E_\theta\, \partial_k E_\theta]$. Separate estimation oscillators store the individual gradient expectations $\mathbb{E}[\partial_j E_\theta]$. A third layer of estimation oscillators, coupled to the outputs of the previous two layers, encodes the full (j,k) component of the Fisher metric via Eq. (48). A block-diagonal approximation to the Fisher matrix can also be constructed layer by layer, potentially reducing the number of estimation oscillators. If one coupled parameter oscillators to the estimation network through a potential, whose equilibrium dynamics shift each parameter by an amount proportional to the corresponding entry of $(\mathcal{I}^{\mathrm{FIM}})^+ \nabla_\theta \mathcal{L}$, this would yield the NGD update of Eq. (49) without any digital matrix inversion.

VII. EXAMPLE ARCHITECTURES & APPLICATIONS

Before getting into a physical implementation of the basic building blocks, we highlight a few example applied demonstrations of what energy-based thermodynamic chips could be used for. We also present some further arguments on the time/energy advantages that could exist. We provide a collection of theoretical outlines and numerical simulations to highlight the scope of applicability of these blocks we have detailed above. While we detail the hardware implementation of one block in Sec. VIII, the focus in this section is on what one could achieve with many of these hardware blocks. These workflows will vary and depend on hardware, but show the generalizability and promise of DAGs and PGMs as expressive models. In this section, to illustrate the enormous scope of possible graphical models that can be assembled using these building blocks, we demonstrate a broad class of more complex tools that can rely on the simple primitives that we demonstrated above, namely: Gaussian PGMs, Gaussian mixture models, Hidden Markov models, continuous Ising models, and a thermodynamic version of the popular transformer model [150]. The following numerics were built on JAX [151, 152] and equinox [153].

A. Gaussian probabilistic graphical model

Even when using only simple Gaussian potentials, powerful models can be constructed. Gaussian PGMs originally gained prominence due to the popularity of algorithms such as Gaussian belief propagation [154–156], and recently we have seen their resurgence at the intersection of Gaussian graphical models and neural networks [157–161]. Gaussian graphical models are among the simplest examples of PGMs possible within our framework. One can simply couple together Gaussian oscillators (with any degree of connectivity that their hardware supports, adding latent variables to make up for the sparsity), and train them using the standard visible or hidden contrastive losses presented above.

A toy example of using Gaussian PGMs as a generative model is shown in Figure 4. In this example, we show slices of a three-dimensional Gaussian (since any

Gaussian graphical model can be represented as a single Gaussian), represented using three one-dimensional Gaussian nodes, trained using the standard CD-learning rules [cf. Eq. (4)]. Here, we use Gibbs sampling to draw samples from the Gaussian PGM.

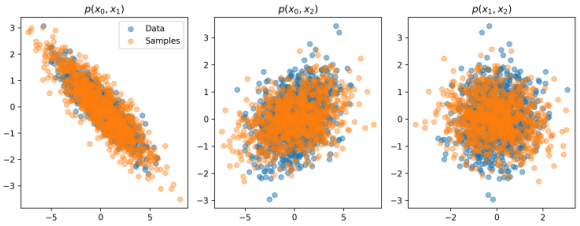


Figure 4. Samples vs. data for a trained Gaussian PGM.

B. Gaussian mixture model

As a simple but highly relevant example, we construct a Gaussian mixture model (GMM) based on a factor graph using the building blocks outlined above. GMMs are a powerful class of models [162, 163] which rely on two key components: a mixing distribution, in this case a categorical distribution, and a component distribution, which is a Gaussian of the form of Eq. (19). Here, we use a hard one-hot selector $z \in \{e_1, \dots, e_K\}$ rather than a finite-λ continuous relaxation; this mixture model therefore does not require any estimation oscillators.

Given a dataset $X = \{x^{(1)}, \dots, x^{(N)}\}$, where each $x^{(i)}$ is a D-dimensional vector, our goal is to model the probability distribution that generated this data using a mixture of Gaussian distributions with K components

$$\pi_\theta(x) = \sum_{k=1}^{K} \mathrm{Cat}_\eta(z = k)\mathcal{N}(x|\mu^{(k)}, \Sigma^{(k)}), \tag{50}$$

with logits $w = (w_1, \dots, w_K)$ and mixture weights $\eta = \mathrm{softmax}(w)$, so that $\sum_k \eta_k = 1$, means $\mu = (\mu^{(1)}, \dots, \mu^{(K)})$, and covariances $\Sigma = (\Sigma^{(1)}, \dots, \Sigma^{(K)})$. The (marginal) log-likelihood of the data given the model parameters $\theta = (w, \mu, \Sigma)$ is

$$\mathcal{L}(\theta) = \sum_{i=1}^{N} \log \pi_\theta(x^{(i)}). \tag{51}$$

We can use gradient ascent to maximize the log-likelihood. The GMM can be represented as a factor graph with K Gaussian EBM nodes defined through Gaussian potentials

$$U_G(x; \mu_k, \Sigma_k) = \frac{1}{2}(x - \mu_k)^\intercal \Sigma_k^{-1}(x - \mu_k) + \frac{1}{2}\log\det(2\pi\Sigma_k), \tag{52}$$

representing the clusters, and one categorical EBM node representing the mixture component selection, for which we use the softmax potential [Eq. (27)]. Finally, in the hard one-hot limit, these components are connected via a factor energy:

$$U_\theta(x, z) = \lambda_1 \sum_{i=1}^{K} z_i^2 (z_i - 1)^2 - \sum_{i=1}^{K} w_i z_i + \lambda_2 \left(\sum_{i=1}^{K} z_i - 1\right)^2 + \sum_{k=1}^{K} z_k U_G(x; \mu^{(k)}, \Sigma^{(k)}). \tag{53}$$

As a motivating illustration of this graphical structure, we show a conventional digital GMM with 20 Gaussians trained using stochastic gradient descent on the negative log-likelihood of the full MNIST dataset in Figure 5. The GMM numerics were computed using distreqx [164].

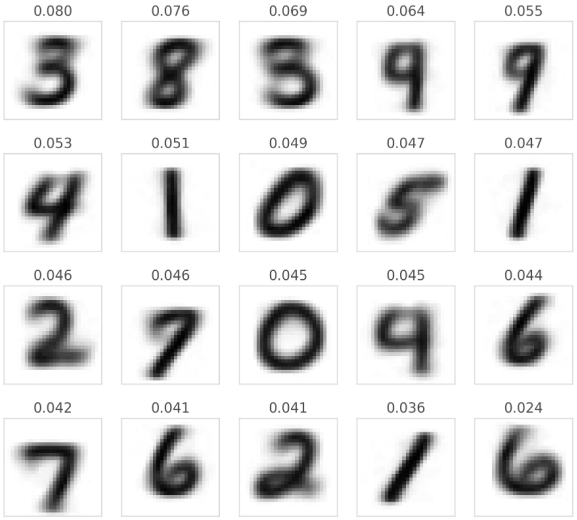


Figure 5. Components of a (purely digital) Gaussian Mixture Model trained on MNIST. The title of each figure represents the associated mixing probability of each component.

C. Hidden Markov model

Hidden Markov models (HMMs) are a specific type of state space model characterized by discrete hidden states and observations that are conditionally independent given these states. HMMs were a backbone of pre-neural network machine learning [165–168] and have seen more recent usage interfacing with neural networks [169–174]. In general, a state space model consists of latent state variables, observation variables, a transition model, and an observation model. HMMs have a finite set of hidden states, while the observations may be discrete or continuous. In contrast to general state space models, the observation model of the HMM $\pi(x^{(t)}|z^{(t)})$ is not conditioned on previous observations $x^{(t-1)}$. The state variables $z^{(t)}$ represent the hidden state of the system at time t, while the observation variables $x^{(t)}$ are the observed data at time t. The transition model $\pi(z^{(t)}|z^{(t-1)})$ defines the probabilistic rules for transitioning from one

state to another, and the observation model $\pi(x^{(t)}|z^{(t)})$ defines the probabilistic relationship between the state and the observation. The probability of the initial hidden state is defined by a prior $\pi(z^{(1)})$. Building an HMM with EBMs involves defining energy functions for the transition and observation model, and the prior. The latent state variables $z^{(t)}$ and the observation variables $x^{(t)}$ are modeled using energy functions that capture the dynamics and the relationship between states and observations.

The transition model in an HMM framework defines the probability of transitioning from one state to another. In the context of EBMs, we define an energy function $E_\theta^{\text{trans}}(z^{(t)}, z^{(t-1)})$ for the transition between states $z^{(t-1)}$ and $z^{(t)}$. The transition probability can then be expressed as:

$$\pi_\theta(z^{(t)}|z^{(t-1)}) = \frac{e^{-E_\theta^{\text{trans}}(z^{(t)}, z^{(t-1)})}}{Z_{\text{trans}}(z^{(t-1)})} \tag{54}$$

where $Z_{\text{trans}}(z^{(t-1)}) = \sum_{z^{(t)}} e^{-E_\theta^{\text{trans}}(z^{(t)}, z^{(t-1)})}$ is the partition function and θ are the parameters of the transition model.

The observation model defines the probability of observing $x^{(t)}$ given the state $z^{(t)}$. Using EBMs, we define an energy function $E_\phi^{\text{obs}}(x^{(t)}, z^{(t)})$ that captures the relationship between the observations and the state. The observation probability is given by:

$$\pi_\phi(x^{(t)}|z^{(t)}) = \frac{e^{-E_\phi^{\text{obs}}(x^{(t)}, z^{(t)})}}{Z_{\text{obs}}(z^{(t)})}$$

where $Z_{\text{obs}}(z^{(t)}) = \sum_{x^{(t)}} e^{-E_\phi^{\text{obs}}(x^{(t)}, z^{(t)})}$ is the partition function and ϕ are the parameters of the observation model. For continuous emission models, sums over x are replaced by integrals.

The joint probability of the state sequence $z^{(1:T)}$ and the observation sequence $x^{(1:T)}$ in a state space model using EBMs can be expressed as:

$$\pi(z^{(1:T)}, x^{(1:T)}) = \pi_\alpha(z^{(1)}) \prod_{t=2}^{T} \pi_\theta(z^{(t)}|z^{(t-1)}) \prod_{t=1}^{T} \pi_\phi(x^{(t)}|z^{(t)}) \tag{55}$$

where the initial state prior is:

$$\pi_\alpha(z^{(1)}) = \frac{e^{-E_\alpha^{\text{init}}(z^{(1)})}}{Z_{\text{init}}}. \tag{56}$$

As for any other EBM model, the optimization objective for the HMM is the likelihood of the observed data: $\log p(x^{(1:T)}) = \log \sum_{z^{(1:T)}} \pi(x^{(1:T)}, z^{(1:T)})$.

Because of the conditional probabilities of the transition and observation model, the gradient is different from the hidden and fully visible PGMs that have been discussed previously in this section. The gradient with respect to the parameters θ of the transition model reads:

$$\sum_{x^{(1:T)} \in \mathcal{B}} \nabla_\theta \log \pi(x^{(1:T)}) = \sum_{x^{(1:T)} \in \mathcal{B}} \mathbb{E}_{z^{(1:T)} \sim \pi(\cdot|x^{(1:T)})} \Bigg[\sum_{t=2}^{T} \Big(-\nabla_\theta E_\theta(z^{(t)}, z^{(t-1)}) + \mathbb{E}_{z'^{(t)} \sim \pi_\theta(\cdot|z^{(t-1)})} \left[\nabla_\theta E_\theta(z'^{(t)}, z^{(t-1)}) \right] \Big) \Bigg]. \tag{57}$$

Hence, for a batch of data trajectories $\mathcal{B}$, the hidden trajectories are sampled $z^{(1:T)}$ and the positive phase is evaluated with these trajectories. For the negative phase we also sample $z'^{(t)} \sim \pi(\cdot|z^{(t-1)})$, where we use the shorthand $z'^{(1:T)} \sim \pi(\cdot|z^{(1:T)})$ that implies that the conditional $z^{(1:T)}$ comes from the previously sampled trajectory.

The gradient for ϕ reads:

$$\sum_{x^{(1:T)} \in \mathcal{B}} \nabla_\phi \log \pi(x^{(1:T)}) = \sum_{x^{(1:T)} \in \mathcal{B}} \mathbb{E}_{z^{(1:T)} \sim \pi(\cdot|x^{(1:T)})} \Bigg[\sum_{t=1}^{T} \Big(-\nabla_\phi E_\phi(x^{(t)}, z^{(t)}) + \mathbb{E}_{x'^{(t)} \sim \pi_\phi(\cdot|z^{(t)})} \left[\nabla_\phi E_\phi(x'^{(t)}, z^{(t)}) \right] \Big) \Bigg]. \tag{58}$$

The full derivation for these gradient expressions is available in Appendix C. As an illustrative example, we show a fully digital Gaussian HMM where $\pi_\theta(x^{(t)}|z^{(t)}) = \mathcal{N}(x^{(t)} \mid \mu_{z^{(t)}}, \Sigma_{z^{(t)}})$ on a simple synthetic time-series dataset. We fit the model using gradient descent and plot the predicted states in Figure 6. This example was created using dynamax [175]. This example highlights the potential of PGMs for time-series data.

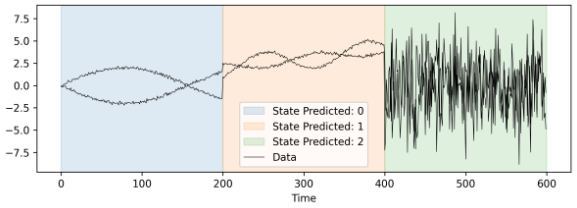


Figure 6. Training a 2D HMM on time series data. Highlighted regions indicate predicted states.

D. Ising model

As a further demonstration of the power of continuous EBMs as building blocks for PGMs, we train a continuous Ising model. The Ising model is a universal computing class which has been studied extensively by the physics community [131, 176] and is of great interest in machine learning and optimization as well [23, 177–179].

As previously discussed, Ising machines (hardware implementations of Ising models) are a prominent example of analog hardware. There are many extensions and variations that increase the trainability [20, 26, 180, 181] and expressivity of Ising models, often relying on using hidden/latent variables [182–184].

Although physical implementations, such as superconductors, lack all to all connectivity, this is common in analog hardware platforms and existing work seeks to address the costs and benefits of these sparse Ising models [20, 32, 102, 179]. We can relax the ubiquitous discrete model to be continuous, i.e., we can replace the binary variables in Eq. (41). In this case, the model has an energy function $E_\theta(s) = \sum_i \left(s_i^2(s_i-1)^2 - hs_i\right) + \sum_{\langle i,j\rangle} J_{ij}s_is_j$, with s_i being a continuous value. This is analogous to an Ising model but with continuous double-wells at each state. We train a fully visible version of this continuous Ising model on a simple bars and stripes dataset. Here, each node of the Ising model represents one corner of the four squares of the bar/strip grid. We use the standard CD-based training methods [Eq. (4) and Eq. (6)] with our continuously relaxed Ising machines and show the convergence of the weight matrix to the ground truth in Figure 7. Although continuous Ising models have been studied previously to some extent [185–187], there is substantial room for further investigation.

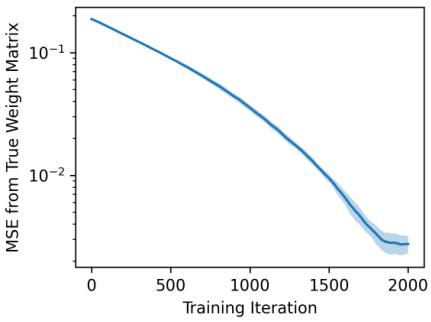


Figure 7. Training a continuous Ising machine. Mean-squared error between the parameter matrix and the optimal set of parameters θ^* as a function of the training iterations.

E. Thermoformer: Thermodynamic Transformer

With matrix-vector products (and, by extension, matrix multiplications, as they consist of multiple matrix-vector products), vector additions, softmax, and a nonlinear activation, we have a suite of core machine learning components. A common activation used in modern models is the swish activation [188], $\text{swish}(x) = x\sigma_{\text{ML}}(x)$, which can trivially be assembled from our previously outlined building blocks (multiplication and sigmoid) [189]. These are sufficient for examples such as MLPs or CNNs. However, if we wish to scale to state-of-the-art architectures, a few additional tools are required. The primary focus here is on one of the most common and widely used architectures in modern machine learning: the transformer-based [150] decoder model [190].

The main component of the transformer architecture that we have not yet specified is the layer norm [191, 192]. Layer norm computes $f(x) = \frac{x-\mathbb{E}[x]}{\sqrt{\text{Var}[x]+\varepsilon}}$. This computation relies on the following: the components of addition (which we have specified), computing the mean (which we have also specified), as well as computing the variance and computing the division, which we specify presently.

To compute the variance, one could construct the potential to compute the variance of z oscillators (where $z \in \mathbb{R}^N$ and $y \in \mathbb{R}$) via

$$U_\theta(x,y,z) = \frac{1}{2}y^2 + \frac{1}{2}\sum_i (x_i - (z_i-\mu))^2 - \frac{1}{N-1}y\sum_i x_i^2, \tag{59}$$

where x_i are intermediate, auxiliary oscillators. In the deterministic limit, the stationary point of this potential is at $y = \text{Var}[z]$. Then, using the potential $U(y,z) = y^3(z+\varepsilon) - 3y$, which (assuming a 0 initialization, and $x > 0$, and $z = \text{Var}[x]$) has its stationary point at $y = \frac{1}{\sqrt{\text{Var}[x]+\varepsilon}}$. This layer norm potential is merely an example and it requires additional stable nonlinear primitives, and is not meant to be an equilibrium realization of the function as this particular example is unbounded. There is an abundance of architectural design and implementation choices that could be used instead. For example, thermodynamic computers may be more amenable to transformers with other local nonlinearities [193–195], since we have already seen how to do sigmoid, instead of layer norm.

With these components in hand, we can now see how to assemble a transformer in thermodynamic hardware (what we call a "thermoformer"). Embed the tokens with position embeddings digitally, clamp oscillators to these values, go through the attention mechanism (where attention $= \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$ where Q, K, V are the results of inputs multiplied by different weight matrices, in the following we assume all to all connectivity for the softmax, but that is not strictly required and with extra degrees of freedom the same output can be obtained [101] with an increase in the time and energy), add a residual connection [196] and norm, and stack these layers to a certain depth. A visualization of the decoder module is shown in Figure 8. There are an endless number of transformer variants [197–199], which could also be implemented if one constructed approximate potentials, but here we focus on the original. Naturally, this approach also scales to transformer variants such as mixture of experts based models [200–203].

With these potentials constructed, we can now approximate the time and energy values of interest for a superconducting circuit implementation. Using the same approach as before, we can scale these approximations due to the sequential nature of the DAG. Specifically, since each operation must first thermalize, then have

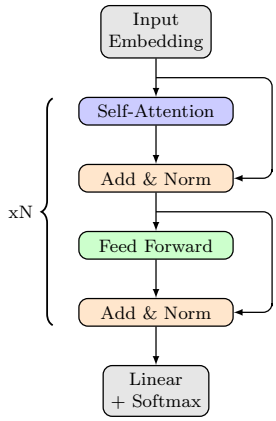


Figure 8. Diagram of transformer-decoder model.

its expectation computed before the next operation can thermalize, the total time can be obtained by summing the thermalization times of each block. Similarly, the total energy can be computed by summing the energy contributions from each step. With values that are not unreasonable in current day superconducting hardware (100 fF, 100 pH, 100 Ω, operating at 50 mK, with λ parameters of 1), the resulting chip projections suggest potentially favorable tokens per joule under the assumed superconducting device parameters. Decoder models of varying depth are displayed in Figure 9, as well as some of the prominent semi-open-weight models, Llama 3 [204]. Note that, regardless of whether the samples are in parallel or sequential they require additional (linearly scaling) energy.

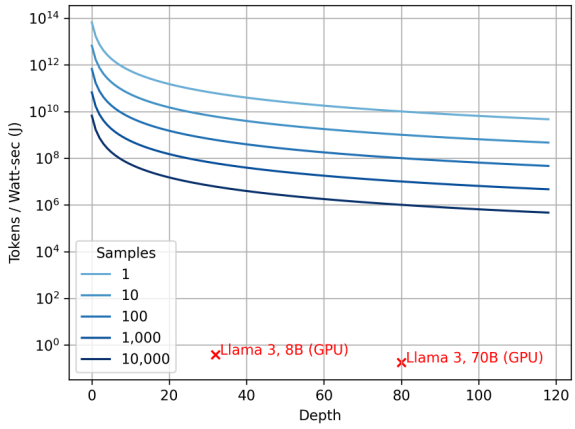


Figure 9. Tokens per Joule for varying decoder depths and varying numbers of samples used. Llama models on H100 GPUs included for reference. Projected chip results exclude cryogenic cooling, control electronics, and calibration overhead.

However, this is an incomplete picture. Because our thermoformer is conducting sample-based approximations to potentials that approximate the deterministic operations, the performance is not the same as the deterministic digital case. We show the performance of thermoformers with varying numbers of samples used to approximate the means compared to a digital decoder, tested on a toy dataset of sequential numbers. The results are shown in Figure 10. As we can see, within a reasonable number of samples, on this toy task, the simulated model approaches the digital baseline as the number of samples increases. This approach uses the gradient estimation rules outlined in Sec. IV A. Additionally, the thermodynamic operations that we used have parameters that make their approximations quite loose (that is to say, even when having converged to the true expected value of the distributions, that value is not necessarily all that similar to the true deterministic value). For example, the λ values of the softmax computation [Eq. (27)], are small (= 1.0), which results in values that have some qualitative similarities to the output distribution (e.g., the biggest logit will be the biggest probability), but are not necessarily quantitatively similar (e.g., the outputs may not be strictly positive, or sum to one). How the required number of samples scales with model depth, sequence length, and task difficulty is an open question that we do not address here. Even in this approximate regime, we are able to achieve good performance on this small problem.

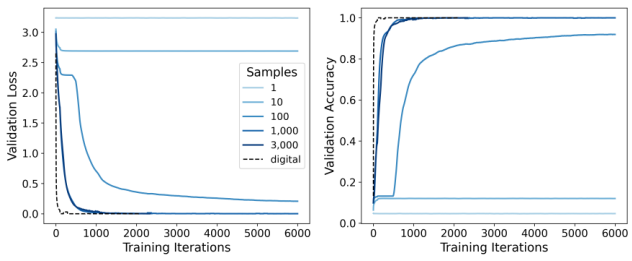


Figure 10. Training of a thermoformer with different numbers of samples used to compute the mean of each operation, compared to a digital decoder.

VIII. IMPLEMENTATION OF BUILDING BLOCKS IN A SUPERCONDUCTING SUBSTRATE

To demonstrate the power of the theory described above, we designed and fabricated a superconducting chip that implements the core double-well building block, a thermodynamic neuron [205]. This is the first step towards building a fully scaled up energy-based thermodynamic computer. In addition, we present an achievable set of next steps and future experiments to help the broader research community advance this technology, noting that our implementation of this design represents but the first few steps in this new direction, and that much innovative research and development is still

required. The experiment described in this section characterizes the simplest elemental block of the framework, a single tunable double-well potential. This fundamental thermodynamic building block can be seen as a continuous version of the discrete probabilistic-bit (and in fact is currently limited to binary readout). By taking the two wells as the two binary positions, a double well thermodynamic neuron can be converted into a probabilistic-bit [20, 206–209].

Superconducting circuits offer a fundamental nonlinear element, the Josephson junction [210, 211]. This non-dissipative element exhibits a sinusoidal relationship between the voltage and current across its two ports. When used in conjunction with the quadratic term from an inductance, this allows us to engineer a system with a double-well potential. The Josephson junction also allows us to engineer nonlinear coupling between nodes. Furthermore, on-chip dissipation in superconducting circuits is naturally very low and allows information processing much closer to the Landauer limit [15]. Additionally, the associated temperature and energy regimes allow us to harness ambient thermal fluctuations, and as such, we do not require the injection of noise algorithmically. Networks with thousands of similar building blocks coupled together have already been implemented for the purpose of quantum annealing [36].

The thermodynamic neuron is a tunable nonlinear system. It is akin to a superconducting flux qubit or fluxmon [212–215], which are normally used for quantum computing, but it is engineered to operate in the thermodynamic domain, where its dynamics are thermally activated and its time evolution can be modeled by the Langevin equation [Eq. (8)] [15, 78, 216, 217]. We focus on its double-well regime and demonstrate the ability to tune the equilibration times through the control of the potential and temperature.

A. Device theory and design: From thermodynamic neurons to thermodynamic chips

The device is composed of three thermodynamic neurons. More precisely, the device is a superconducting aluminum-on-silicon chip hosting three uncoupled thermodynamic neurons, each having two dedicated control lines. Each thermodynamic neuron is inductively coupled to a coplanar-waveguide (CPW) $\lambda/4$ readout resonator, which is in turn coupled capacitively to a shared CPW transmission line. The resonance frequencies of the three readout resonators are designed to be between 11.7 and 11.9 GHz spaced 100 MHz apart for frequency addressability. The chip contains two thermodynamic neuron design variants: one coplanar and one lumped element. In this article, we focus on the coplanar variant.

The thermodynamic neuron comprises a CPW section similar to that of a $\lambda/4$ resonator but shunted to ground at the open end by a dc-SQUID loop formed by a pair of Josephson junctions. The distributed inductance and capacitance of the CPW line together with the dc-SQUID give rise to a system that can be modeled with an equivalent lumped capacitance C, inductance L, and Josephson junction critical current I_c, connected in parallel, as shown in the circuit diagram of Figure 11. The classical Hamiltonian of the thermodynamic neuron circuit is given by

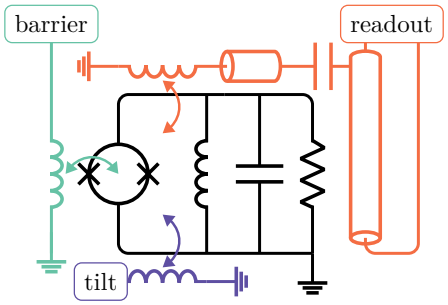


Figure 11. Circuit diagram of the thermodynamic neuron with a nonlinear tunable double-well potential made with a dc SQUID, an inductor, and a capacitor in parallel. The dissipation and noise of the system is modeled with a shunt resistance. The inductively coupled barrier and tilt lines allow control over the Josephson energy and linear bias. Mutual inductances are indicated by an arc between an inductor and the relevant loop. The system is measured by probing the transmission line that is coupled to a readout resonator whose resonance is dependent on the state of the thermodynamic neuron through the dispersive shift.

$$H = \frac{1}{2C}q^2 + \underbrace{\frac{1}{2L}(\phi - \phi_{\text{tilt}})^2 - E_J(\phi_{\text{bar}})\cos\left(2\pi\frac{\phi}{\Phi_0}\right)}_{U(\phi)}, \tag{60}$$

where the charge q and flux ϕ are conjugate variables. Physically, the flux ϕ corresponds to the voltage across the open end of the $\lambda/4$ section, which is therefore also the voltage across the dc-SQUID. The charge q is accumulated between the open end and the ground plane. $\Phi_0 \equiv h/2e$ is the magnetic flux quantum, $E_J(\phi_{\text{bar}}) \equiv E_{J_{\max}}\cos(\pi\phi_{\text{bar}}/\Phi_0)$ is the effective Josephson energy, with $E_{J_{\max}} = \Phi_0 I_c/2\pi$, and ϕ_{bar} and ϕ_{tilt} are control fluxes. These control fluxes can be tuned via two independent flux lines and affect the cosine and quadratic terms, thereby determining the shape of the potential $U(\phi)$. One line is coupled inductively to the dc-SQUID loop and controls the effective Josephson energy, via control flux ϕ_{bar}. The cosine term can form a barrier within the quadratic potential thus creating a double-well system when E_J is negative, depending on the values of $E_{J_{\max}}$ and L. The second flux line is coupled inductively to the current mode of the CPW segment and shifts the position of the cosine relative to the parabola by ϕ_{tilt}. In the double-well regime, this results in one well being tilted lower than the other. In practice, voltages applied to the control lines are not directly proportional to the control fluxes ϕ_{bar} and ϕ_{tilt} due to crosstalk. This is

covered in Section VIII B. The Hamiltonian in Eq. (60) represents a simplified model that neglects the junctions' asymmetry and intrinsic capacitances and inductances. For the analysis of our experimental data, we used a more complex model that provides a better fit to reality, see Section H.

At low temperature, loss, and noise, the circuit is expected to behave according to quantum mechanics. In fact, the thermodynamic neuron circuit is identical to the fluxmon of Ref. [212] where it is used as a flux qubit for quantum annealing applications. In the quantum regime, the classical Hamiltonian of Eq. (60) may be quantized by converting variables q and ϕ to quantum operators $\hat{q}$ and $\hat{\phi}$, according to the commutation relation between $\hat{q}$ and $\hat{\phi}$. Thus, there exist two regimes to describe the thermodynamic neuron: the quantum and thermal regimes. As noise increases as a function of the temperature of the system, the transition between the two regimes is generally characterized by a temperature T_{cross}. When the thermodynamic neuron potential is biased to a double-well shape, this temperature denotes the point at which crossings from one well to the other stop being dominated by macroscopic resonant quantum tunneling (MRT), and instead become caused by thermal activation. It is expressed by [218–223],

$$T_{\text{cross}} = \frac{\hbar\omega_{\text{b}}}{2\pi k_{\text{B}}}, \tag{61}$$

where $C\omega_{\text{b}}^2 = -U''(0)$ is the curvature of the potential at the barrier peak ϕ_{b} and we assume a symmetric well, meaning $\phi_{\text{tilt}} = 0$.

In the thermal regime, we can describe the thermodynamic neuron via its Langevin equation of motion [Eq. (8)]. In order to account for thermal fluctuations, we model loss and noise by adding a resistor R in parallel to the rest of the circuit [224], see Figure 11. This parallel resistance represents a normal current channel (non-superconducting), and is therefore dissipative. Since a high resistance will lead to a smaller current through the resistor, a high R leads to lower loss. The noise caused by this resistance is Johnson-Nyquist noise, represented with independent Wiener processes $dW_t^{(i)}$ [78, 216, 217].

Although the circuit is made out of superconducting metal, at non-dc frequencies the resistance is still nonzero. In addition, quasiparticles, interactions with lossy dielectrics and coupling to the environment all contribute to losses. In particular, as temperature rises, the increasing quasiparticle density leads to higher losses [225]. Thus it is important to keep in mind that R is temperature-dependent, which affects both the resistance (loss) and noise.

Taking the flux through the inductor to be the degree of freedom of this system, the Langevin equation of motion (with $p \equiv q$) is

$$\begin{aligned} d\phi &= \frac{p}{C}dt \\ dp &= -\left(\frac{\partial U(\phi)}{\partial\phi} + \frac{1}{RC}\,p\right)dt + \sqrt{\frac{2}{R\beta}}\,dW_t. \end{aligned} \tag{62}$$

In analogy to a mechanical system, we can view the system as a flux "particle" of mass C with position ϕ and momentum q moving in a potential landscape $U(\phi)$ that is parametrized by ϕ_{bar} and ϕ_{tilt}. The inverse resistance $1/R$ corresponds to friction slowing down the particle.

As mentioned above, the potential $U(\phi)$ has two main configurations: single- or double-well. The single well case occurs when either the cosine and quadratic terms have their minima aligned, for which the well will look somewhat quartic, or when the cosine term is much smaller than the quadratic term, in which case the well is near harmonic. For the double-well, the cosine needs to be flipped such that the maximum is aligned with the quadratic potential minimum. If, additionally, $E_J > E_L$, this will create a barrier between two wells bounded on the left and right by the harmonic potential. Just like a simple SQUID or Josephson junction, the thermodynamic neuron can be described by a plasma frequency ω_{p}. This is the frequency of small oscillations at the bottom of a well, whether in the single or double-well configuration. The plasma frequency approximately corresponds to the frequency between the two lowest lying quantum eigenstates, and is the frequency we refer to as the thermodynamic neuron frequency throughout the text

In this work, we are interested in the thermally activated dynamics, in which noise acts as the only driving force on the system. In particular, we want to measure the rate Γ at which a flux particle in one of the wells "escapes" to the other well by crossing over the barrier (classically or thermally). Given a thermodynamic neuron with population initially starting all in one well, we expect that the population will decrease exponentially in time to some equilibrium value. The defining timescale for this behavior is the thermalization time τ_{therm}, which is simply the inverse of the escape rate out of one well. The escape rate for the double-well system can be described by an Arrhenius-type law [77, 78, 216, 226]

$$\Gamma = a_t \frac{\omega_{\text{p}}}{2\pi} \exp\left(-\frac{\Delta U}{E_{\text{esc}}}\right), \tag{63}$$

where ω_{p} is the plasma frequency, a_t is a factor dependent on the damping coefficient $\eta = 1/RC$, ΔU is the height of the potential barrier, T is the sample temperature, and E_{esc} is the escape energy. For thermally activated dynamics $E_{\text{esc}} = k_{\text{B}}T$. There are three relevant damping regimes for a_t, namely (1) heavy damping, (2) small damping and (3) extreme underdamping [78]. For these regimes, we have $a_t = |\omega_{\text{b}}|/\eta$, $a_t = 1$, and $a_t \propto \eta\sqrt{C\Delta U}/k_{\text{B}}T$, respectively. Deriving Eq. (63) generally involves assuming $k_{\text{B}}T \ll \Delta U$, but the expression has

been shown to be effective even when $k_B T \sim \Delta U$ [216]. Note that below the crossover temperature, barrier crossings are caused by MRT, and we therefore expect that E_{esc} will not vary as a function of temperature.

As the system dynamics rely on superconductivity, we are restricted to work below the critical temperature of aluminum ($T_{crit} = 1.2\,\mathrm{K}$). Furthermore, it should be noted that the population of quasiparticles in aluminum increases significantly beyond 160 mK [225], increasing losses by lowering the parallel resistance R and affecting the thermodynamic neuron dynamics.

B. Experimental methods

The main experiment presented in this article is the measurement of the escape energy E_{esc} of the thermodynamic neuron as a function of temperature. For this purpose, we need to

1. determine the various circuit parameters, i.e., C, L, and I_c,
2. be able to initialize the system in a known state and measure whether the flux particle is in the left or right well, and
3. control the potential shape and temperature in order to measure relaxation curves, that is, to measure the population of the left and right wells as a function of time and barrier height.

In order to properly characterize the system, and in particular, to fit the parameters in the Hamiltonian of Eq. (60), we must know the relation between the voltages that we apply from our room temperature electronics and the effective flux being coupled into the SQUID barrier loop ϕ_{bar} and the main tilt loop ϕ_{tilt}. The approach we take to this calibration is detailed in Appendix D.

1. *Initialization and readout*

Once the flux axes have been calibrated, we can obtain a rough idea of what parameters to use in order to initialize and read out the state. The procedure consists in first lowering the barrier with a ϕ_{bar} pulse such that the thermodynamic neuron population will equilibrate to a single well. We then tilt the well with a ϕ_{tilt} pulse to the left or right before raising the barrier with a final ϕ_{bar} pulse. Assuming that equilibration time is sufficient and that the barrier is not raised "too fast", this sequence deterministically confines the thermodynamic neuron population within the left or right well.

Because we observe an anti-crossing with the resonator, we know that for control fluxes within the relaxation operating regime (see the triangle in Figure 15) the thermodynamic neuron has a frequency lower than that of the resonator. From simulations, we can estimate the rough location where we expect the thermodynamic neuron to be in the single-well regime, and therefore where we may initialize the population. Near the middle of the triangle is a good point to start. Once a basic readout has been achieved, we can sweep over the various parameters involved to optimize it.

Readout is performed dispersively [212], allowing for fast discrete measurements of the flux degree of freedom of the thermodynamic neuron in the left (L) or right (R) well. It is achieved by inductively coupling the thermodynamic neuron to a CPW readout resonator that in turn is capacitively coupled to a $50\,\Omega$ transmission line. Occupancy of either well induces a dispersive shift to the readout resonator frequency. These two dispersive shifts depend on the control biases and differ when an asymmetry is introduced through the tilt control; for more details see Appendix E. This tilt asymmetry is crucial, since at zero tilt, the two well states have exactly the same frequency, thereby causing an identical frequency shift to the resonator. By probing the transmission line at an appropriate frequency, the flux can be inferred from the state-dependent response when this frequency difference is resolvable.

Note that the measurement procedure interrupts the flux trajectory, and is therefore "destructive," as it involves adjusting the thermodynamic neuron potential to an asymmetric double-well with a sufficient dispersive shift difference. More details on readout can be found in Section I. The result of each individual measurement is a binary variable, 1 or 0, corresponding to the left or right well. To obtain population statistics, we repeat the experimental sequence 1000 times and average the result, which we denote P.

Once the correct flux amplitudes, pulse timings, and measurement frequencies have been determined, we may characterize the initialization and readout performance with a measurement of the "s-curve". For this experiment, we perform normal state initialization and readout, but we sweep over the initial tilt pulse amplitude. For a double-well system, the expected value of the flux particle population with respect to ϕ_{tilt} is sigmoidal. At large initialization tilts, we expect to measure the full population P in the left ($P = 1$) or right ($P = 0$) well. Near zero tilt, the population should be approximately $P = 0.5$. We use the following empirical relationship to characterize this

$$P(\phi_{tilt}) = \frac{1}{2} - \frac{1}{\pi} \arctan\left(\frac{\pi}{2} \frac{\phi_{tilt}}{w}\right) \tag{64}$$

where w is the s-curve sigmoid width parameter. This width largely depends on two effects: how quickly the barrier is raised and the thermalization temperature. In principle, raising the barrier slowly with even a small amount of tilt would confine the full population to the corresponding well, leading to a narrow width. However, for very slow barrier ramps, thermal noise will lead the population to continuously re-equilibrate, leading to a larger width. Figure 12 presents the result of the s-curve characterization. In Figure 12 (a), we show a few

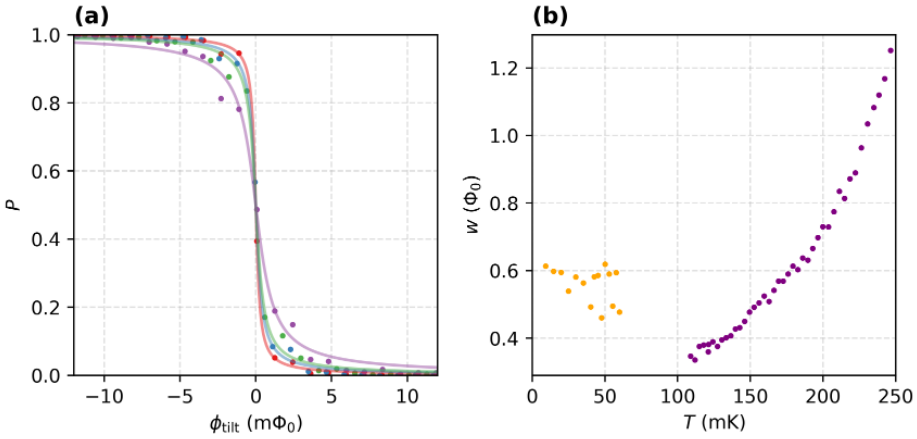


Figure 12. Sigmoidal relationship between the expectation value of position and a linear force for the double-well per Eq. (64). (a) Example tilt calibration. (b) Temperature-dependence of the width parameter w. The orange markers designate data taken below 60 mK and the violet markers designate data taken above 100 mK.

selected s-curves measured at various temperatures, but with constant barrier ramp time. For all temperature points, the state initialization and readout is perfect for tilts $\phi_{\text{tilt}} > 10\text{m}\Phi_0$. In Figure 12 (b), we directly plot the fitted value of the width, which increases with temperature.

In our experiments, the ϕ_{bar} and ϕ_{tilt} pulses used to initialize the population last between 3 and 5 µs, and we use a linear ramp of 26.67 ns to raise the barrier for readout.

2. *Calculation of escape energy*

Since each of the relaxation curves that we measure is an exponential decay, it is possible to fit them to a simple exponential model with three parameters:

$$P(t) = A\exp(-t/\tau) + B \tag{65}$$

where $\tau = 1/\Gamma$ is the relaxation time constant, and A and B are used as fit parameters and are near 0.5.

In order to calculate the effective escape energy E_{esc}, we must fit the relaxation rates obtained in the experiment according to the model of Eq. (63). We rely on our knowledge of the system to estimate the plasma frequency ω_{p} and the barrier height ΔU as a function of ϕ_{bar}. We may compute the natural logarithm of $\Gamma/\frac{\omega_{\text{p}}}{2\pi}$ and fit it linearly against ΔU. Then, the inverse negative slope of this line is the activation energy, and the intercept is the natural logarithm of a_t:

$$\ln\left(\Gamma/\frac{\omega_{\text{p}}}{2\pi}\right) = \ln(a_t) - \frac{\Delta U}{E_{\text{esc}}}. \tag{66}$$

C. Experimental results

Figure 13 presents the main results of this section. In Figure 13 (a), we plot population P data from the relaxation experiment for a few select barrier settings and temperatures. With a perfect initialization in the left or right well, at $t = 0$ the thermodynamic neuron is always found in the well it was prepared in. Then depending on the barrier height and thermal energy, the average population decays exponentially until it stabilizes at approximately $P = 0.5$. We fit each relaxation curve and extract the time constant τ. Over all measured barriers and temperatures, we notice that the fit quality as reported via the residuals sum of squares (RSS) is better (i.e., lower RSS) for larger barriers and lower temperatures.

We obtain τ for both left and right initializations. We then average them and inverse the result to obtain the relaxation rate Γ, which we plot in Figure 13 (b) as a function of estimated barrier height ΔU for each temperature. When plotted with a logarithmic scale and for constant temperature, the rates trace a straight line, decreasing exponentially as a function of ΔU. This allows us to fit each Γ curve according to the exponential model of Eq. (63), as described in Section VIII B 2. Then, the negative inverse slope gives the escape energy E_{esc}, which, when well within the thermal regime, should be equal to $k_{\text{B}}T$.

We plot the final result, E_{esc} as a function of T, in Figure 13. Due to the control issue potentially caused by contamination (see Section J), we do not have data points between 60 and 100 mK. At low temperature, the escape energy appears to be constant. Above 100 mK, E_{esc} increases linearly until approximately 165 to 200 mK, at which point the escape energy stops increasing and instead appears to taper off.

The first feature we discuss is the escape energy at low temperature. This energy corresponds to the quan-

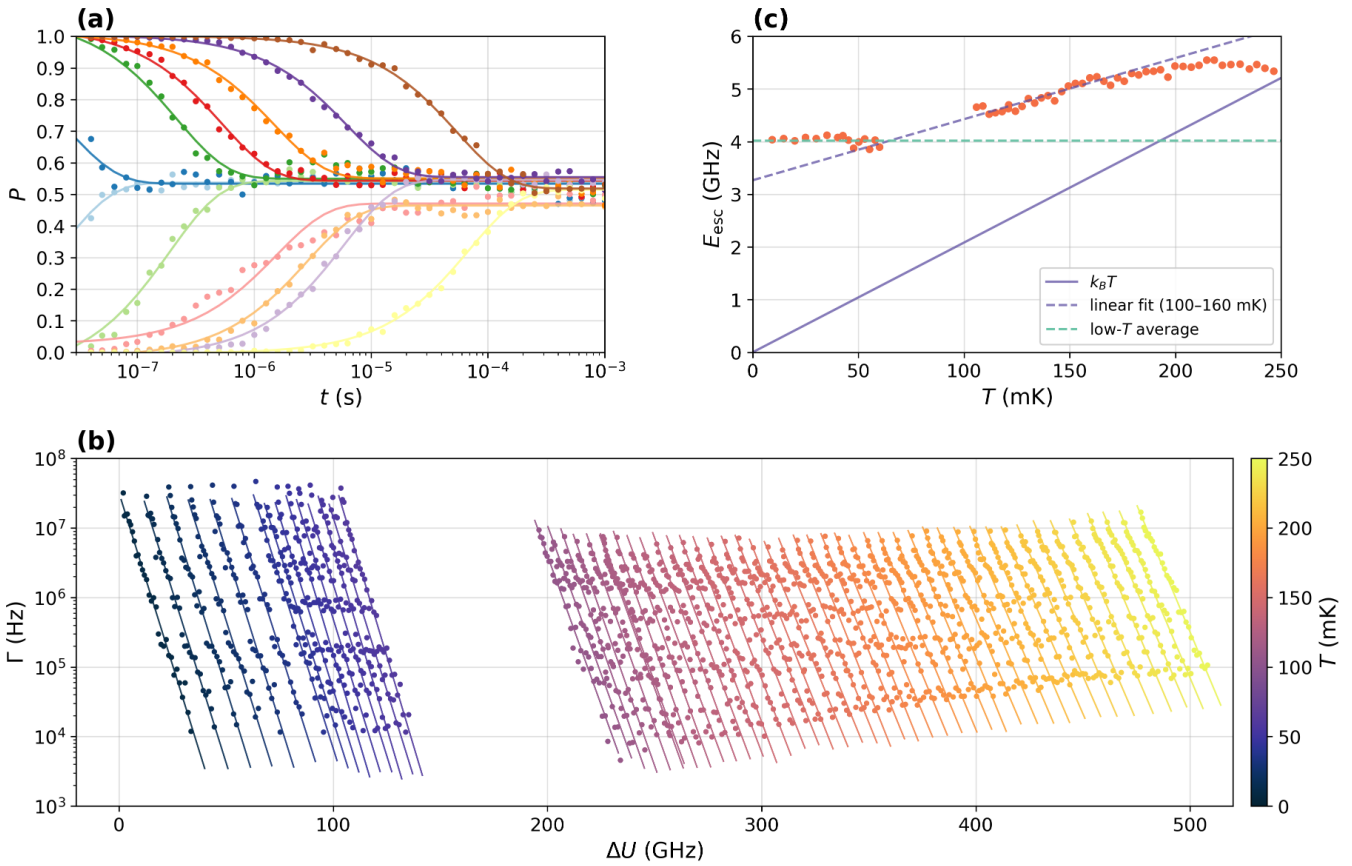


Figure 13. Results of the relaxation experiments. (a) Trajectories of the left well population P versus time for a selection of barrier heights and temperatures. The lines are fits to the exponential model described in Section VIII B 2; as expected, the data are well described by an exponential. (b) Relaxation rate Γ versus barrier height ΔU for all measured temperatures. The straight lines are linear fits in logarithmic scale and are plotted for ΔU from 1 to 40 GHz. For clarity, each curve is offset horizontally according to the temperature difference between them, with a spacing of 2 GHz mK^{-1} relative to the first curve at 9 mK. (c) Escape energies E_{esc} versus temperature. The energies are extracted from the inverse negative slope of a linear fit to $\ln(\Gamma/\omega_p)$ based on Eq. (63). The data is shown in orange. The violet solid line indicates $E_{esc} = k_B T$, and the associated dashed line is a fit to the data between 100 and 160 mK, ignoring higher temperatures because of potential quasiparticle effects. The mint dashed line shows the average E_{esc} in the low-temperature regime where the data is flat.

tum tunneling through the barrier, and is therefore not temperature-dependent; this is what we observe. However, the value of the energy in the quantum regime should correspond to $k_B T_{cross}$, and can therefore be estimated from our knowledge of the potential $U(\phi)$. Given the circuit parameters determined in Section H, the frequency corresponding to the inverse curvature at the barrier top ω_b varies between 6 and 10 GHz for the range of ϕ_{bar} used. According to Eq. (61), this leads to T_{cross} between 46 and 76 mK. If we average the points below 60 mK, we obtain an escape energy of $E_{esc}/h = 4.02$ GHz, which corresponds to a temperature of 193 mK. Thus, the escape energy we measure is larger than expected by a factor of almost 3. Here, we emphasize that both the ratio between the measured E_{esc} and ω_b depend quite strongly on the Hamiltonian parameters of the thermodynamic neuron, that is, C, L, and I_c. Given the very large uncertainty on these parameters, it is not unlikely that wrong values could be causing this difference.

Above the crossover temperature, the escape energy should start increasing and eventually should be equal to $k_B T$. While we cannot observe the section where this transition occurs, due to the control issue of Section J, we can see that the escape energy does increase linearly at a rate of approximately 12 GHz K^{-1}. The rate of increase, however, is smaller than the expected $k_B/h = 20.8$ GHz K^{-1}. Additionally, E_{esc} gradually stops increasing above 165 mK. There are multiple potential reasons for this. One obvious candidate is the quasiparticle population in the superconducting metal. Indeed, for aluminum, the equilibrium quasiparticle population starts to increase above 160 mK, causing a large increase in loss. This effect has been measured in superconducting qubits and resonators. The model for the escape energy increasing as $k_B T$ is for constant loss (i.e., friction, $1/R$). Here, as T increases, $1/R$ also increases. In addition,

as mentioned in Section VIII A, there are multiple functional forms for a_t in Eq. (63) that depend on damping. It is possible that the increased loss changes the value of a_t as temperature increases, something we have not accounted for.

A related possibility is that our "crossover" region may be particularly large, and that therefore, we never fully enter the thermal regime before quasiparticle loss takes over. Indeed, because we measure relaxation curves that include very small barriers, the range of ω_b and ω_p covered by each ϕ_{bar} sweep is quite large, increasing by a factor 2 to 3 over the sweep. Thus, there is not a single ω_b or ω_p that conclusively characterize the transition, but a range of values, thereby leading to a range of T_{cross}.

IX. DISCUSSION

There are a number of converging lines of argument that independently motivate the idea of thermodynamic computing. The first line of argument follows from the way in which scale-dependent effects will impede current hardware miniaturization efforts. What's known as Moore's law is an informal law, encoding the empirical observation that compute density has tended to roughly double every two years, as measured by the number of transistors on a microchip. On current estimates, by the end of this decade, circuits will become so small and dense that scale-dependent quantum effects will begin to disrupt the determinism of binary logic upon which current forms of digital computing depend [227]. Along the current path, development of digital computing hardware will therefore run into some physical hard limits. We argue that thermodynamic computing provides an alternative to the current paradigm that can avoid this dead end. Additionally, thermodynamic computing promises both a more efficient path to the development of computing technologies (and of artificial intelligence in particular). Recent growth in the demand for compute resources has led to an associated growth in large-scale projects aiming to vastly increase energy production. Given the orders-of-magnitude efficiency increases that it enables, we believe that thermodynamic computing opens paths to scaling up artificial intelligence without imposing unreasonable demands on modern power grids, thus promising to reduce environmental and financial costs.

A. Superconducting outlook

Multiple technical hurdles have to be overcome to achieve a large-scale thermodynamic computer based on superconducting circuits. Although devices with thousands of similar nodes have been demonstrated [36], scalability remains a key challenge, in particular with regard to control and calibration. Primarily driven by the field of superconducting quantum computing, efforts are underway to make the platform more scalable through CMOS-compatible fabrication [228], 3D integration [229–232] and multiplexed control [233]. Additionally, there are pathways to miniaturize the circuit components. For example, the capacitive elements could be miniaturized by introducing higher-permittivity dielectric materials and inductors can be sized down by utilizing high kinetic inductance materials [234]. A thermodynamic computer, contrary to a superconducting quantum computer, does not rely on highly quantum coherent operations and is therefore less prone to dissipative dielectrics.

Another challenge lies in the readout method, as the presented method interrupts the thermodynamic neuron trajectory and is limited to solely measuring the discrete left or right position. Future work could improve readout by implementing continuous measurements, enabling broader applications like monitoring the state continuously to estimate work statistics [82]. More subtle approaches have already been presented in other works, including a magnetometer based on a resonator terminated by an rf-SQUID [214]. Another digital approach to measuring stationary fields confined in cavities has been proposed [235] but this technique is experimentally challenging and less scalable.

For any actual thermodynamic calculation, multiple thermodynamic neurons will need to be coupled together in order to form a large potential landscape with many degrees of freedom. Such coupling can be made with tunable inductive couplers, as shown in D-Wave devices or flux qubits [212]. Additionally, such couplers could be engineered to generate a 3-body interaction.

While noise and thermal fluctuations can be controlled via the temperature of the device, it could be useful to also directly control the effective friction or resistance of the system. There are a few potential avenues for doing so. One is simply to control resistance directly via the temperature, which changes the quasiparticle population. This may not be the most useful technique since it forces the resistance to change in lockstep with the noise, but does not require any experimental additions. Another way would be to inject quasiparticles from room temperature, either with a strong microwave pulse or an infrared source. Finally, a more complex method would be to directly fabricate a normal metal resistance on chip. These methods could also be used together for a more complete control over the friction of individual thermodynamic neurons.

While the device's on-chip power is low, the total energy consumption is considerable and near-exclusively determined by cooling infrastructure. However, there are pathways to reduce the required cooling power. These include moving to a new material platform with a higher critical temperature such as niobium ($T_{\text{crit}} = 9.3\,\text{K}$). Moreover, the thermal load can be reduced by integrating the control electronics into the cryogenic setup, reducing the thermal load.

Another challenge is that estimation oscillators are likely a difficult operation to perform on superconducting hardware (due to the magnitude of change in capacitance

required), but there are some proposals that one could consider developing further [236].

B. Other applications and future directions

As outlined in Sec. VII, there are a number of other potential applications that can be explored. Each application or hardware modality may have a specific set of requirements or require adaptations, but the framework that we have proposed is general and flexible enough to integrate many workflows. For example, recent work has shown the power of quartic potentials similar to those presented here, which undergo (overdamped) Langevin dynamics and which can even be trained directly as diffusion models [237].

We have demonstrated that our energy-based thermodynamic computing framework can successfully implement a broad range of different probabilistic machine learning methods. Thermodynamic computers need not be used in isolation and could substantially improve the efficiency of both deterministic and quantum computing stacks. One potential first such application of superconducting thermodynamic computers is as co-processors with superconducting quantum computing chips, for example, to help with error correction (which digital graphical models are already used for [238, 239]).

Having already outlined further steps to advance superconducting hardware, we note in closing that the ideas of this framework could be implemented in a variety of modalities and are not intrinsically tied to superconductors. We have explored similar directions in recent CMOS work [20, 33].

X. CONCLUSION

In this paper, we presented a framework for energy-based thermodynamic computing based on probabilistic graphical models and superconducting circuits driven by thermal fluctuations. We have described the relevant building blocks and carried out idealized analyses of equilibration times for relevant operations of the device. These results suggest a variety of avenues for further explorations. Our work contributes to the exciting development of physics-based computing hardware at scale for meaningful machine learning workflows, which dovetails with neuromorphic approaches in neuroscience and machine learning. We hope that this work helps to inspire the research and development of large-scale energy-based thermodynamic computers.

XI. ACKNOWLEDGMENTS

The authors thank Jason Shi for their contributions to thermoformer numerics, Ian MacCormack for their contributions to the Fokker-Planck analytics, Geremia Massarelli and Jeremy Rothschild for their work on timescale separation, Maxwell Ramstead for their writing help, and Thomas Hubregtsen for early numerics infrastructure.

Experiments were performed at Espace Quantique 1 at the DistriQ Quantum Innovation Zone in Sherbrooke, Canada. Fabrication was performed at the Interdisciplinary Institute for Technological Innovation (3IT) and l'Infrastructure matériaux et dispositifs quantiques (IMDQ) facilities of l'Université de Sherbrooke. We thank the cleanroom staff as well as the fabrication team of Nord Quantique for their kind assistance with developing the fabrication process.

[1] A. de Vries, The growing energy footprint of artificial intelligence, Joule **7**, 2191 (2023).
[2] A. Reuther, P. Michaleas, M. Jones, V. Gadepally, S. Samsi, and J. Kepner, Ai and ml accelerator survey and trends, in *2022 IEEE High Performance Extreme Computing Conference (HPEC)* (IEEE, 2022) pp. 1–10.
[3] S. Luccioni, Y. Jernite, and E. Strubell, Power hungry processing: Watts driving the cost of ai deployment?, in *The 2024 ACM Conference on Fairness, Accountability, and Transparency* (2024) pp. 85–99.
[4] E. Masanet, A. Shehabi, N. Lei, S. Smith, and J. Koomey, Recalibrating global data center energy-use estimates, Science **367**, 984 (2020).
[5] M. Mohseni, A. Scherer, K. G. Johnson, O. Wertheim, M. Otten, N. A. Aadit, K. M. Bresniker, K. Y. Camsari, B. Chapman, S. Chatterjee, *et al.*, How to build a quantum supercomputer: Scaling challenges and opportunities, arXiv preprint arXiv:2411.10406 (2024).
[6] N. P. De Leon, K. M. Itoh, D. Kim, K. K. Mehta, T. E. Northup, H. Paik, B. Palmer, N. Samarth, S. Sangtawesin, and D. W. Steuerman, Materials challenges and opportunities for quantum computing hardware, Science **372**, eabb2823 (2021).
[7] G. Wendin, Quantum information processing with superconducting circuits: a review, Reports on Progress in Physics **80**, 106001 (2017).
[8] D. H. Wolpert, The stochastic thermodynamics of computation, Journal of Physics A: Mathematical and Theoretical **52**, 193001 (2019).
[9] D. H. Wolpert, J. Korbel, C. W. Lynn, F. Tasnim, J. A. Grochow, G. Kardeş, J. B. Aimone, V. Balasubramanian, E. De Giuli, D. Doty, *et al.*, Is stochastic thermodynamics the key to understanding the energy costs of computation?, Proceedings of the National Academy of Sciences **121**, e2321112121 (2024).
[10] S. Hooker, The hardware lottery, Communications of the ACM **64**, 58 (2021).
[11] T. Conte, E. DeBenedictis, N. Ganesh, T. Hylton, J. P. Strachan, R. S. Williams, A. Alemi, L. Altenberg, G. Crooks, J. Crutchfield, *et al.*, Thermodynamic com-

puting, arXiv preprint arXiv:1911.01968 (2019).
[12] P. Langevin, Sur la théorie du mouvement brownien (1908).
[13] G. E. Crooks, *Excursions in Statistical Dynamics*, Ph.D. thesis, University of California at Berkeley, Berkeley, California, United States of America (1999).
[14] A. A. Golubov, M. Y. Kupriyanov, and E. Il'Ichev, The current-phase relation in josephson junctions, Reviews of modern physics **76**, 411 (2004).
[15] O.-P. Saira, M. H. Matheny, R. Katti, W. Fon, G. Wimsatt, J. P. Crutchfield, S. Han, and M. L. Roukes, Nonequilibrium thermodynamics of erasure with superconducting flux logic, Physical Review Research **2**, 013249 (2020), publisher: American Physical Society.
[16] P. Huembeli, J. M. Arrazola, N. Killoran, M. Mohseni, and P. Wittek, The physics of energy-based models, Quantum Machine Intelligence **4**, 1 (2022).
[17] Y. LeCun, S. Chopra, R. Hadsell, M. Ranzato, F. Huang, *et al.*, A tutorial on energy-based learning, Predicting structured data **1** (2006).
[18] Y. Du and I. Mordatch, Implicit generation and modeling with energy based models, Advances in Neural Information Processing Systems **32** (2019).
[19] Y. Song and D. P. Kingma, How to train your energy-based models, arXiv preprint arXiv:2101.03288 (2021).
[20] A. Jelinčič, O. Lockwood, A. Garlapati, P. Schillinger, I. L. Chuang, G. Verdon, and T. McCourt, An efficient probabilistic hardware architecture for diffusion-like models, npj Unconventional Computing **3**, 30 (2026).
[21] W. Moy, I. Ahmed, P.-w. Chiu, J. Moy, S. S. Sapatnekar, and C. H. Kim, A 1,968-node coupled ring oscillator circuit for combinatorial optimization problem solving, Nature Electronics **5**, 310 (2022).
[22] T. Inagaki, Y. Haribara, K. Igarashi, T. Sonobe, S. Tamate, T. Honjo, A. Marandi, P. L. McMahon, T. Umeki, K. Enbutsu, *et al.*, A coherent ising machine for 2000-node optimization problems, Science **354**, 603 (2016).
[23] N. Mohseni, P. L. McMahon, and T. Byrnes, Ising machines as hardware solvers of combinatorial optimization problems, Nature Reviews Physics **4**, 363 (2022).
[24] N. A. Aadit, A. Grimaldi, M. Carpentieri, L. Theogarajan, J. M. Martinis, G. Finocchio, and K. Y. Camsari, Massively parallel probabilistic computing with sparse ising machines, Nature Electronics **5**, 460 (2022).
[25] N. S. Singh, K. Kobayashi, Q. Cao, K. Selcuk, T. Hu, S. Niazi, N. A. Aadit, S. Kanai, H. Ohno, S. Fukami, *et al.*, Cmos plus stochastic nanomagnets enabling heterogeneous computers for probabilistic inference and learning, Nature Communications **15**, 2685 (2024).
[26] J. Laydevant, D. Marković, and J. Grollier, Training an ising machine with equilibrium propagation, Nature Communications **15**, 3671 (2024).
[27] J. Chou, S. Bramhavar, S. Ghosh, and W. Herzog, Analog coupled oscillator based weighted ising machine, Scientific reports **9**, 14786 (2019).
[28] K. Y. Camsari, R. Faria, B. M. Sutton, and S. Datta, Stochastic p-bits for invertible logic, Physical Review X **7**, 031014 (2017).
[29] K. Y. Camsari, B. M. Sutton, and S. Datta, P-bits for probabilistic spin logic, Applied Physics Reviews **6** (2019).
[30] J. Kaiser and S. Datta, Probabilistic computing with p-bits, Applied Physics Letters **119** (2021).
[31] S. Chowdhury, A. Grimaldi, N. A. Aadit, S. Niazi, M. Mohseni, S. Kanai, H. Ohno, S. Fukami, L. Theogarajan, G. Finocchio, *et al.*, A full-stack view of probabilistic computing with p-bits: devices, architectures, and algorithms, IEEE Journal on Exploratory Solid-State Computational Devices and Circuits **9**, 1 (2023).
[32] S. Niazi, S. Chowdhury, N. A. Aadit, M. Mohseni, Y. Qin, and K. Y. Camsari, Training deep boltzmann networks with sparse ising machines, Nature Electronics , 1 (2024).
[33] N. Freitas, G. Massarelli, J. Rothschild, D. Keane, E. Dawe, S. Hwang, A. Garlapati, and T. McCourt, Taming nonequilibrium thermal fluctuations in subthreshold cmos circuits (2026).
[34] M. W. Johnson, M. H. Amin, S. Gildert, T. Lanting, F. Hamze, N. Dickson, R. Harris, A. J. Berkley, J. Johansson, P. Bunyk, *et al.*, Quantum annealing with manufactured spins, Nature **473**, 194 (2011).
[35] A. D. King, S. Suzuki, J. Raymond, A. Zucca, T. Lanting, F. Altomare, A. J. Berkley, S. Ejtemaee, E. Hoskinson, S. Huang, *et al.*, Coherent quantum annealing in a programmable 2,000 qubit ising chain, Nature Physics **18**, 1324 (2022).
[36] A. D. King, J. Raymond, T. Lanting, R. Harris, A. Zucca, F. Altomare, A. J. Berkley, K. Boothby, S. Ejtemaee, C. Enderud, *et al.*, Quantum critical dynamics in a 5,000-qubit programmable spin glass, Nature **617**, 61 (2023).
[37] J. M. Shainline, S. M. Buckley, R. P. Mirin, and S. W. Nam, Superconducting optoelectronic circuits for neuromorphic computing, Physical Review Applied **7**, 034013 (2017).
[38] A. Kumar, U. S. Goteti, E. Cubukcu, R. C. Dynes, and D. Kuzum, Evaluation of fluxon synapse device based on superconducting loops for energy efficient neuromorphic computing, Frontiers in Neuroscience **19**, 1511371 (2025).
[39] D. Kudithipudi, C. Schuman, C. M. Vineyard, T. Pandit, C. Merkel, R. Kubendran, J. B. Aimone, G. Orchard, C. Mayr, R. Benosman, *et al.*, Neuromorphic computing at scale, Nature **637**, 801 (2025).
[40] J. B. Aimone, Neuromorphic computing: A theoretical framework for time, space, and energy scaling, arXiv preprint arXiv:2507.17886 (2025).
[41] P. J. Coles, C. Szczepanski, D. Melanson, K. Donatella, A. J. Martinez, and F. Sbahi, Thermodynamic ai and the fluctuation frontier, in *2023 IEEE International Conference on Rebooting Computing (ICRC)* (IEEE, 2023) pp. 1–10.
[42] M. Aifer, K. Donatella, M. H. Gordon, S. Duffield, T. Ahle, D. Simpson, G. Crooks, and P. J. Coles, Thermodynamic linear algebra, npj Unconventional Computing **1**, 13 (2024).
[43] D. Melanson, M. A. Khater, M. Aifer, K. Donatella, M. H. Gordon, T. Ahle, G. Crooks, A. J. Martinez, F. Sbahi, and P. J. Coles, Thermodynamic computing system for ai applications, arXiv preprint arXiv:2312.04836 (2023).
[44] M. Aifer, S. Duffield, K. Donatella, D. Melanson, P. Klett, Z. Belateche, G. Crooks, A. J. Martinez, and P. J. Coles, Thermodynamic bayesian inference, arXiv preprint arXiv:2410.01793 (2024).
[45] K. Donatella, S. Duffield, M. Aifer, D. Melanson, G. Crooks, and P. J. Coles, Thermodynamic natural gradient descent, arXiv preprint arXiv:2405.13817

(2024).
[46] M. Osadchy, M. Miller, and Y. Cun, Synergistic face detection and pose estimation with energy-based models, Advances in neural information processing systems **17** (2004).
[47] M. Ranzato, C. Poultney, S. Chopra, and Y. Cun, Efficient learning of sparse representations with an energy-based model, Advances in neural information processing systems **19** (2006).
[48] S. Zhai, Y. Cheng, W. Lu, and Z. Zhang, Deep structured energy based models for anomaly detection, in *International conference on machine learning* (PMLR, 2016) pp. 1100–1109.
[49] N. Liu, S. Li, Y. Du, A. Torralba, and J. B. Tenenbaum, Compositional visual generation with composable diffusion models, in *European Conference on Computer Vision* (Springer, 2022) pp. 423–439.
[50] G. E. Hinton, Training products of experts by minimizing contrastive divergence, Neural computation **14**, 1771 (2002).
[51] M. A. Carreira-Perpinan and G. Hinton, On contrastive divergence learning, in *International workshop on artificial intelligence and statistics* (PMLR, 2005) pp. 33–40.
[52] Y. Bengio and O. Delalleau, Justifying and generalizing contrastive divergence, Neural computation **21**, 1601 (2009).
[53] I. Sutskever and T. Tieleman, On the convergence properties of contrastive divergence, in *Proceedings of the thirteenth international conference on artificial intelligence and statistics* (JMLR Workshop and Conference Proceedings, 2010) pp. 789–795.
[54] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, Score-based generative modeling through stochastic differential equations, arXiv preprint arXiv:2011.13456 (2020).
[55] X. Cheng and P. Bartlett, Convergence of langevin mcmc in kl-divergence, in *Algorithmic Learning Theory* (PMLR, 2018) pp. 186–211.
[56] X. Cheng, N. S. Chatterji, P. L. Bartlett, and M. I. Jordan, Underdamped langevin mcmc: A non-asymptotic analysis, in *Conference on learning theory* (PMLR, 2018) pp. 300–323.
[57] R. Zhang, X. Liu, and Q. Liu, A langevin-like sampler for discrete distributions, in *International Conference on Machine Learning* (PMLR, 2022) pp. 26375–26396.
[58] H. Sun, H. Dai, B. Dai, H. Zhou, and D. Schuurmans, Discrete langevin samplers via wasserstein gradient flow, in *International Conference on Artificial Intelligence and Statistics* (PMLR, 2023) pp. 6290–6313.
[59] G. O. Roberts and R. L. Tweedie, Exponential convergence of Langevin distributions and their discrete approximations, Bernoulli **2**, 341 (1996).
[60] V. Balakrishnan, Fluctuation-dissipation theorems from the generalised langevin equation, Pramana **12**, 301 (1979).
[61] S. Särkkä and A. Solin, *Applied stochastic differential equations*, Vol. 10 (Cambridge University Press, 2019).
[62] H. Risken, *The Fokker-Planck Equation: Methods of Solution and Applications*, second edition ed., edited by H. Haken (Springer, Berlin, 1989).
[63] C. H. Bennett, The thermodynamics of computation—a review, International Journal of Theoretical Physics **21**, 905 (1982).
[64] J. A. Vaccaro and S. M. Barnett, Information erasure without an energy cost, Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences **467**, 1770 (2011).
[65] E. Fredkin and T. Toffoli, Conservative logic, International Journal of theoretical physics **21**, 219 (1982).
[66] S. Shankar, Energy estimates across layers of computing: from devices to large-scale applications in machine learning for natural language processing, scientific computing, and cryptocurrency mining, in *2023 IEEE High Performance Extreme Computing Conference (HPEC)* (IEEE, 2023) pp. 1–6.
[67] M. Nakazato and S. Ito, Geometrical aspects of entropy production in stochastic thermodynamics based on wasserstein distance, Physical Review Research **3**, 043093 (2021).
[68] M. Reilly and S. Lloyd, Physical complexity and black hole quantum computers, in *Journal of Physics: Conference Series*, Vol. 3017 (IOP Publishing, 2025) p. 012010.
[69] S. Lahiri, J. Sohl-Dickstein, and S. Ganguli, A universal tradeoff between power, precision and speed in physical communication, arXiv preprint arXiv:1603.07758 (2016).
[70] S. Ito, Stochastic thermodynamic interpretation of information geometry, Physical review letters **121**, 030605 (2018).
[71] S. Ito and A. Dechant, Stochastic time evolution, information geometry, and the cramér-rao bound, Physical Review X **10**, 021056 (2020).
[72] S. Ito, Geometric thermodynamics for the fokker–planck equation: stochastic thermodynamic links between information geometry and optimal transport, Information Geometry **7**, 441 (2024).
[73] J. Hnybida and S. Verret, Minimal-dissipation learning for energy-based models, arXiv preprint arXiv:2510.03137 (2025).
[74] C. Villani *et al.*, *Optimal transport: old and new*, Vol. 338 (Springer, 2009).
[75] J. Klinger and G. M. Rotskoff, Minimally dissipative multi-bit logical operations, arXiv preprint arXiv:2506.24021 (2025).
[76] A. Rolandi, P. Abiuso, P. Lipka-Bartosik, M. Aifer, P. J. Coles, and M. Perarnau-Llobet, Energy-time-accuracy tradeoffs in thermodynamic computing, arXiv preprint arXiv:2601.04358 (2026).
[77] H. A. Kramers, Brownian motion in a field of force and the diffusion model of chemical reactions, physica **7**, 284 (1940).
[78] M. Büttiker, E. Harris, and R. Landauer, Thermal activation in extremely underdamped josephson-junction circuits, Physical Review B **28**, 1268 (1983).
[79] H. Risken and K. Voigtlaender, Eigenvalues and eigenfunctions of the fokker-planck equation for the extremely underdamped brownian motion in a double-well potential, Journal of statistical physics **41**, 825 (1985).
[80] K. Sekimoto, Stochastic energetics (2010).
[81] K. J. Ray and J. P. Crutchfield, Gigahertz sub-landauer momentum computing, Physical Review Applied **19**, 014049 (2023).
[82] G. Wimsatt, O.-P. Saira, A. B. Boyd, M. H. Matheny, S. Han, M. L. Roukes, and J. P. Crutchfield, Harnessing fluctuations in thermodynamic computing via time-reversal symmetries, Physical Review Research **3**, 033115 (2021).

[83] C. Jarzynski, Equalities and inequalities: Irreversibility and the second law of thermodynamics at the nanoscale, in *Time: Poincaré Seminar 2010* (Springer, 2012) pp. 145–172.

[84] N. Shiraishi, An introduction to stochastic thermodynamics, Fundamental Theories of Physics. Springer, Singapore (2023).

[85] S. Vaikuntanathan and C. Jarzynski, Escorted free energy simulations, The Journal of chemical physics **134** (2011).

[86] S. Park, F. Khalili-Araghi, E. Tajkhorshid, and K. Schulten, Free energy calculation from steered molecular dynamics simulations using jarzynski's equality, The Journal of chemical physics **119**, 3559 (2003).

[87] S. Whitelam and C. Casert, Thermodynamic computing out of equilibrium, arXiv preprint arXiv:2412.17183 (2024).

[88] F. Schäfer, M. A. Bastarrachea-Magnani, A. U. Lode, L. d. F. de Parny, and A. Buchleitner, Spectral structure and many-body dynamics of ultracold bosons in a double-well, Entropy **22**, 382 (2020).

[89] S. Borah, B. Sarma, M. Kewming, G. J. Milburn, and J. Twamley, Measurement-based feedback quantum control with deep reinforcement learning for a double-well nonlinear potential, Physical review letters **127**, 190403 (2021).

[90] Q. Wu, L. Mancino, M. Carlesso, M. A. Ciampini, L. Magrini, N. Kiesel, and M. Paternostro, Nonequilibrium quantum thermodynamics of a particle trapped in a controllable time-varying potential, PRX Quantum **3**, 010322 (2022).

[91] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, *et al.*, Scipy 1.0: fundamental algorithms for scientific computing in python, Nature methods **17**, 261 (2020).

[92] P. Kidger, *On Neural Differential Equations*, Ph.D. thesis, University of Oxford (2021).

[93] J. Foster, G. dos Reis, and C. Strange, High order splitting methods for sdes satisfying a commutativity condition, arXiv:2210.17543 (2023).

[94] J. Foster, T. Lyons, and H. Oberhauser, The shifted ode method for underdamped langevin mcmc, arXiv preprint arXiv:2101.03446 (2021).

[95] J. Choi, M. Dukhan, X. Liu, and R. Vuduc, Algorithmic time, energy, and power on candidate hpc compute building blocks, in *2014 IEEE 28th international parallel and distributed processing symposium* (IEEE, 2014) pp. 447–457.

[96] E. García-Martín, C. F. Rodrigues, G. Riley, and H. Grahn, Estimation of energy consumption in machine learning, Journal of Parallel and Distributed Computing **134**, 75 (2019).

[97] S. Whitelam, Training thermodynamic computers by gradient descent, arXiv preprint arXiv:2509.15324 (2025).

[98] A. Borle and S. J. Lomonaco, Analyzing the quantum annealing approach for solving linear least squares problems, in *International Workshop on Algorithms and Computation* (Springer, 2018) pp. 289–301.

[99] C. Chamberland and G. Verdon-Akzam, Thermodynamic computing relay gadget, US Patent Application Publication US 2025/0284867 A1 (2025), Assignee: Extropic Corp. Status: pending.

[100] C. Chamberland and G. Verdon-Akzam, Thermodynamic computing relay gadget for multi-well potentials, US Patent Application Publication US 2025/0373202 A1 (2025), Assignee: Extropic Corp. Status: pending.

[101] C. Chamberland and G. Verdon-Akzam, Thermodynamic computing system configured to implement transformer based architecture, US Patent Application Publication US 2025/0284949 A1 (2025), Assignee: Extropic Corp. Status: pending.

[102] M. M. H. Sajeeb, N. A. Aadit, S. Chowdhury, T. Wu, C. Smith, D. Chinmay, A. Raut, K. Y. Camsari, C. Delacour, and T. Srimani, Scalable connectivity for ising machines: Dense to sparse, Physical Review Applied **24**, 014005 (2025).

[103] A. Agrawal, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, and R. Ramjee, Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills, arXiv preprint arXiv:2308.16369 (2023).

[104] Z. Yuan, Y. Shang, Y. Zhou, Z. Dong, Z. Zhou, C. Xue, B. Wu, Z. Li, Q. Gu, Y. J. Lee, *et al.*, Llm inference unveiled: Survey and roofline model insights, arXiv preprint arXiv:2402.16363 (2024).

[105] J. Bolte and E. Pauwels, A mathematical model for automatic differentiation in machine learning, Advances in Neural Information Processing Systems **33**, 10809 (2020).

[106] Y. LeCun, D. Touresky, G. Hinton, and T. Sejnowski, A theoretical framework for back-propagation, in *Proceedings of the 1988 connectionist models summer school*, Vol. 1 (1988) pp. 21–28.

[107] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, Automatic differentiation in machine learning: a survey, Journal of machine learning research **18**, 1 (2018).

[108] C. C. Margossian, A review of automatic differentiation and its efficient implementation, Wiley interdisciplinary reviews: data mining and knowledge discovery **9**, e1305 (2019).

[109] F. Schäfer, M. Tarek, L. White, and C. Rackauckas, Abstractdifferentiation. jl: Backend-agnostic differentiable programming in julia, arXiv preprint arXiv:2109.12449 (2021).

[110] W. S. Moses, V. Churavy, L. Paehler, J. Hückelheim, S. H. K. Narayanan, M. Schanen, and J. Doerfert, Reverse-mode automatic differentiation and optimization of gpu kernels via enzyme, in *Proceedings of the international conference for high performance computing, networking, storage and analysis* (2021) pp. 1–16.

[111] G. Arya, M. Schauer, F. Schäfer, and C. Rackauckas, Automatic differentiation of programs with discrete randomness, Advances in Neural Information Processing Systems **35**, 10435 (2022).

[112] G. Arya, R. Seyer, F. Schäfer, K. Chandra, A. K. Lew, M. Huot, V. K. Mansinghka, J. Ragan-Kelley, C. Rackauckas, and M. Schauer, Differentiating metropolis-hastings to optimize intractable densities, arXiv preprint arXiv:2306.07961 (2023).

[113] C. Chamberland and G. Verdon-Akzam, Thermodynamic computing mean-field forwards and backwards propagation, US Patent Application Publication US 2025/0284959 A1 (2025), Assignee: Extropic Corp. Status: pending.

[114] C. Chamberland and G. Verdon-Akzam, Thermodynamic computing mean-field forwards and back-

wards propagation, PCT Application Publication WO 2025/189010 A8 (2025), Assignee: Extropic Corp. Status: pending.
[115] B. Scellier and Y. Bengio, Equilibrium propagation: Bridging the gap between energy-based models and backpropagation, Frontiers in computational neuroscience **11**, 24 (2017).
[116] J. Kendall, R. Pantone, K. Manickavasagam, Y. Bengio, and B. Scellier, Training end-to-end analog neural networks with equilibrium propagation, arXiv preprint arXiv:2006.01981 (2020).
[117] M. Stern, D. Hexner, J. W. Rocks, and A. J. Liu, Supervised learning in physical networks: From machine learning to learning machines, Physical Review X **11**, 021045 (2021).
[118] S. Massar and B. M. Mognetti, Equilibrium propagation: the quantum and the thermal cases, Quantum Studies: Mathematics and Foundations **12**, 6 (2025).
[119] M. Fortunato, M. G. Azar, B. Piot, J. Menick, I. Osband, A. Graves, V. Mnih, R. Munos, D. Hassabis, O. Pietquin, C. Blundell, and S. Legg, Noisy networks for exploration, CoRR **abs/1706.10295** (2017), 1706.10295.
[120] M. Plappert, R. Houthooft, P. Dhariwal, S. Sidor, R. Y. Chen, X. Chen, T. Asfour, P. Abbeel, and M. Andrychowicz, Parameter space noise for exploration, arXiv preprint arXiv:1706.01905 (2017).
[121] O. Eberhard, J. Hollenstein, C. Pinneri, and G. Martius, Pink noise is all you need: Colored noise exploration in deep reinforcement learning, in *The Eleventh International Conference on Learning Representations* (2023).
[122] Y. Gal and Z. Ghahramani, Dropout as a bayesian approximation: Representing model uncertainty in deep learning, in *international conference on machine learning* (PMLR, 2016) pp. 1050–1059.
[123] Y. Gal, J. Hron, and A. Kendall, Concrete dropout, Advances in neural information processing systems **30** (2017).
[124] O. Lockwood and M. Si, A review of uncertainty for deep reinforcement learning, in *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, Vol. 18 (2022) pp. 155–162.
[125] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, Dropout: a simple way to prevent neural networks from overfitting, The journal of machine learning research **15**, 1929 (2014).
[126] D. P. Kingma, T. Salimans, and M. Welling, Variational dropout and the local reparameterization trick, Advances in neural information processing systems **28** (2015).
[127] X. Shen, X. Tian, T. Liu, F. Xu, and D. Tao, Continuous dropout, IEEE transactions on neural networks and learning systems **29**, 3926 (2017).
[128] D. Molchanov, A. Ashukha, and D. Vetrov, Variational dropout sparsifies deep neural networks, in *International conference on machine learning* (PMLR, 2017) pp. 2498–2507.
[129] D. Koller, Probabilistic graphical models: Principles and techniques (2009).
[130] H.-A. Loeliger, An introduction to factor graphs, IEEE Signal Processing Magazine **21**, 28 (2004).
[131] B. A. Cipra, An introduction to the ising model, The American Mathematical Monthly **94**, 937 (1987).
[132] M. Drton and M. H. Maathuis, Structure learning in graphical modeling, Annual Review of Statistics and Its Application **4**, 365 (2017).
[133] Y. Du and L. Kaelbling, Compositional generative modeling: A single model is not all you need, arXiv preprint arXiv:2402.01103 (2024).
[134] C. Chamberland and G. Verdon-Akzam, Gibbs sampling methods using thermodynamic computing, US Patent Application Publication US 2025/0284562 A1 (2025), Assignee: Extropic Corp. Status: pending.
[135] C. P. Robert, G. Casella, and G. Casella, *Monte Carlo statistical methods*, Vol. 2 (Springer, 1999).
[136] C. M. De Sa, C. Zhang, K. Olukotun, and C. Ré, Rapidly mixing gibbs sampling for a class of factor graphs using hierarchy width, Advances in neural information processing systems **28** (2015).
[137] J. Gonzalez, Y. Low, A. Gretton, and C. Guestrin, Parallel gibbs sampling: From colored fields to thin junction trees, in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics* (JMLR Workshop and Conference Proceedings, 2011) pp. 324–332.
[138] A. Terenin, D. Simpson, and D. Draper, Asynchronous gibbs sampling, in *International Conference on Artificial Intelligence and Statistics* (PMLR, 2020) pp. 144–154.
[139] C. Daskalakis, N. Dikkala, and S. Jayanti, Hogwild!-gibbs can be panaccurate, Advances in Neural Information Processing Systems **31** (2018).
[140] J. S. Yedidia, W. Freeman, and Y. Weiss, Generalized belief propagation, Advances in neural information processing systems **13** (2000).
[141] C. M. Bishop, Latent variable models, in *Learning in graphical models* (Springer, 1998) pp. 371–403.
[142] D. de Bos and M. Serra-Garcia, Learning in a multifield coherent ising machine (2025), arXiv:2502.12020 [cond-mat.mes-hall].
[143] S. Lloyd, Thermodynamics+ natural selection= bayesian inference, arXiv preprint arXiv:2511.17641 (2025).
[144] C. Bösch, G. Roeder, M. Serra-Garcia, and R. P. Adams, Local learning rules for out-of-equilibrium physical generative models, arXiv preprint arXiv:2506.19136 (2025).
[145] V. Lopez-Pastor and F. Marquardt, Self-learning machines based on hamiltonian echo backpropagation, Physical Review X **13**, 031020 (2023).
[146] J. Martens, New insights and perspectives on the natural gradient method, Journal of Machine Learning Research **21**, 1 (2020).
[147] C. Chamberland and G. Verdon-Akzam, Thermodynamic computing system configured to use natural gradient descent techniques to determine updated weights and biases, US Patent Application Publication US 2025/0238670 A1 (2025), Assignee: Extropic Corp. Status: pending.
[148] C. Chamberland and G. Verdon-Akzam, Thermodynamic computing system configured to update weights and biases based on gradient values obtained by relay oscillators, US Patent Application Publication US 2025/0390737 A1 (2025), Assignee: Extropic Corp. Status: pending.
[149] C. Chamberland and G. Verdon-Akzam, Self-learning thermodynamic computing system, US Patent Application Publication US 2025/0165761 A1 (2025), Assignee:

Extropic Corp. Status: pending.
[150] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, Attention is all you need, CoRR **abs/1706.03762** (2017), 1706.03762.
[151] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, JAX: composable transformations of Python+NumPy programs (2018).
[152] DeepMind, I. Babuschkin, K. Baumli, A. Bell, S. Bhupatiraju, J. Bruce, P. Buchlovsky, D. Budden, T. Cai, A. Clark, I. Danihelka, A. Dedieu, C. Fantacci, J. Godwin, C. Jones, R. Hemsley, T. Hennigan, M. Hessel, S. Hou, S. Kapturowski, T. Keck, I. Kemaev, M. King, M. Kunesch, L. Martens, H. Merzic, V. Mikulik, T. Norman, G. Papamakarios, J. Quan, R. Ring, F. Ruiz, A. Sanchez, L. Sartran, R. Schneider, E. Sezener, S. Spencer, S. Srinivasan, M. Stanojević, W. Stokowiec, L. Wang, G. Zhou, and F. Viola, The DeepMind JAX Ecosystem (2020).
[153] P. Kidger and C. Garcia, Equinox: neural networks in JAX via callable PyTrees and filtered transformations, Differentiable Programming workshop at Neural Information Processing Systems 2021 (2021).
[154] D. Bickson, Gaussian belief propagation: Theory and aplication, arXiv preprint arXiv:0811.2518 (2008).
[155] Q. Su and Y.-C. Wu, On convergence conditions of gaussian belief propagation, IEEE Transactions on Signal Processing **63**, 1144 (2015).
[156] J. Ortiz, T. Evans, and A. J. Davison, A visual introduction to gaussian belief propagation, arXiv preprint arXiv:2107.02308 (2021).
[157] V. G. Satorras and M. Welling, Neural enhanced belief propagation on factor graphs, in *International Conference on Artificial Intelligence and Statistics* (PMLR, 2021) pp. 685–693.
[158] M. Liang and F. Meyer, Neural enhanced belief propagation for cooperative localization, in *2021 IEEE Statistical Signal Processing Workshop (SSP)* (IEEE, 2021) pp. 326–330.
[159] A. Patwardhan, R. Murai, and A. J. Davison, Distributing collaborative multi-robot planning with gaussian belief propagation, IEEE Robotics and Automation Letters **8**, 552 (2022).
[160] J. Ortiz, *Gaussian belief propagation for real-time decentralised inference*, Ph.D. thesis, Imperial College London (2023).
[161] M. Liang and F. Meyer, Neural enhanced belief propagation for multiobject tracking, IEEE Transactions on Signal Processing (2023).
[162] Y. Dandi, L. Stephan, F. Krzakala, B. Loureiro, and L. Zdeborová, Universality laws for gaussian mixtures in generalized linear models, Advances in Neural Information Processing Systems **36** (2024).
[163] H. Shi and M. Drton, On universal inference in gaussian mixture models, arXiv preprint arXiv:2407.19361 (2024).
[164] O. Lockwood, distreqx: Distributions and bijectors in jax, `https://github.com/lockwo/distreqx` (2024).
[165] L. R. Rabiner, A tutorial on hidden markov models and selected applications in speech recognition, Proceedings of the IEEE **77**, 257 (1989).
[166] S. R. Eddy, What is a hidden markov model?, Nature biotechnology **22**, 1315 (2004).
[167] A. Krogh, M. Brown, I. S. Mian, K. Sjölander, and D. Haussler, Hidden markov models in computational biology: Applications to protein modeling, Journal of molecular biology **235**, 1501 (1994).
[168] R. S. Mamon and R. J. Elliott, *Hidden Markov models in finance*, Vol. 4 (Springer, 2007).
[169] J. He, G. Neubig, and T. Berg-Kirkpatrick, Unsupervised learning of syntactic structure with invertible neural projections, arXiv preprint arXiv:1808.09111 (2018).
[170] D. Liu, A. Honoré, S. Chatterjee, and L. K. Rasmussen, Powering hidden markov model by neural network based generative models, CoRR **abs/1910.05744** (2019), 1910.05744.
[171] A. Ghosh, A. Honoré, D. Liu, G. E. Henter, and S. Chatterjee, Normalizing flow based hidden markov models for classification of speech phones with explainability, arXiv preprint arXiv:2107.00730 (2021).
[172] E. Azeraf, E. Monfrini, E. Vignon, and W. Pieczynski, Introducing the hidden neural markov chain framework, arXiv preprint arXiv:2102.11038 (2021).
[173] H. Gangloff, K. Morales, and Y. Petetin, A general parametrization framework for pairwise markov models: An application to unsupervised image segmentation, in *2021 IEEE 31st International Workshop on Machine Learning for Signal Processing (MLSP)* (IEEE, 2021) pp. 1–6.
[174] H. Gangloff, K. Morales, and Y. Petetin, Deep parameterizations of pairwise and triplet markov models for unsupervised classification of sequential data, Computational Statistics & Data Analysis **180**, 107663 (2023).
[175] S. W. Linderman, P. Chang, G. Harper-Donnelly, A. Kara, X. Li, G. Duran-Martin, and K. Murphy, Dynamax: A Python package for probabilistic state space modeling with JAX (2025).
[176] S. G. Brush, History of the lenz-ising model, Reviews of modern physics **39**, 883 (1967).
[177] S. Patel, L. Chen, P. Canoza, and S. Salahuddin, Ising model optimization problems on a fpga accelerated restricted boltzmann machine, arXiv preprint arXiv:2008.04436 (2020).
[178] D. H. Ackley, G. E. Hinton, and T. J. Sejnowski, A learning algorithm for boltzmann machines, Cognitive science **9**, 147 (1985).
[179] S. Nikhar, S. Kannan, N. A. Aadit, S. Chowdhury, and K. Y. Camsari, All-to-all reconfigurability with sparse and higher-order ising machines, Nature Communications **15**, 8977 (2024).
[180] G. Bresler, Efficiently learning ising models on arbitrary graphs, in *Proceedings of the forty-seventh annual ACM symposium on Theory of computing* (2015) pp. 771–782.
[181] A. Y. Lokhov, M. Vuffray, S. Misra, and M. Chertkov, Optimal structure and parameter learning of ising models, Science advances **4**, e1700791 (2018).
[182] R. Salakhutdinov and G. Hinton, Deep boltzmann machines, in *Artificial intelligence and statistics* (PMLR, 2009) pp. 448–455.
[183] B. Dunn and Y. Roudi, Learning and inference in a nonequilibrium ising model with hidden nodes, Physical Review E—Statistical, Nonlinear, and Soft Matter Physics **87**, 022127 (2013).
[184] F. Nussbaum and J. Giesen, Ising models with latent conditional gaussian variables, in *Algorithmic Learning*

Theory (PMLR, 2019) pp. 669–681.
[185] K.-i. Nishikawa and H. Nakano, A continuous ising model exhibiting phase transitions of first or second order, Progress of Theoretical Physics **56**, 773 (1976).
[186] H. van Beijeren and G. S. Sylvester, Phase transitions for continuous-spin ising ferromagnets, Journal of Functional Analysis **28**, 145 (1978).
[187] E. Bayong and H. Diep, Effect of long-range interactions on the critical behavior of the continuous ising model, Physical Review B **59**, 11919 (1999).
[188] P. Ramachandran, B. Zoph, and Q. V. Le, Searching for activation functions, arXiv preprint arXiv:1710.05941 (2017).
[189] C. Chamberland and G. Verdon-Akzam, Thermodynamic computing swish gadget, US Patent Application US 18/937,670 (2024), Assignee: Extropic Corp. Filed: Nov. 5, 2024. Status: pending; yet to be published.
[190] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever, *et al.*, Language models are unsupervised multitask learners, OpenAI blog **1**, 9 (2019).
[191] J. L. Ba, J. R. Kiros, and G. E. Hinton, Layer normalization (2016), arXiv:1607.06450 [stat.ML].
[192] R. Xiong, Y. Yang, D. He, K. Zheng, S. Zheng, C. Xing, H. Zhang, Y. Lan, L. Wang, and T. Liu, On layer normalization in the transformer architecture, in *International Conference on Machine Learning* (PMLR, 2020) pp. 10524–10533.
[193] J. Zhu, X. Chen, K. He, Y. LeCun, and Z. Liu, Transformers without normalization, in *Proceedings of the Computer Vision and Pattern Recognition Conference* (2025) pp. 14901–14911.
[194] M. Chen, T. Lu, J. Zhu, M. Sun, and Z. Liu, Stronger normalization-free transformers (2025), arXiv:2512.10938 [cs.LG].
[195] N. Leroux, P.-P. Manea, C. Sudarshan, J. Finkbeiner, S. Siegel, J. P. Strachan, and E. Neftci, Analog in-memory computing attention mechanism for fast and energy-efficient large language models, Nature Computational Science **5**, 813 (2025).
[196] K. He, X. Zhang, S. Ren, and J. Sun, Deep residual learning for image recognition, in *Proceedings of the IEEE conference on computer vision and pattern recognition* (2016) pp. 770–778.
[197] T. Lin, Y. Wang, X. Liu, and X. Qiu, A survey of transformers, AI open **3**, 111 (2022).
[198] A. Khan, Z. Rauf, A. Sohail, A. R. Khan, H. Asif, A. Asif, and U. Farooq, A survey of the vision transformers and their cnn-transformer based variants, Artificial Intelligence Review **56**, 2917 (2023).
[199] E. Min, R. Chen, Y. Bian, T. Xu, K. Zhao, W. Huang, P. Zhao, J. Huang, S. Ananiadou, and Y. Rong, Transformer for graphs: An overview from architecture perspective, arXiv preprint arXiv:2202.08455 (2022).
[200] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, Outrageously large neural networks: The sparsely-gated mixture-of-experts layer, arXiv preprint arXiv:1701.06538 (2017).
[201] W. Fedus, B. Zoph, and N. Shazeer, Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity, Journal of Machine Learning Research **23**, 1 (2022).
[202] C. Chamberland and G. Verdon-Akzam, Mixture of experts energy based model gadget, US Patent Application Publication US 2025/0284998 A1 (2025), Assignee: Extropic Corp. Status: pending.
[203] C. Chamberland and G. Verdon-Akzam, Selection of experts energy based model gadget, US Patent Application Publication US 2025/0284999 A1 (2025), Assignee: Extropic Corp. Status: pending.
[204] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan, *et al.*, The llama 3 herd of models, arXiv preprint arXiv:2407.21783 (2024).
[205] C. Chamberland and G. Verdon-Akzam, Superconducting thermodynamic neuron, US Patent Application Publication US 2025/0284924 A1 (2025), Assignee: Extropic Corp. Status: pending.
[206] K. Y. Camsari, S. Salahuddin, and S. Datta, Implementing p-bits with embedded mtj, IEEE Electron Device Letters **38**, 1767 (2017).
[207] N. Freitas, J.-C. Delvenne, and M. Esposito, Stochastic thermodynamics of nonlinear electronic circuits: A realistic framework for computing around k t, Physical Review X **11**, 031064 (2021).
[208] S. Yang, A. Grimaldi, Y. Bao, E. Raimondo, J. Si, G. Finocchio, and H. Yang, 250 magnetic tunnel junctions-based probabilistic ising machine, arXiv preprint arXiv:2506.14590 (2025).
[209] H. Rhee, G. Kim, H. Song, W. Park, D. H. Kim, J. H. In, Y. Lee, and K. M. Kim, Probabilistic computing with nbox metal-insulator transition-based self-oscillatory pbit, Nature communications **14**, 7199 (2023).
[210] B. D. Josephson, Possible new effects in superconductive tunnelling, Physics Letters **1**, 251 (1962).
[211] B. D. Josephson, The discovery of tunnelling supercurrents, Reviews of Modern Physics **46**, 251 (1974), publisher: American Physical Society.
[212] C. Quintana, *Superconducting flux qubits for high-connectivity quantum annealing without lossy dielectrics*, Ph.D., UC Santa Barbara, Santa Barbara, California, USA (2017).
[213] R. Harris, J. Johansson, A. J. Berkley, M. W. Johnson, T. Lanting, S. Han, P. Bunyk, E. Ladizinsky, T. Oh, I. Perminov, E. Tolkacheva, S. Uchaikin, E. M. Chapple, C. Enderud, C. Rich, M. Thom, J. Wang, B. Wilson, and G. Rose, Experimental demonstration of a robust and scalable flux qubit, Physical Review B **81**, 134510 (2010), publisher: American Physical Society.
[214] S. Novikov, R. Hinkey, S. Disseler, J. I. Basham, T. Albash, A. Risinger, D. Ferguson, D. A. Lidar, and K. M. Zick, Exploring More-Coherent Quantum Annealing (2018), arXiv:1809.04485.
[215] M. Khezri, J. A. Grover, J. I. Basham, S. M. Disseler, H. Chen, S. Novikov, K. M. Zick, and D. A. Lidar, Anneal-path correction in flux qubits (2021), arXiv:2002.11217.
[216] S. Han, J. Lapointe, and J. E. Lukens, Effect of a two-dimensional potential on the rate of thermally induced escape over the potential barrier, Physical Review B **46**, 6338 (1992), publisher: American Physical Society.
[217] C. Z. Pratt, K. J. Ray, and J. P. Crutchfield, Extracting equations of motion from superconducting circuits, Physical Review Research **7**, 013014 (2025), publisher: American Physical Society.
[218] H. Grabert and U. Weiss, Crossover from Thermal Hopping to Quantum Tunneling, Physical Review Letters **53**, 1787 (1984), publisher: American Physical Society.
[219] P. Hanggi, H. Grabert, G.-L. Ingold, and U. Weiss,

Quantum Theory of Activated Events in Presence of Long-Time Memory, Physical Review Letters **55**, 761 (1985), publisher: American Physical Society.
[220] M. H. Devoret, J. M. Martinis, and J. Clarke, Measurements of Macroscopic Quantum Tunneling out of the Zero-Voltage State of a Current-Biased Josephson Junction, Physical Review Letters **55**, 1908 (1985), publisher: American Physical Society.
[221] S.-X. Li, Y. Yu, Y. Zhang, W. Qiu, S. Han, and Z. Wang, Quantitative Study of Macroscopic Quantum Tunneling in a dc SQUID: A System with Two Degrees of Freedom, Physical Review Letters **89**, 098301 (2002), publisher: American Physical Society.
[222] D. Massarotti, L. Longobardi, L. Galletti, D. Stornaiuolo, D. Montemurro, G. Pepe, G. Rotoli, A. Barone, and F. Tafuri, Escape dynamics in moderately damped Josephson junctions (Review Article), Low Temperature Physics **38**, 263 (2012).
[223] I. Affleck, Quantum-Statistical Metastability, Physical Review Letters **46**, 388 (1981), publisher: American Physical Society.
[224] J. M. Martinis, M. H. Devoret, and J. Clarke, Experimental tests for the quantum behavior of a macroscopic degree of freedom: The phase difference across a Josephson junction, Physical Review B **35**, 4682 (1987), publisher: American Physical Society.
[225] A. Anferov, K.-H. Lee, F. Zhao, J. Simon, and D. I. Schuster, Improved coherence in optically defined niobium trilayer-junction qubits, Physical Review Applied **21**, 024047 (2024), publisher: American Physical Society.
[226] S. Han, J. Lapointe, and J. E. Lukens, Thermal activation in a two-dimensional potential, Physical Review Letters **63**, 1712 (1989), publisher: American Physical Society.
[227] J. Shalf, The future of computing beyond moore's law, Philosophical Transactions of the Royal Society A **378**, 20190061 (2020).
[228] J. Van Damme, S. Massar, R. Acharya, T. Ivanov, D. Perez Lozano, Y. Canvel, M. Demarets, D. Vangoidsenhoven, Y. Hermans, J. G. Lai, A. M. Vadiraj, M. Mongillo, D. Wan, J. De Boeck, A. Potočnik, and K. De Greve, Advanced CMOS manufacturing of superconducting qubits on 300 mm wafers, Nature **634**, 74 (2024), publisher: Nature Publishing Group.
[229] D. Rosenberg, D. Kim, R. Das, D. Yost, S. Gustavsson, D. Hover, P. Krantz, A. Melville, L. Racz, G. O. Samach, S. J. Weber, F. Yan, J. L. Yoder, A. J. Kerman, and W. D. Oliver, 3D integrated superconducting qubits, npj Quantum Information **3**, 1 (2017), publisher: Nature Publishing Group.
[230] D. R. W. Yost, M. E. Schwartz, J. Mallek, D. Rosenberg, C. Stull, J. L. Yoder, G. Calusine, M. Cook, R. Das, A. L. Day, E. B. Golden, D. K. Kim, A. Melville, B. M. Niedzielski, W. Woods, A. J. Kerman, and W. D. Oliver, Solid-state qubits integrated with superconducting through-silicon vias, npj Quantum Information **6**, 1 (2020), publisher: Nature Publishing Group.
[231] J. L. Mallek, D.-R. W. Yost, D. Rosenberg, J. L. Yoder, G. Calusine, M. Cook, R. Das, A. Day, E. Golden, D. K. Kim, J. Knecht, B. M. Niedzielski, M. Schwartz, A. Sevi, C. Stull, W. Woods, A. J. Kerman, and W. D. Oliver, Fabrication of superconducting through-silicon vias (2021), arXiv:2103.08536.
[232] M. Vahidpour, W. O'Brien, J. T. Whyland, J. Angeles, J. Marshall, D. Scarabelli, G. Crossman, K. Yadav, Y. Mohan, C. Bui, V. Rawat, R. Renzas, N. Vodrahalli, A. Bestwick, and C. Rigetti, Superconducting Through-Silicon Vias for Quantum Integrated Circuits (2017), arXiv:1708.02226.
[233] R. Acharya, S. Brebels, A. Grill, J. Verjauw, T. Ivanov, D. P. Lozano, D. Wan, J. Van Damme, A. M. Vadiraj, M. Mongillo, B. Govoreanu, J. Craninckx, I. P. Radu, K. De Greve, G. Gielen, F. Catthoor, and A. Potočnik, Multiplexed superconducting qubit control at millikelvin temperatures with a low-power cryo-CMOS multiplexer, Nature Electronics **6**, 900 (2023), publisher: Nature Publishing Group.
[234] V. Gupta, P. Winkel, N. Thakur, P. v. Vlaanderen, Y. Wang, S. Ganjam, L. Frunzio, and R. J. Schoelkopf, Low loss lumped-element inductors made from granular aluminum (2024), arXiv:2411.12611 [quant-ph].
[235] I. Strandberg, A. M. Eriksson, B. Royer, M. Kervinen, and S. Gasparinetti, Digital Homodyne and Heterodyne Detection for Stationary Bosonic Modes, Physical Review Letters **133**, 063601 (2024), publisher: American Physical Society.
[236] H. Paik, F. Strauch, R. Ramos, A. Berkley, H. Xu, S. Dutta, P. Johnson, A. Dragt, J. Anderson, C. Lobb, *et al.*, Cooper-pair box as a variable capacitor, IEEE transactions on applied superconductivity **15**, 884 (2005).
[237] S. Whitelam, Generative thermodynamic computing, arXiv preprint arXiv:2506.15121 (2025).
[238] Y.-H. Liu and D. Poulin, Neural belief-propagation decoders for quantum error-correcting codes, Physical review letters **122**, 200501 (2019).
[239] J. Old and M. Rispler, Generalized belief propagation algorithms for decoding of surface codes, Quantum **7**, 1037 (2023).
[240] X. Dai, D. Tennant, R. Trappen, A. Martinez, D. Melanson, M. Yurtalan, Y. Tang, S. Novikov, J. Grover, S. Disseler, J. Basham, R. Das, D. Kim, A. Melville, B. Niedzielski, S. Weber, J. Yoder, D. Lidar, and A. Lupascu, Calibration of Flux Crosstalk in Large-Scale Flux-Tunable Superconducting Quantum Circuits, PRX Quantum **2**, 040313 (2021), publisher: American Physical Society.
[241] A. Potts, P. R. Routley, G. J. Parker, J. J. Baumberg, and P. A. J. de Groot, Novel fabrication methods for submicrometer Josephson junction qubits, Journal of Materials Science: Materials in Electronics **12**, 289 (2001).
[242] N. Muthusubramanian, P. Duivestein, C. Zachariadis, M. Finkel, S. L. M. v. d. Meer, H. M. Veen, M. W. Beekman, T. Stavenga, A. Bruno, and L. DiCarlo, Wafer-scale uniformity of Dolan-bridge and bridgeless Manhattan-style Josephson junctions for superconducting quantum processors (2023), arXiv:2304.09111.

Appendix A: Dimensionless Langevin equations

This appendix records a convenient nondimensionalization that connects the standard dimensionful underdamped Langevin equation used in this work (cf. Eq. (8)) to the dimensionless form used in Sec. VI.

Consider a set of coordinates $\tilde{x}_i(\tilde{t})$ with conjugate momenta $\tilde{p}_i(\tilde{t})$ evolving in a potential energy $\tilde{U}(\tilde{x})$ at inverse temperature $\beta = (k_B T)^{-1}$. The dimensionful SDEs may be written as

$$\begin{aligned} d\tilde{x}_i &= \frac{\tilde{p}_i}{m_i} d\tilde{t} \\ d\tilde{p}_i &= -\Big(\partial_{\tilde{x}_i} \tilde{U}(\tilde{x}) + \frac{\tilde{\gamma}_i}{m_i} \tilde{p}_i\Big) d\tilde{t} + \sqrt{\frac{2\tilde{\gamma}_i}{\beta}}\, d\tilde{W}_i(\tilde{t}), \end{aligned} \tag{A1}$$

with independent Wiener processes satisfying $\langle d\tilde{W}_i\, d\tilde{W}_j \rangle = \delta_{ij}\, d\tilde{t}$.

To convert to dimensionless, define the thermal velocity scale $u_i := \sqrt{k_B T / m_i} = \sqrt{1/(\beta m_i)}$ and pick an arbitrary inverse-time scale Γ. We introduce dimensionless variables

$$\begin{aligned} t &:= \Gamma \tilde{t}, \qquad x_i := \Gamma \frac{\tilde{x}_i}{u_i}, \qquad p_i := \frac{\tilde{p}_i}{m_i u_i}, \\ \zeta_i &:= \frac{\tilde{\gamma}_i}{m_i \Gamma}, \qquad U(x) := \beta\, \tilde{U}(\tilde{x}), \qquad W_i(t) := \sqrt{\Gamma}\, \tilde{W}_i(t/\Gamma). \end{aligned} \tag{A2}$$

Equivalently,

$$dW_i(t) = \xi_i(t) dt, \tag{A3}$$

the independent Gaussian white noises satisfy

$$\mathbb{E}[\xi_i(t)] = 0, \qquad \mathbb{E}\left[\xi_i(t) \xi_j(t')\right] = \delta_{ij} \delta(t - t'). \tag{A4}$$

Then the SDEs become

$$\begin{aligned} dx_i &= p_i dt \\ dp_i &= \Big[-\partial_{x_i} U(x) - \zeta_i p_i\Big] dt + \sqrt{2\zeta_i}\, \xi_i(t) dt. \end{aligned} \tag{A5}$$

This is the prototype form used in Sec. VI. The same construction applies componentwise when enlarging the state space to include additional (latent or parameter) variables.

Appendix B: Further subleading-order terms in the kinematics

In the following, we assume that the initial velocity vanishes, which is a sensible assumption for this driving protocol; a more general scenario is considered later. Let us define the following stochastic observable:

$$\Phi_i^{(\theta)}(t) := \frac{2\big(\theta_i(t) - \theta_i^0\big)}{t^2}. \tag{B1}$$

To leading order in time, its expectation value and variance go as

$$\Big\langle \Phi_i^{(\theta)}(t) \Big\rangle = F_i^0 - \frac{t}{3} \zeta_i F_i^0 + \mathcal{O}(t^2), \tag{B2a}$$

$$\mathrm{var}\Big(\Phi_i^{(\theta)}(t)\Big) = \frac{8\zeta_i}{3t} - 2\zeta_i^2 + \mathcal{O}(t). \tag{B2b}$$

We see that as $t \to 0$, $\langle \Phi_i^{(\theta)}(t) \rangle$ tends to F_i^0. Naïvely, this suggests that repeated observations of $\Phi_i^{(\theta)}(t)$ at the earliest times will yield the best estimates of F_i^0, so that the subleading-order term remains small. However, the standard deviation grows as $t^{-1/2}$ at early times, and the number of repeated measurements needed to capture the mean with some fixed uncertainty grows as $N \propto t^{-1}$ for small t. Hence, in practice, a tradeoff must be struck between earlier times at which statistical errors dominate, and later times at which systematic errors may become sizeable.

There are other stochastic observables that can play the same role as $\Phi_i^{(\theta)}(t)$. In the following, we also define $\Phi_i^{(p)}(t)$, which involves measurements of p_i rather than θ_i, and $\Phi_i^{(\theta p)}(t)$, which involves measurements of both, but has the advantage of converging faster to F_i^0 for small t.

The effective equations of motion for θ are Eq. (45). Whereas in the text, we assumed that the different Gaussian white noise channels were uncorrelated, here we allow correlation:

$$\mathbb{E}[\xi_i(t)\xi_j(t')] = q_{ij}\delta(t-t'). \tag{B3}$$

The early-time solution for the means is

$$\mathbb{E}[\theta_i(t)] = \theta_i^0 + tp_i^0 + \frac{t^2}{2}\left(F_i^0 - \zeta_i p_i^0\right) + \frac{t^3}{3!}\left(\sum_j J_{ij}^0 p_j^0 - \zeta_i F_i^0 + \zeta_i^2 p_i^0\right) + \frac{t^4}{4!}\left(\zeta_i^2(F_i^0 - \zeta_i p_i^0) + \sum_j J_{ij}^0\left(F_j^0 - \zeta_j p_j^0 - \zeta_i p_j^0\right)\right) + \mathcal{O}(t^5), \tag{B4a}$$

$$\mathbb{E}[p_i(t)] = p_i^0 + t\left(F_i^0 - \zeta_i p_i^0\right) + \frac{t^2}{2}\left(\sum_j J_{ij}^0 p_j^0 - \zeta_i F_i^0 + \zeta_i^2 p_i^0\right) + \frac{t^3}{3!}\left(\zeta_i^2(F_i^0 - \zeta_i p_i^0) + \sum_j J_{ij}^0\left(F_j^0 - \zeta_j p_j^0 - \zeta_i p_j^0\right)\right) + \mathcal{O}(t^4), \tag{B4b}$$

while for the variances we have

$$\mathrm{var}\big(\theta_i(t)\big) = \frac{2}{3}q_{ii}\zeta_i t^3 - \frac{1}{2}q_{ii}\zeta_i^2 t^4 + \mathcal{O}(t^5), \tag{B5a}$$

$$\mathrm{var}\big(p_i(t)\big) = 2q_{ii}\zeta_i t - 2q_{ii}\zeta_i^2 t^2 + \mathcal{O}(t^3). \tag{B5b}$$

As mentioned in the main text, we consider two additional stochastic observables:

$$\Phi_i^{(\theta)}(t) := \frac{2}{t}\left(\frac{\theta_i(t) - \theta_i^0}{t} - p_i^0\right) + \zeta_i p_i^0, \tag{B6a}$$

$$\Phi_i^{(p)}(t) := \frac{p_i(t) - p_i^0}{t} + \zeta_i p_i^0, \tag{B6b}$$

$$\Phi_i^{(\theta p)}(t) := \frac{p_i(t) - p_i^0 + \zeta_i\big(\theta_i(t) - \theta_i^0\big)}{t}. \tag{B6c}$$

Any of these could potentially be used to measure the force F^0. To sub-leading order, their early-time statistical behavior is given by

$$\mathbb{E}\left[\Phi_i^{(\theta)}(t)\right] = F_i^0 - \frac{t}{3}\left(\zeta_i F_i^0 - \zeta_i^2 p_i^0 - \sum_j J_{ij}^0 p_j^0\right) + \frac{t^2}{12}\left(\zeta_i^2 F_i^0 - \zeta_i^3 p_i^0 + \sum_j J_{ij}^0\Big(F_j^0 - (\zeta_i + \zeta_j)p_j^0\Big)\right) + \mathcal{O}(t^3), \tag{B7a}$$

$$\mathbb{E}\left[\Phi_i^{(p)}(t)\right] = F_i^0 - \frac{t}{2}\left(\zeta_i F_i^0 - \zeta_i^2 p_i^0 - \sum_j J_{ij}^0 p_j^0\right) + \frac{t^2}{3!}\left(\zeta_i^2 F_i^0 - \zeta_i^3 p_i^0 + \sum_j J_{ij}^0\Big(F_j^0 - (\zeta_i + \zeta_j)p_j^0\Big)\right) + \mathcal{O}(t^3), \tag{B7b}$$

$$\mathbb{E}\left[\Phi_i^{(\theta p)}(t)\right] = F_i^0 + \frac{t}{2}\sum_j J_{ij}^0 p_j^0 + \frac{t^2}{3!}\sum_j J_{ij}^0\left(F_j^0 - \zeta_j p_j^0\right) + \mathcal{O}(t^3), \tag{B7c}$$

together with

$$\mathrm{var}\Big(\Phi_i^{(\theta)}(t)\Big) = \frac{8}{3}q_{ii}\zeta_i\frac{1}{t} - 2q_{ii}\zeta_i^2 + \mathcal{O}(t), \tag{B8a}$$

$$\mathrm{var}\Big(\Phi_i^{(p)}(t)\Big) = 2q_{ii}\zeta_i\frac{1}{t} - 2q_{ii}\zeta_i^2 + \mathcal{O}(t), \tag{B8b}$$

$$\mathrm{var}\Big(\Phi_i^{(\theta p)}(t)\Big) = 2q_{ii}\zeta_i\frac{1}{t} + \frac{2}{3}\,t\sum_j J_{ij}^0 q_{ij}\sqrt{\zeta_i\zeta_j} + \mathcal{O}(t^2). \tag{B8c}$$

Here, we see the advantage of using $\Phi^{(\theta p)}(t)$: presuming the initial condition satisfies $p^0 = 0$, the subleading-order contributions to both $\mathbb{E}[\Phi^{(\theta p)}(t)]$ and $\mathrm{var}\big(\Phi^{(\theta p)}(t)\big)$ vanish, making it easier to strike a balance between statistical noise and systematic errors.

Appendix C: HMM derivations

In the following we denote a set of samples at the steps $1:T$ with the superscript $x^{(1:T)}$. As a reminder the probability density of an HMM is given by:

$$\pi(x^{(1:T)}, z^{(1:T)}) = \pi(z^{(1)}) \prod_{t=2}^{T} \pi(z^{(t)}|z^{(t-1)}) \prod_{t=1}^{T} \pi(x^{(t)}|z^{(t)}). \tag{C1}$$

If we use EBMs to parameterize the transition and emission probabilities we get:

$$\pi(x^{(1:T)}, z^{(1:T)}) = \frac{e^{-E_\pi(z^{(1)})}}{Z_\pi} \prod_{t=2}^{T} \frac{e^{-E_\theta(z^{(t)}, z^{(t-1)})}}{Z_\theta(z^{(t-1)})} \prod_{t=1}^{T} \frac{e^{-E_\phi(x^{(t)}, z^{(t)})}}{Z_\phi(z^{(t)})}, \tag{C2}$$

where the partition functions are:

$$\begin{aligned} Z_\theta(z^{(t-1)}) &= \sum_{z^{(t)}} e^{-E_\theta(z^{(t)}, z^{(t-1)})} \\ Z_\phi(z^{(t)}) &= \sum_{x^{(t)}} e^{-E_\phi(x^{(t)}, z^{(t)})} \\ Z_\pi &= \sum_{z^{(1)}} e^{-E_\pi(z^{(1)})}. \end{aligned} \tag{C3}$$

To avoid confusion, we briefly show on the example of $\pi(x^{(t)}|z^{(t)})$ how it can be rewritten in the form of Eq. (C2). Basic probability states that:

$$\pi(x^{(t)}|z^{(t)}) = \frac{\pi(x^{(t)}, z^{(t)})}{\pi(z^{(t)})} = \frac{\pi(x^{(t)}, z^{(t)})}{\sum_{x^{(t)}} \pi(x^{(t)}, z^{(t)})}. \tag{C4}$$

The joint probability is parameterized by an EBM, hence:

$$\pi(x^{(t)}, z^{(t)}) = \frac{e^{-E_\phi(x^{(t)}, z^{(t)})}}{Z_\phi}. \tag{C5}$$

The partition function without argument $Z_\phi = \sum_{x^{(t)}, z^{(t)}} e^{-E_\phi(x^{(t)}, z^{(t)})}$ is traced out over both variables. Dividing $\pi(x^{(t)}, z^{(t)})/\pi(z^{(t)})$ in Eq. (C4) cancels the partition functions Z_ϕ. So we are left with:

$$\pi(x^{(t)}|z^{(t)}) = \frac{e^{-E_\phi(x^{(t)}, z^{(t)})}}{\sum_{x^{(t)}} e^{-E_\phi(x^{(t)}, z^{(t)})}} = \frac{e^{-E_\phi(x^{(t)}, z^{(t)})}}{Z_\phi(z^{(t)})}, \tag{C6}$$

using the shorthand defined in Eq. (C3).

As for any latent variable model, we will have to trace out latent variables to obtain the marginal distribution over our data. The optimization objective is:

$$\log \pi(x^{(1:T)}) = \log \sum_{z^{(1:T)}} \pi(x^{(1:T)}, z^{(1:T)}). \tag{C7}$$

After taking the gradient with respect to any of the parameters, we obtain the well-known form from latent variable EBMs

$$\nabla \log \pi(x^{(1:T)}) = \frac{1}{\pi(x^{(1:T)})} \nabla \sum_{z^{(1:T)}} \frac{e^{-E_\pi(z^{(1)})}}{Z_\pi} \prod_{t=1}^{T} \frac{e^{-E_\phi(x^{(t)}, z^{(t)})}}{Z_\phi(z^{(t)})} \prod_{t=2}^{T} \frac{e^{-E_\theta(z^{(t)}, z^{(t-1)})}}{Z_\theta(z^{(t-1)})}. \tag{C8}$$

The main difference to the latent variable case, is that the partition functions Z_ϕ and Z_θ depend on $z^{(1:T)}$.

First, let's look at the gradient with respect to θ. For simplicity we denote everything in the gradient function that does not depend on θ as π_{rest}.

$$
\begin{aligned}
\nabla_\theta \log \pi(x^{(1:T)}) &= \frac{1}{\pi(x^{(1:T)})} \sum_{z^{(1:T)}} \pi_{\text{rest}} \nabla_\theta \prod_{t=2}^{T} \frac{e^{-E_\theta(z^{(t)}, z^{(t-1)})}}{Z_\theta(z^{(t-1)})} \\
&= \frac{1}{\pi(x^{(1:T)})} \sum_{z^{(1:T)}} \pi_{\text{rest}} \sum_t \nabla_\theta \pi(z^{(t)}|z^{(t-1)}) \prod_{s \neq t} \pi(z_s|z_{s-1}) \\
&= \frac{1}{\pi(x^{(1:T)})} \sum_{z^{(1:T)}} \pi_{\text{rest}} \sum_t \prod_{s \neq t} \pi(z_s|z_{s-1}) \\
&\left[\frac{-\nabla_\theta E_\theta(z^{(t)}, z^{(t-1)}) e^{-E_\theta(z^{(t)}, z^{(t-1)})} Z_\theta(z^{(t-1)}) - e^{-E_\theta(z^{(t)}, z^{(t-1)})} \nabla_\theta Z_\theta(z^{(t-1)})}{Z_\theta(z^{(t-1)})^2} \right] \\
&= \frac{1}{\pi(x^{(1:T)})} \sum_{z^{(1:T)}} \pi_{\text{rest}} \sum_t \prod_{s \neq t} \pi(z_s|z_{s-1}) \\
&\left[-\nabla_\theta E_\theta(z^{(t)}, z^{(t-1)}) \pi(z^{(t)}|z^{(t-1)}) - \pi(z^{(t)}|z^{(t-1)}) \sum_{z'^{(t)}} -\nabla_\theta E_\theta(z'^{(t)}, z^{(t-1)}) \pi(z'^{(t)}|z^{(t-1)}) \right] \\
&= \frac{1}{\pi(x^{(1:T)})} \sum_{z^{(1:T)}} \pi_{\text{rest}} \prod_s \pi(z_s|z_{s-1}) \sum_t \\
&\left[-\nabla_\theta E_\theta(z^{(t)}, z^{(t-1)}) - \sum_{z'^{(t)}} -\nabla_\theta E_\theta(z'^{(t)}, z^{(t-1)}) \pi(z'^{(t)}|z^{(t-1)}) \right] \\
&= - \sum_{z^{(1:T)}} \pi(z^{(1:T)}|x^{(1:T)}) \sum_t \left[\nabla_\theta E_\theta(z^{(t)}, z^{(t-1)}) - \sum_{z'^{(t)}} \nabla_\theta E_\theta(z'^{(t)}, z^{(t-1)}) \pi(z'^{(t)}|z^{(t-1)}) \right]
\end{aligned}
\tag{C9}
$$

Averaged over many data trajectories from a batch $\mathcal{B}$, we obtain:

$$
\sum_{x^{(1:T)} \in \mathcal{B}} \nabla_\theta \log \pi(x^{(1:T)}) = \sum_{x^{(1:T)} \in \mathcal{B}} \mathbb{E}_{z^{(1:T)} \sim \pi(\cdot|x^{(1:T)})} \left[\sum_{t=2}^{T} \left(-\nabla_\theta E_\theta(z^{(t)}, z^{(t-1)}) + \mathbb{E}_{z'^{(t)} \sim \pi_\theta(\cdot|z^{(t-1)})} \left[\nabla_\theta E_\theta(z'^{(t)}, z^{(t-1)}) \right] \right) \right]. \tag{C10}
$$

Now, let's calculate the gradient with respect to ϕ. For simplicity, we rewrite:

$$
\prod_{t=1}^{T} \frac{e^{-E_\phi(x^{(t)}, z^{(t)})}}{Z_\phi(z^{(t)})} = \frac{e^{-\sum_t E_\phi(x^{(t)}, z^{(t)})}}{\sum_{x^{(1:T)}} e^{-\sum_t E_\phi(x^{(t)}, z^{(t)})}} = \frac{e^{-\mathcal{E}_\phi(x^{(1:T)}, z^{(1:T)})}}{\mathcal{Z}_\phi(z^{(1:T)})}, \tag{C11}
$$

where we denote $\mathcal{E}_\phi(x^{(1:T)}, z^{(1:T)}) = \sum_t E_\phi(x^{(t)}, z^{(t)})$ and $\mathcal{Z}_\phi(z^{(1:T)}) = \sum_{x^{(1:T)}} e^{-\sum_t E_\phi(x^{(t)}, z^{(t)})}$. Taking the gradient of this returns:

$$
\nabla_\phi \frac{e^{-\mathcal{E}_\phi(x^{(1:T)}, z^{(1:T)})}}{\mathcal{Z}_\phi(z^{(1:T)})} = \pi(x^{(1:T)}|z^{(1:T)}) \left(-\nabla_\phi \mathcal{E}_\phi - \frac{e^{-\mathcal{E}_\phi}}{\mathcal{Z}_\phi} \sum_{x^{(1:T)}} -\nabla_\phi \mathcal{E}_\phi \pi(x^{(1:T)}|z^{(1:T)}) \right), \tag{C12}
$$

where we used that

$$
\pi(x^{(1:T)}|z^{(1:T)}) = \frac{e^{-\mathcal{E}_\phi(x^{(1:T)}, z^{(1:T)})}}{\mathcal{Z}_\phi(z^{(1:T)})}. \tag{C13}
$$

Also, if we now denote

$$
\pi_{\text{rest}} = \frac{1}{\pi(x^{(1:T)})} \sum_{z^{(1:T)}} \frac{e^{-E_\pi(z^{(1)})}}{Z_\pi} \prod_{t=2}^{T} \frac{e^{-E_\theta(z^{(t)}, z^{(t-1)})}}{Z_\theta(z^{(t-1)})}, \tag{C14}
$$

we have:

$$\pi_{\text{rest}}\pi(x^{(1:T)}|z^{(1:T)}) = \sum_{z^{(1:T)}} \frac{\pi(z^{(1)})\prod_{t=2}^{T}\pi(z^{(t)}|z^{(t-1)})\pi(x^{(1:T)}|z^{(1:T)})}{\pi(x^{(1:T)})}$$
$$= \sum_{z^{(1:T)}} \frac{\pi(x^{(1:T)}, z^{(1:T)})}{\pi(x^{(1:T)})} = \sum_{z^{(1:T)}} \pi(z^{(1:T)}|x^{(1:T)}). \tag{C15}$$

Using this we get:

$$\nabla_\phi \log \pi(x^{(1:T)}) = -\sum_{z^{(1:T)}} \pi(z^{(1:T)}|x^{(1:T)})\big(\nabla_\phi \mathcal{E}_\phi - \sum_{x^{(1:T)}} \nabla_\phi \mathcal{E}_\phi \pi(x^{(1:T)}|z^{(1:T)})\big). \tag{C16}$$

The complete loss is an average over a batch $\mathcal{B}$ of trajectories $x^{(1:T)}$ that we sample from a data distribution, hence the complete gradient reads:

$$\sum_{x^{(1:T)}\in\mathcal{B}} \nabla_\phi \log \pi(x^{(1:T)}) = -\sum_{x^{(1:T)}\in\mathcal{B}} \sum_{z^{(1:T)}} \pi(z^{(1:T)}|x^{(1:T)})\big(\nabla_\phi \mathcal{E}_\phi - \sum_{x'^{(1:T)}} \nabla_\phi \mathcal{E}_\phi \pi(x'^{(1:T)}|z^{(1:T)})\big)$$
$$= -\sum_{x^{(1:T)}\in\mathcal{B}} \left\{ \mathbb{E}_{z^{(1:T)}\sim\pi(\cdot|x^{(1:T)})}\left[\nabla_\phi \mathcal{E}_\phi\right] - \mathbb{E}_{\substack{z^{(1:T)}\sim\pi(\cdot|x^{(1:T)}) \\ x'^{(1:T)}\sim\pi(\cdot|z^{(1:T)})}}\left[\nabla_\phi \mathcal{E}_\phi\right] \right\}. \tag{C17}$$

Appendix D: Crosstalk and scale calibration

Two effects complicate our calibration goals: First, there is significant crosstalk between the two on-chip flux lines and the thermodynamic neuron loops, meaning the flux induced by current through the barrier (tilt) line partially couples into the tilt (barrier) loop. Second, there is almost always some residual flux trapped in the loop causing an effective flux offset, and therefore moving the zero flux point to some non-zero line voltage.

We can compensate for these two effects by finding the affine relationship between the control voltages and the true flux coordinates [212, 214, 215, 240].

$$\begin{bmatrix}\phi_{\text{bar}} \\ \phi_{\text{tilt}}\end{bmatrix} = \begin{bmatrix}m_{11} & m_{12} \\ m_{21} & m_{22}\end{bmatrix}\begin{bmatrix}V_{\text{bar}} \\ V_{\text{tilt}}\end{bmatrix} + \begin{bmatrix}\phi^0_{\text{bar}} \\ \phi^0_{\text{tilt}}\end{bmatrix} \tag{D1}$$

where V_{bar} (V_{tilt}) is the control voltage applied to the barrier (tilt) line, and ϕ^0_{bar} (ϕ^0_{tilt}) is the offset flux in the barrier (tilt) loop that is set during the normal-superconducting transition. The parameters m_{ij} can be found by using the periodicity in ϕ_{bar} and ϕ_{tilt} of the readout resonator frequency. It is thus possible to measure the resonator frequency for a wide range of control voltages and correct for the crosstalk. The offset does not need to be compensated for, we simply need to know it. In experiments, we immediately compensate for crosstalk in the applied voltages and we rescale the compensated voltages to flux in a post-processing step.

To facilitate the rescaling, we implement a simple method to measure and automatically detect the edges of the triangular features that appear in the S_{21} map (Figure 14) via peak finding. We then fit the two edges of each triangle to lines and determine the location where they intercept, i.e., at the apex of the triangle. Because this triangle feature is repeated at regular interval on the ϕ_{bar} and ϕ_{tilt} flux map, given enough apex points, it is possible to linearly fit for the volt-to-flux scale and offset.

The device displays an unexpected non-affine relationship between the flux and voltage in the temperature region between 60 and 100 mK. We suspect that this is due to contamination of the base layer film, see Section J. Outside of this temperature region, we still observe some mild drift in the crosstalk and scale. We thus re-measure the triangle scale at each temperature point to ensure a proper calibration of the barrier flux.

Appendix E: Relaxation time experiment

Figure 15 presents the procedure for the main relaxation experiment. It is similar to the initialization/readout experiment, but with an added wait time prior to fully raising the barrier for readout. We use the dc offset voltage functionality of our arbitrary waveform generator (AWG) to set the dc flux at the $(\phi_{\text{bar}}, \phi_{\text{tilt}})$ point used during the relaxation time, denoted by the star in Figure 15 (b), and use pulses to initialize the population and readout. After initialization (step 1), but before going to the relaxation point, we raise the barrier, center the tilt flux, and wait

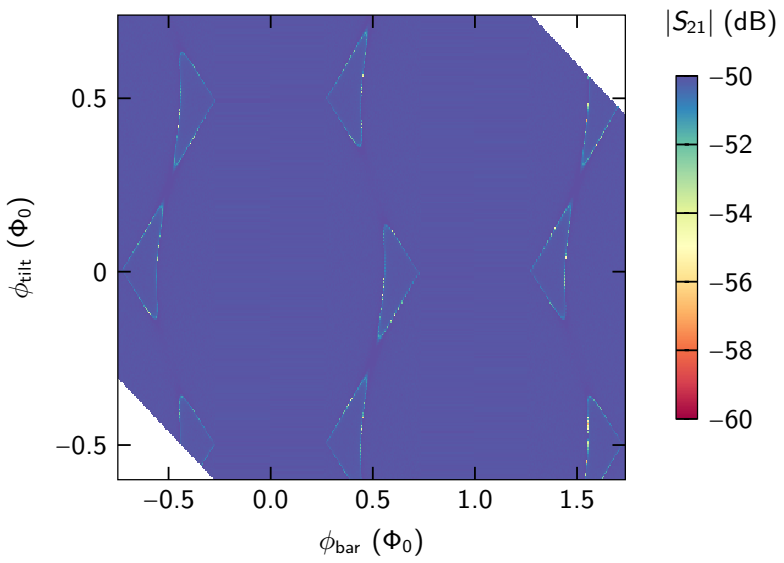


Figure 14. Heatmap of $|S_{21}|$ data versus ϕ_{bar} and ϕ_{tilt} for readout frequency $f_{\text{probe}} = 11.790\,\text{GHz}$. The mapping between control voltage and flux was done using Eq. (D1). The data on the corners is missing due to voltage amplitude limits of the AWG.

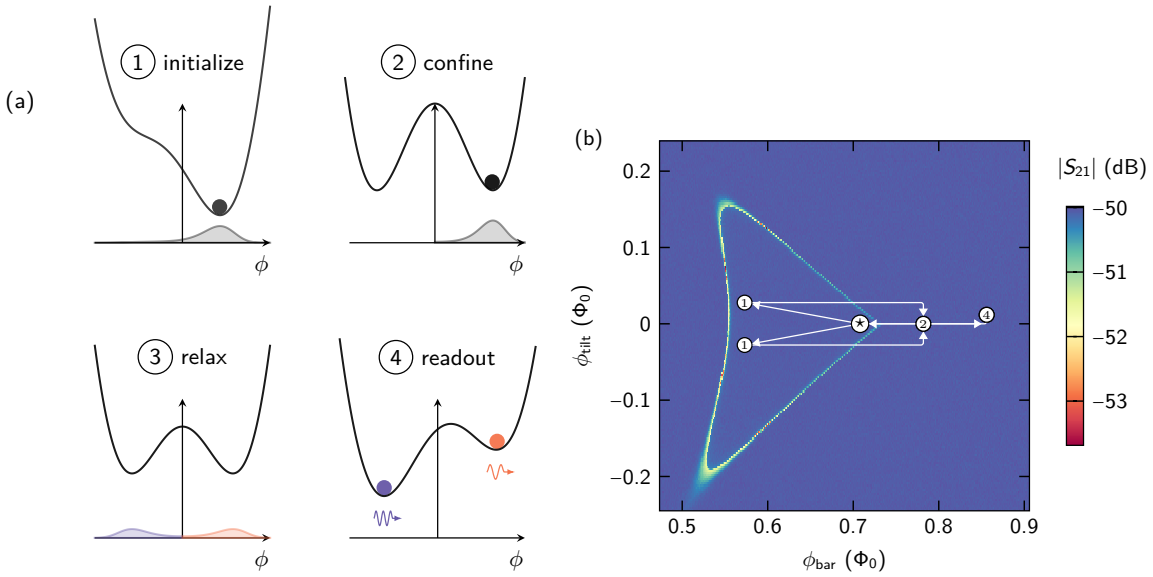


Figure 15. (a) Experimental sequence of the relaxation experiment. The system is initialized in either the left or right well by tilting the potential and then raising the barrier. Next, the system is held at a bias point $(\phi_{\text{bar}}, \phi_{\text{tilt}})$ and allowed to relax for a variable time τ. The system is then brought to the readout point by raising the barrier and then introducing asymmetry through the tilt control. Lastly, the transmission line is probed to detect the thermodynamic neuron position into the left or right well. (b) Control parameter trajectory in flux-space overlaid on $|S_{21}|$ transmission data. The star designates both the starting point of the sequence and the relaxation point.

1.75–3 µs, (step 2). This ensures that the population is fully confined in one well. Then we only need to lower the barrier to the relaxation point and wait some time t (step 3). Finally we raise the barrier for readout (step 4).

We measure relaxation curves for a range of barrier settings, but always in the symmetric double-well configuration. To calibrate the tilt, we measure the population P at a long wait time $t = 1\,\text{ms}$ over a range of tilt flux settings and choose the value resulting in a 50-50 population. This calibration is done for every barrier flux point. We also ensure that the measured barrier range is as similar as possible for the different temperatures by choosing barrier voltages relative to the triangle apex identified for the scale calibration. We choose a voltage range between −40 and −10 mV

at the AWG, corresponding to ϕ_{bar} between 0.680 and 0.715 Φ_0. To properly characterize the exponential decay of the population, we pick wait times logarithmically distributed between 40 ns and 1 ms. Finally, we measure these relaxation curves over temperatures ranging from 9 mK, the base temperature, to 250 mK. Due to the control issue appearing between 60 and 100 mK forcing us to recalibrate the crosstalk compensation, we acquire two separate datasets, one for 9 to 60 mK, and another for 105 to 250 mK.

Appendix F: Setup and sample

The sample is a chip comprising three independent thermodynamic neurons that are fabricated with evaporated aluminum on a silicon wafer. Each thermodynamic neuron is coupled to a readout resonator. Additionally, the device hosts four coplanar waveguide resonators for design and fabrication verification. The three readout resonators are designed with frequencies between 11.8 and 12 GHz, spaced 100 MHz apart. Analogously, the four test resonators are designed between 11.4 and 11.7 GHz. We use the thermodynamic neuron designed with the 12 GHz resonator for the experiments presented in this work, though because of the unaccounted coupling to the thermodynamic neuron, the effective frequency ends up near 11.8 GHz. Flux pulses to control the dc-SQUID and the tilt of a thermodynamic neuron are made via two independent coplanar waveguide (CPW) lines terminating in a short. Measurement is performed by means of a high-power readout scheme using a resonator inductively coupled to the thermodynamic neuron, see Section I for more details. Images of an identically-fabricated chip can be seen in Figure 16. The package-to-chip connections are made with 25 µm aluminum wire-bonds. The chip is wire-bonded to a commercially available package, the `QCage.24`, and shielded with a combination mu-metal and aluminum can made by QDevil.

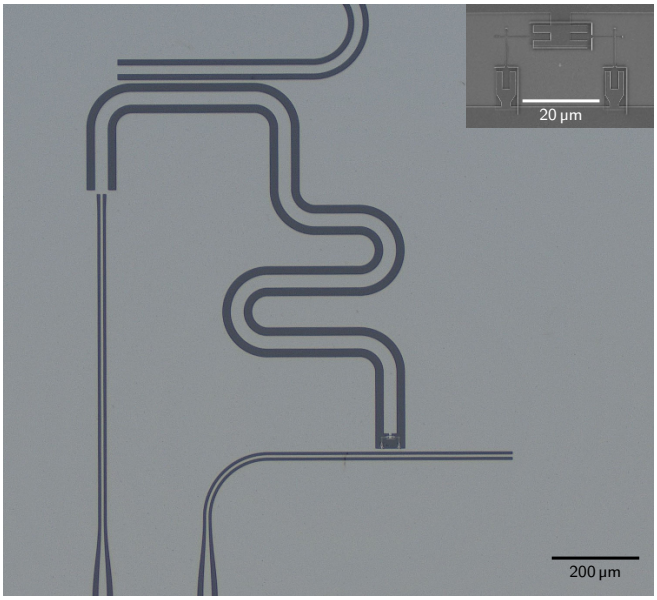


Figure 16. Optical image of the thermodynamic neuron, with the tilt flux line on the left at the shorted end, readout resonator above (only the coupler is visible), and SQUID flux bias lines below. The $\lambda/4$ CPW segment of the thermodynamic neuron effectively functions as a parallel LC-circuit, while the SQUID acts as a tunable nonlinear inductor. (Inset) Scanning electron microscope (SEM) image of the dc-SQUID, which is located at the open end of the CPW segment of the thermodynamic neuron.

The setup schematics of the dilution refrigerator, including instrument and wiring details can be found in Figure 20. The two lines used to bias the thermodynamic neuron and the readout input line are filtered with Eccosorb-based coaxial filters made by Quantum Microwave. The readout line output is filtered with a low-loss IR filter made by Sweden Quantum and a Low Noise Factory triple-junction isolator/circulator. We use the Keysight Quantum Control System to send microwave pulses for readout and DC pulses for flux control.

Appendix G: Device fabrication

The device is fabricated using an undoped high-resistivity (>20 kΩ·cm) silicon substrate using a two-layer process. Firstly, alignment markers are patterned using masked lithography and etched into the silicon with a Bosch etch using an `STS` plasma etcher. Following the etch and a solvent clean, the silicon wafer is cleaned using piranha solution, followed by an HF-dip to remove native oxides, after which it is transferred into an `Angstrom` electron-beam (e-beam) evaporator system to deposit the 100 nm aluminum base layer that makes up the ground plane and large-scale features. These features are patterned with a 100 kV e-beam lithography system `EBPG5200` into `ZEP520A` e-beam resist and developed with `ZED-N50`. Consequently, the pattern is plasma-etched with an $Ar/Cl_2/BCl_3$ chemistry in an `STS` etcher. After stripping the remaining resist with 1-methyl-2-pyrrolidinone (NMP), the wafer is diced into 10×10 mm^2 chips. The Manhattan-style Josephson junctions [241, 242] are patterned with e-beam lithography in a PMMA/MMA (780 nm/430 nm) bilayer after dicing. Development was done cold with an IPA:H_2O (9:1) solution. The Al/AlOx/Al junctions are then deposited at a 45° angle in a `Plassys` e-beam evaporator, using a double-angle technique. This evaporation was performed directly from the hearth, without the use of a crucible liner. We perform liftoff using NMP and sonication in acetone and isopropyl alcohol. Lastly, the sample is wirebonded to a printed circuit board and packaged.

Appendix H: Hamiltonian fit

To determine the thermodynamic neuron parameters, we perform a Hamiltonian fit to measured readout resonator frequency data. We use the *1D no caps* model presented in Ref. [212]. This model, based on a Born-Oppenheimer approximation, accounts for the inline inductances and capacitances in the dc-SQUID, along with potential Josephson junction asymmetry. It is given by

$$\begin{aligned} H_{\mathrm{BO}} = \frac{q^2}{2C_{\mathrm{eff}}} + \frac{1}{2L}(\phi-\phi_{\mathrm{tilt}})^2 \\ - E_{J_1}\cos_{\beta_1}(\phi+\tfrac{1}{2}\phi_{\mathrm{bar}}) - E_{J_2}\cos_{\beta_2}(\phi-\tfrac{1}{2}\phi_{\mathrm{bar}}), \end{aligned} \tag{H1}$$

where $C_{\mathrm{eff}} = C + C_1 + C_2$ incorporates the inline capacitances of the two junctions that are designated by the subscript $i \in \{1,2\}$, L is the inductance of the main loop, E_{J_1} and E_{J_2} are the Josephson energies, and

$$\cos_{\beta_i}(\phi) = 1 + \sum_{k\in\mathbb{N}} \frac{2J_k(k\beta_i)}{k^2\beta_i}\left(\cos\left(\tfrac{2\pi}{\Phi_0}k\phi\right)-1\right) \tag{H2}$$

where $\beta_i \equiv E_{J_i}/E_{L_i}$ is the fraction between junction i's Josephson energy and its inductive energy $E_{L_i} \equiv \frac{\Phi_0^2}{8\pi^2 L_i}$ associated with the inline inductance L_i, and $J_k(x)$ is the kth Bessel function. The junction asymmetry is characterized by $\chi = (I_{c1}-I_{c2})/(I_{c1}+I_{c2})$, with $I_c = I_{c1}+I_{c2}$.

We add the Hamiltonian of the readout resonator, along with an inductive coupling between the thermodynamic neuron and resonator. The resonator Hamiltonian is given by

$$H_{\mathrm{r}} = \frac{q_{\mathrm{r}}^2}{2C_{\mathrm{r}}} + \frac{\phi_{\mathrm{r}}^2}{2L_{\mathrm{r}}}, \tag{H3}$$

where q_{r} and ϕ_{r} are the conjugate charge and flux of the resonator, C_{r} and L_{r} are the corresponding capacitance and inductance. The coupling term is written as

$$H_{\mathrm{c}} = -\frac{M}{\alpha L_{\mathrm{r}} L}\phi_{\mathrm{r}}\phi \tag{H4}$$

where M is the mutual inductance between the resonator and the main loop of the thermodynamic neuron, and $\alpha = 1 - M^2/(L_{\mathrm{r}}L)$.

We calculate the theoretical resonator frequency in the quantum ground state by diagonalizing the ϕ-space discretized Hamiltonian for all measured combinations of ϕ_{bar} and ϕ_{tilt}, and fit with the Levenberg–Marquardt algorithm. Due to the large number of free parameters (11), the lack of direct measurement of the thermodynamic neuron frequency, and potential residual crosstalk or control imperfections, the fitting results encompass a very large region of the L, C, and I_c parameter space, over which equally-good convergence is achieved. To bypass this problem, we choose to fix the capacitance to a realistic value determined via simulations. With this added constraint, we achieve a robust convergence over the 10 other parameters.

The parameters of the device used in the experiment are specified in Table I. The resonator frequencies measured at 10 mK are shown in Figure 17 together with the frequencies based on the Hamiltonian fit.

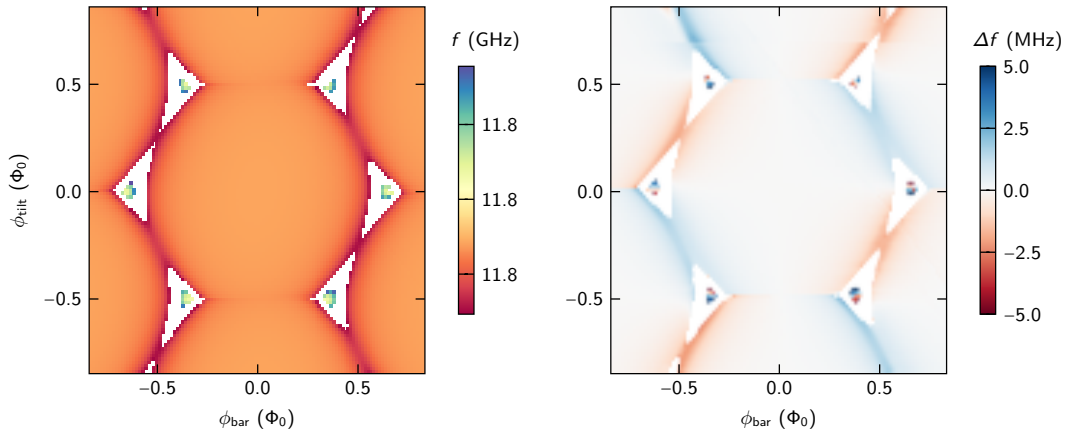


Figure 17. Readout resonator frequency as function of ϕ_{bar} and ϕ_{tilt}. (left) Extracted resonator frequency from S_{21} transmission measurements. (right) Residuals between the experimental data and the Hamiltonian fit with the parameters listed in Table I.

Appendix I: Readout

Readout is performed by utilizing the dispersive shift allowing the system to be projected in the left or right well, see Ref. [212]. To this end, the thermodynamic neuron is inductively coupled to a coplanar LC-oscillator, the readout resonator. Occupancy in either well of the thermodynamic neuron will induce a shift in the readout resonator frequency. These two dispersive shifts differ when an asymmetry between the two wells is introduced through the tilt control. The readout resonator is capacitively coupled to the transmission line from which it can be probed. The thermodynamic neuron state can be inferred using homodyne detection of the output field at an appropriate readout frequency f_{probe}. We use a readout pulse with a flattop Gaussian shape of 5 µs, with a 200 ns risetime. After down-conversion and time averaging, the measured in-phase and quadrature (IQ) points are used to discriminate the two states. For the experiments presented in the main text, we calibrate the readout by preparing the flux particle in the left (right) well, pulse to an asymmetric flux coordinate with sufficiently deep wells and finally probe the transmission line. We find a triplet $(V_{\text{bar}}, V_{\text{tilt}}, f_{\text{r}})$ of control voltages and readout frequency that optimizes the visibility, defined as $\mathcal{V} = P_{L|L} + P_{R|R} - 1$, with $P_{L|L}$ being the proportion of measurement shots classified as L when the thermodynamic neuron was prepared in the L state and equivalently for $P_{R|R}$. When characterizing readout at optimized settings over 1000 single-shot measurements, we find the proportion in either target well with a 100.0% visibility. Figure 18 shows the visibility against the barrier and tilt voltage, where the readout frequency is selected per barrier voltage.

Appendix J: Non-affine flux-voltage relationship

In the temperature region between 60 mK and 100 mK we observed a non-affine relationship between the applied control voltages and the flux coordinates, see Figure 19. This inhibits performing experiments in this temperature

Table I. Device parameters for the thermodynamic neuron used in the experiment. M is the mutual inductance between the thermodynamic neuron and its readout resonator. The Hamiltonian parameters are determined via a fit of the resonator frequency over a large range of bias voltages. The uncertainty of the C and L thermodynamic neuron parameters should be considered fairly large since we only fit the resonator frequency spectrum, and not that of the thermodynamic neuron. Note that the resonator frequency shown is that of the bare resonator.

Parameter	Value
C (fF)	120
L (pH)	750
$L_{1,2}$ (pH)	50
I_c (µA)	0.997
χ	0.0104
f_{res} (GHz)	11.804856
M (pH)	24.7

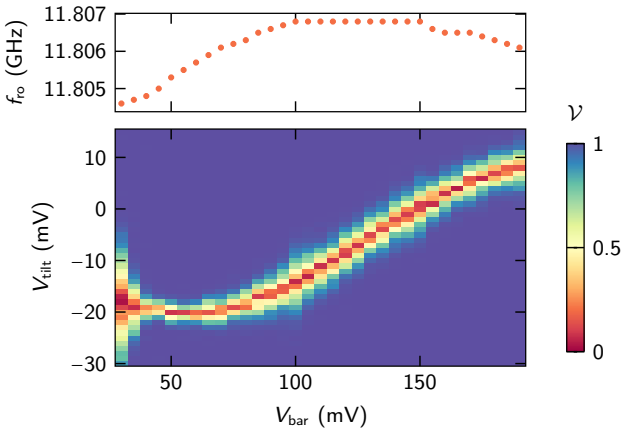


Figure 18. (bottom) Visibility of the readout procedure at 120 mK for variable flux and tilt control voltages where the readout frequency is chosen per barrier voltage (top). At the tilt-symmetric curve the visibility reduces to zero as the dispersive shifts of both wells become nearly identical. This curve is not a straight line both due to nonlinear crosstalk caused by junction asymmetry [212, 215].

region as we do not have a faithful mapping between control voltages and fluxes. Below and beyond this range the voltage-flux relationship was found to be adequately characterized by Eq. (D1), though still with a small drift that must be corrected for.

We suspect that this unexpected behavior could be due to Boron contamination that was introduced from an intermetallic crucible in the evaporator during the base layer deposition. The Josephson junction deposition was performed in a different evaporator from the hearth without the use of a crucible liner, see Section G.

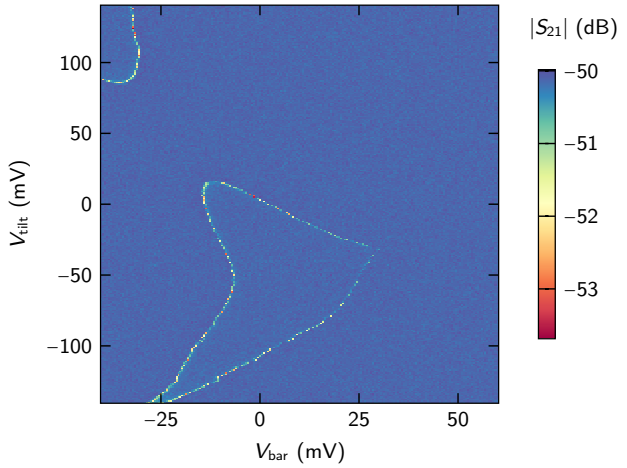


Figure 19. Non-affine voltage-flux relationship at 80 mK.

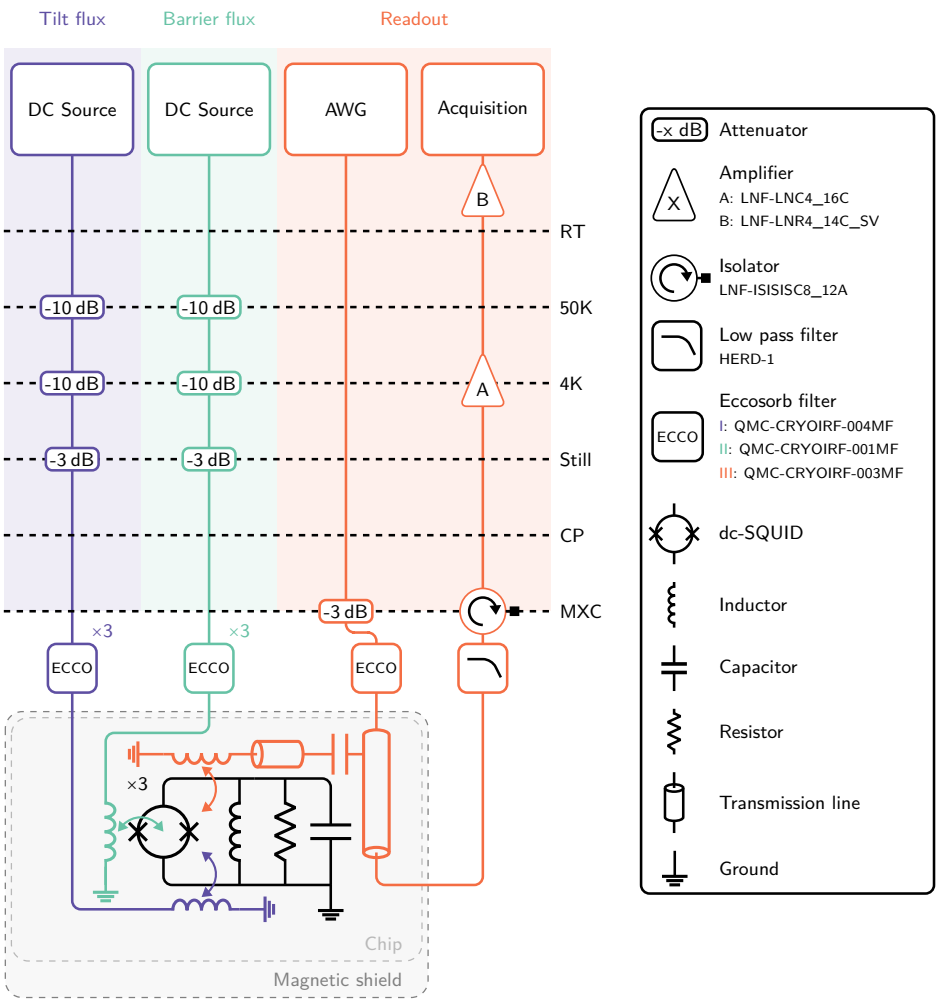


Figure 20. Experimental setup diagram of the Bluefors LD400 dilution refrigerator and the wiring. The refrigerator has a room temperature (RT), 50K, 4K, still and mixing chamber (MXC) flange. The cold plate (CP) is situated between the still and mixing chamber flanges. Placement on a flange means that the component is thermally anchored to it. The triple-junction isolator is mounted on top of the MXC flange and includes a mu-metal shield. The cold amplifier is mounted to the bottom of the 4K flange. The sample chip is packaged and placed inside a magnetic shield that is mounted underneath the MXC flange. The chip hosts three superconducting thermodynamic neurons, each with dedicated tilt and barrier flux lines that are controlled by DC sources at room temperature. Mutual inductances are indicated by an arc between an inductor and the relevant loop. Readout of a thermodynamic neuron is performed through its inductively coupled readout resonator that is capacitively coupled to a single shared transmission line. The readout pulses are supplied by an arbitrary waveform generator (AWG) that has an integrated acquisition system for down-conversion and digitization of the transmitted signal. Four coplanar waveguide verification resonators that are also capacitively coupled to the transmission line are omitted from the diagram.